Digital Notes

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and
G.O.Ms.No.14, Higher Education (UE) Department)

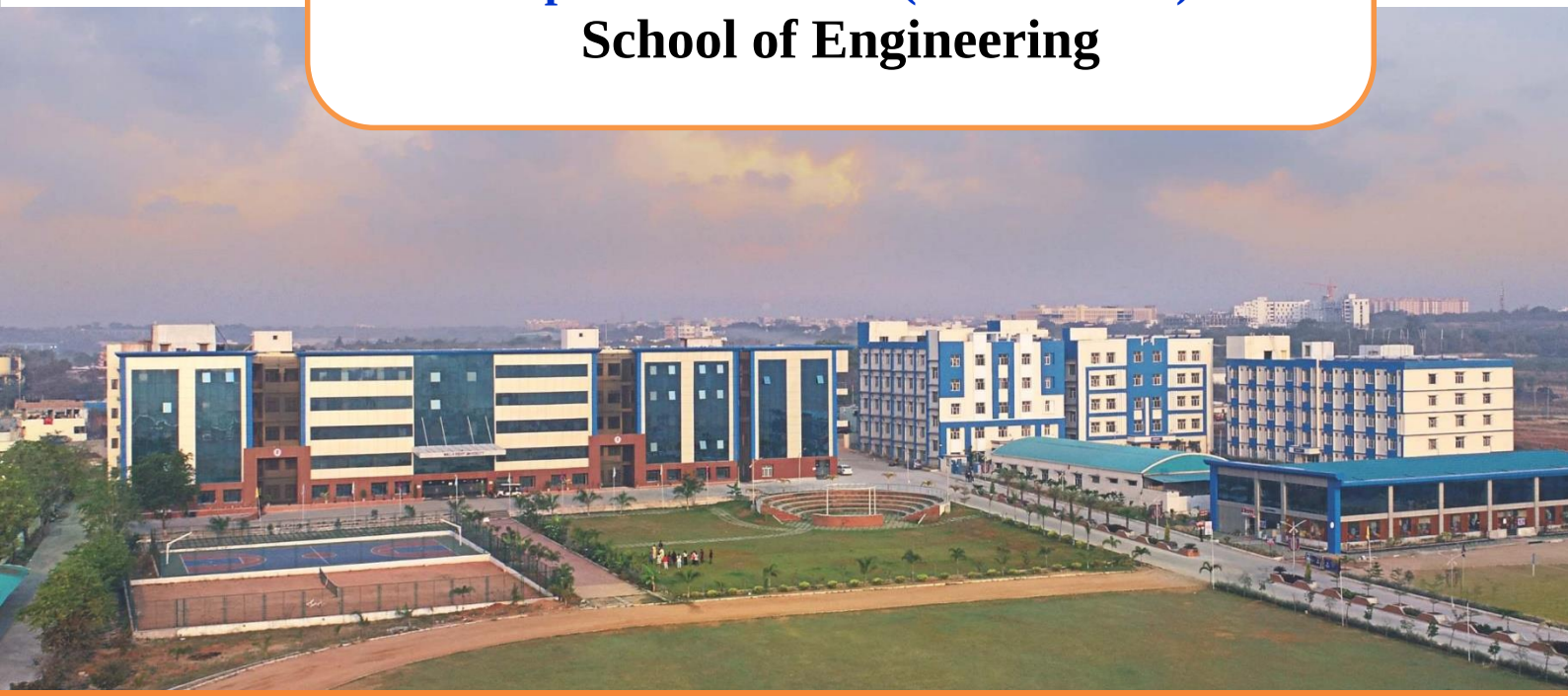
Distributed Databases

III B.Tech II Semester

(2024-25)

Department of CSE (Data Science)

School of Engineering



UNIT-I: Introduction to Distributed Database System: Distributed Data Processing, Distributed Database System, Promises of DDBSs, Problem areas Distributed DBMS, Architecture: Architectural Models for Distributed DBMS, Distributed Database Design: Alternative Design Strategies, Distribution Design issues, Fragmentation, Allocation

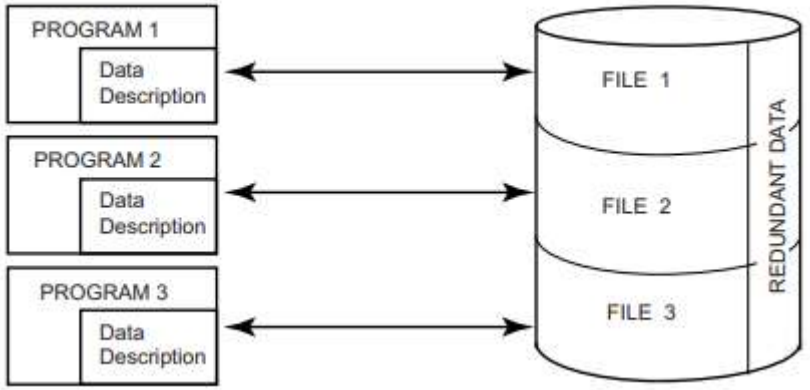
Introduction

Distributed database system (DDBS) technology is the union of what appear to be two diametrically opposed approaches to data processing:

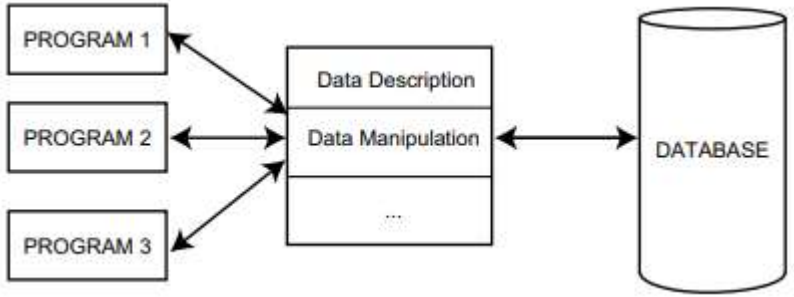
1. **Database system** and
2. **Computer network** technologies.

Data independence, whereby the application programs are immune to changes in the logical or physical organization of the data, and vice versa.

Traditional File System

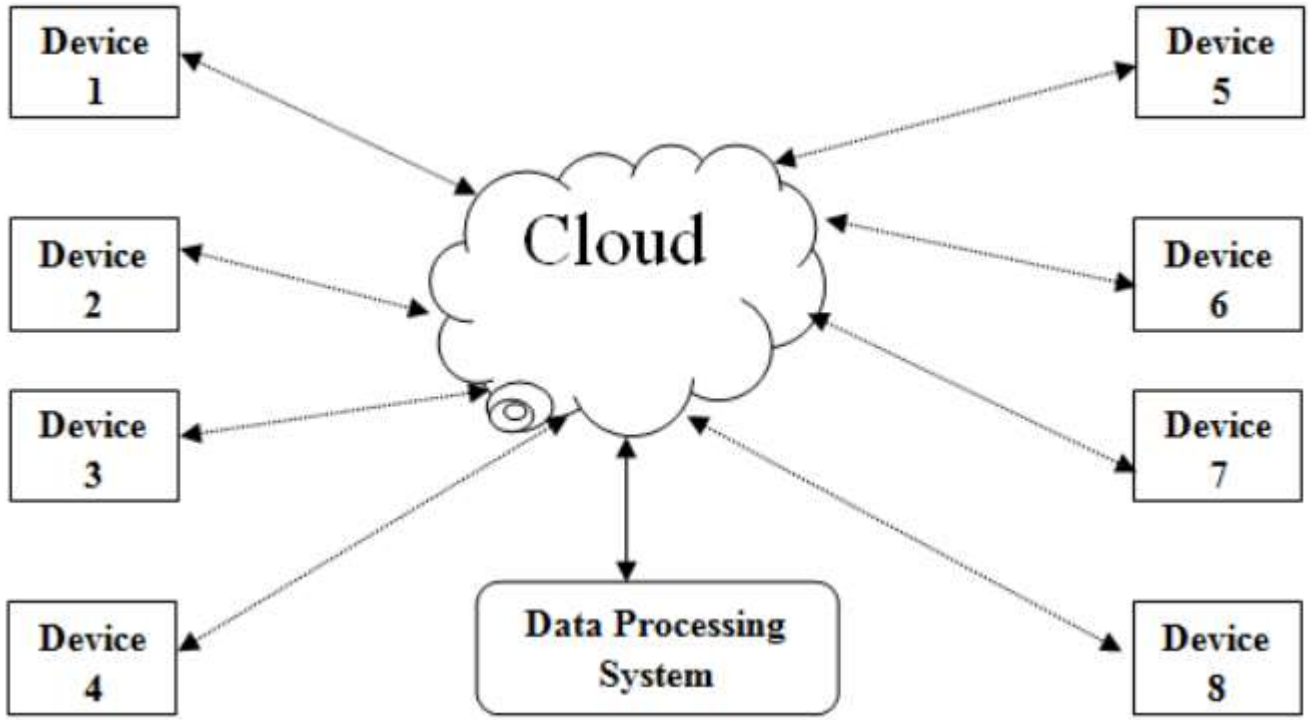


Database Processing



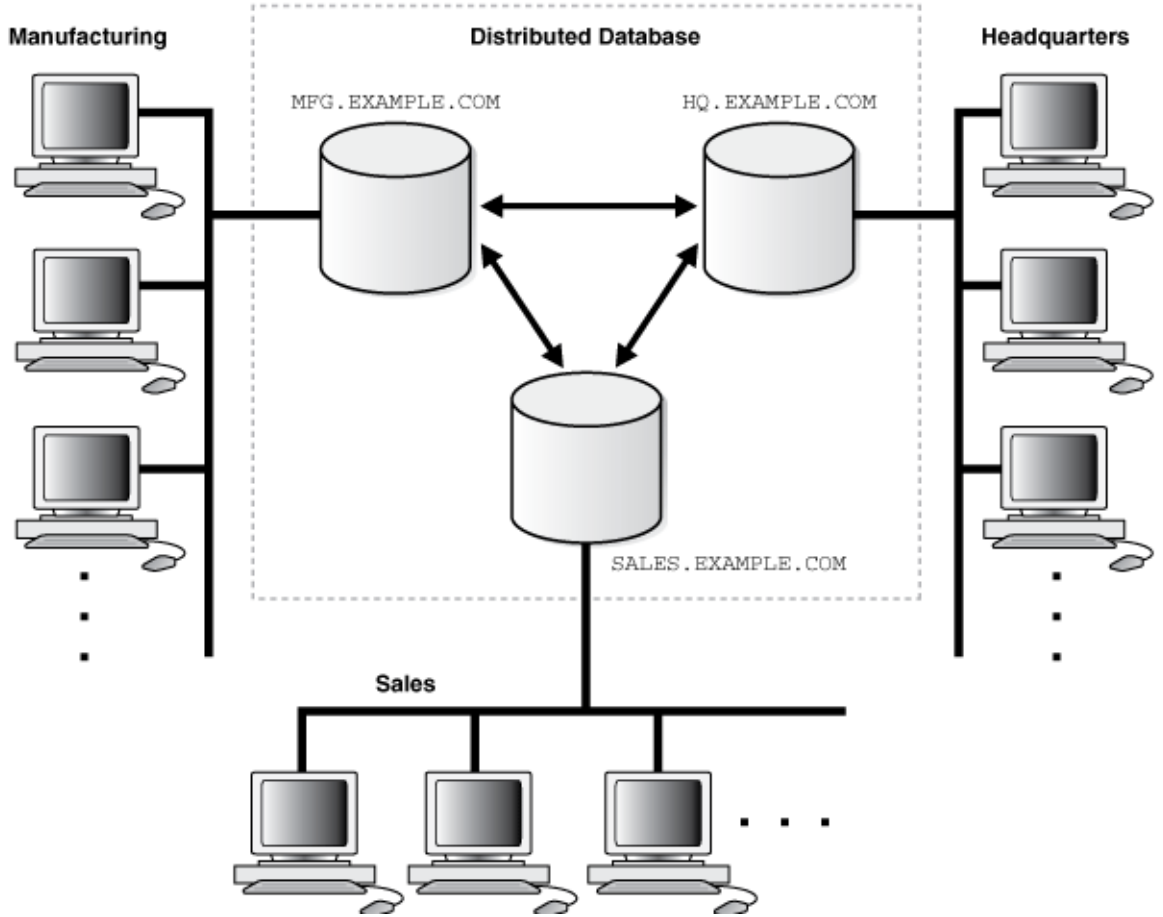
Distributed Data Processing

- Distributed data processing refers to the approach of handling and analyzing data across **multiple interconnected devices** or nodes.
- In **contrast** to **centralized data processing**, where all data operations occur on a single, powerful system.



Distributed Data Processing

- Distributed processing decentralizes these tasks across a network of computers. This method leverages the collective computing power **of interconnected devices, enabling parallel processing and faster data analysis.**



Benefits of Distributed Data Processing

Scalability - As data volumes grow, organizations can expand their **processing capabilities** by adding more nodes to the network. This scalability ensures that the system can handle increasing workloads without a significant drop in performance, providing a flexible and adaptive solution to the challenges posed by big data.

Fault Tolerance-In a distributed environment, if one node fails, the remaining nodes can continue processing data, reducing the risk of a complete system failure. This resilience is crucial for maintaining uninterrupted data operations in mission-critical applications.

Performance-By breaking down complex tasks into smaller subtasks distributed across nodes, the system can process data more quickly and efficiently. This results in reduced processing times and improved overall performance, enabling organizations to derive insights from data in a timely manner.

Efficient Handling of Large Volumes of Data-This approach not only accelerates data processing through parallelism but also optimizes use of resources. Each node focuses on a specific subset of the data, ensuring that the system operates efficiently and effectively. The ability to efficiently **handle large volumes of data positions organizations to extract meaningful insights, identify patterns, and make informed decisions**.

Challenges and Considerations of Distributed Data Processing

Data Consistency- In a decentralized environment, where data is processed simultaneously across multiple nodes, ensuring that all nodes have access to the most recent and accurate data becomes complex.

Network Latency- As nodes communicate and share data, the time it takes for information to traverse the network can impact the overall performance of the system.

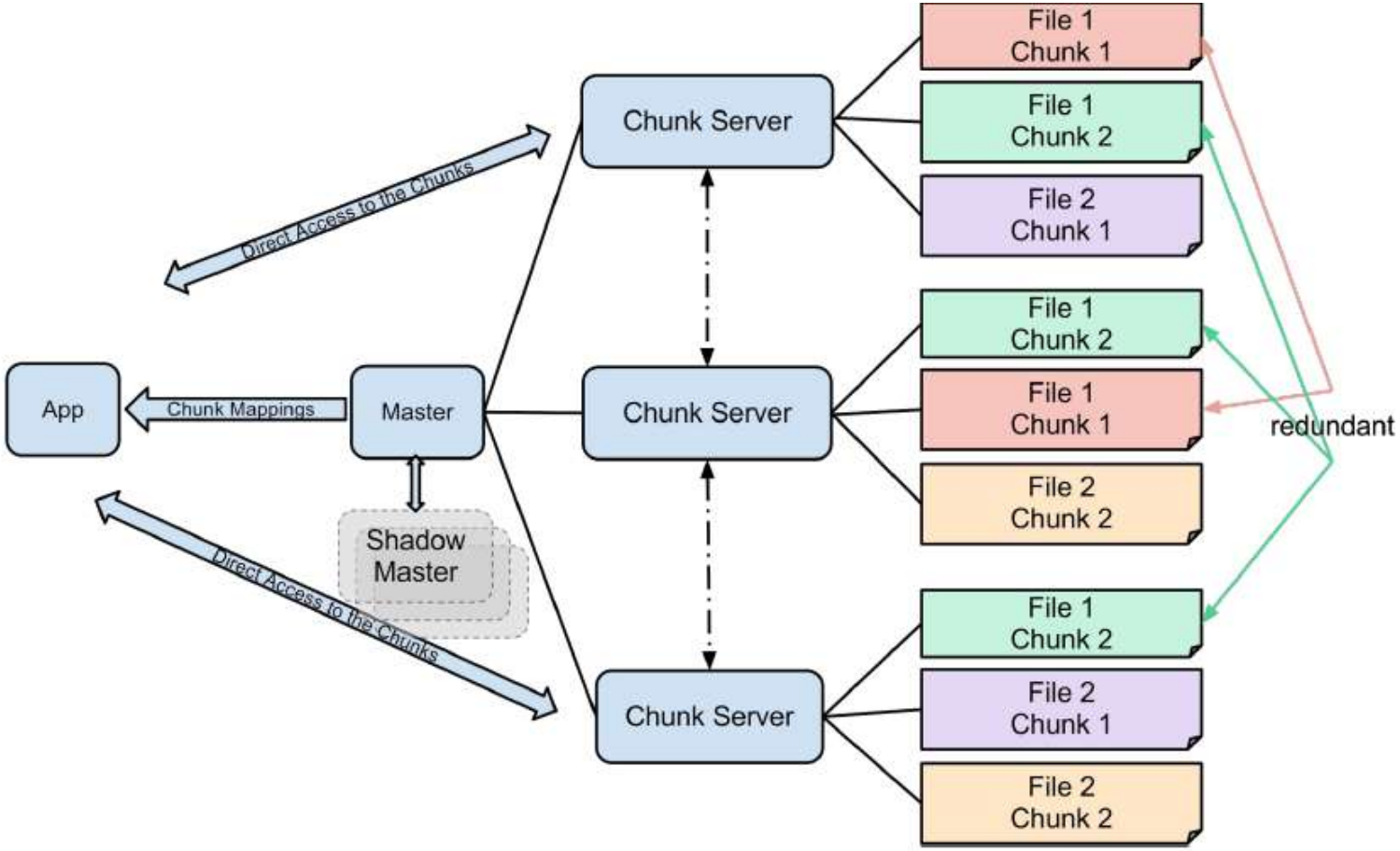
System Complexity- Coordinating tasks, managing nodes, and ensuring fault tolerance in a decentralized environment requires a nuanced understanding of system intricacies.

Ensuring Data Security- With data distributed across nodes, organizations must implement robust measures to protect sensitive information from potential threats and unauthorized access.

Distributed Data Processing in Action: Real-world Examples

- **Finance**: Fraud Detection and Risk Management
- **E-commerce**: Personalized Recommendations
- **Healthcare**: Genome Sequencing and Drug Discovery
- **Telecommunications**: Network Monitoring and Optimization

Big Data – Distributed Data Processing



What is a Distributed Database System?

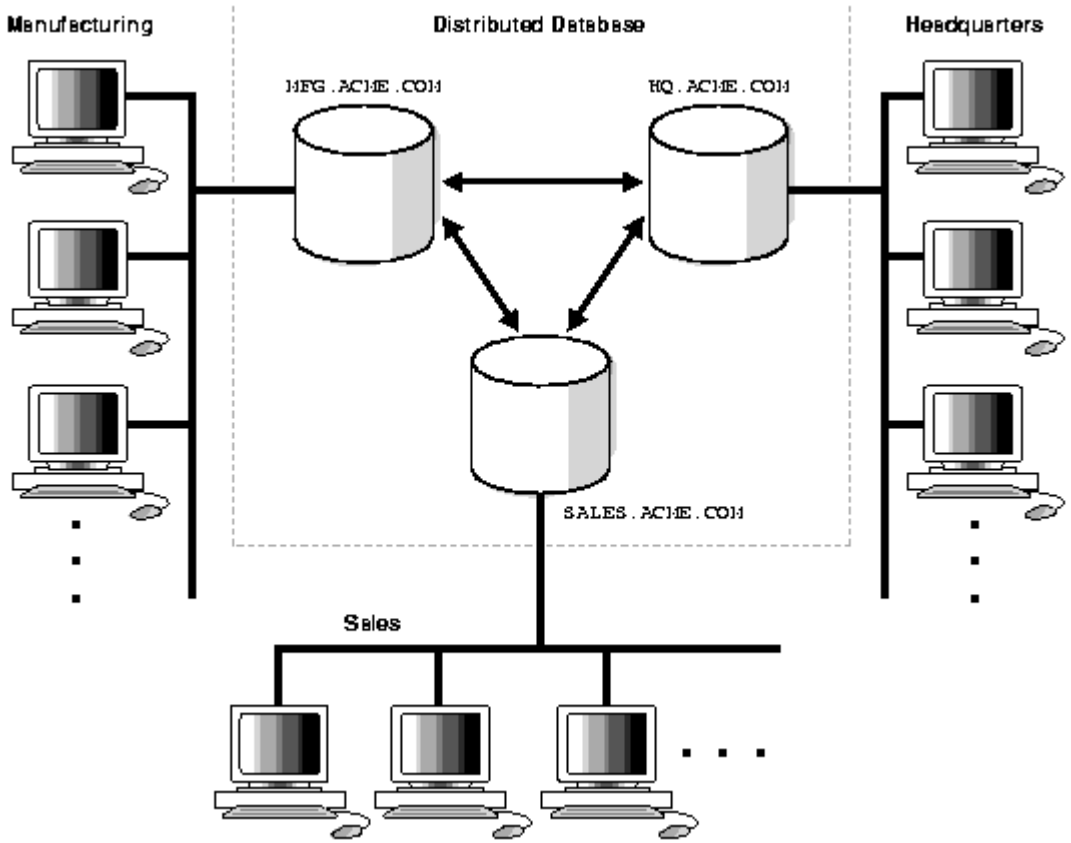
- We define a distributed database as a ***collection of multiple, logically interrelated databases distributed over a computer network.***

- The two important terms in these definitions are "***logically interrelated***" and "***distributed over a computer network***"

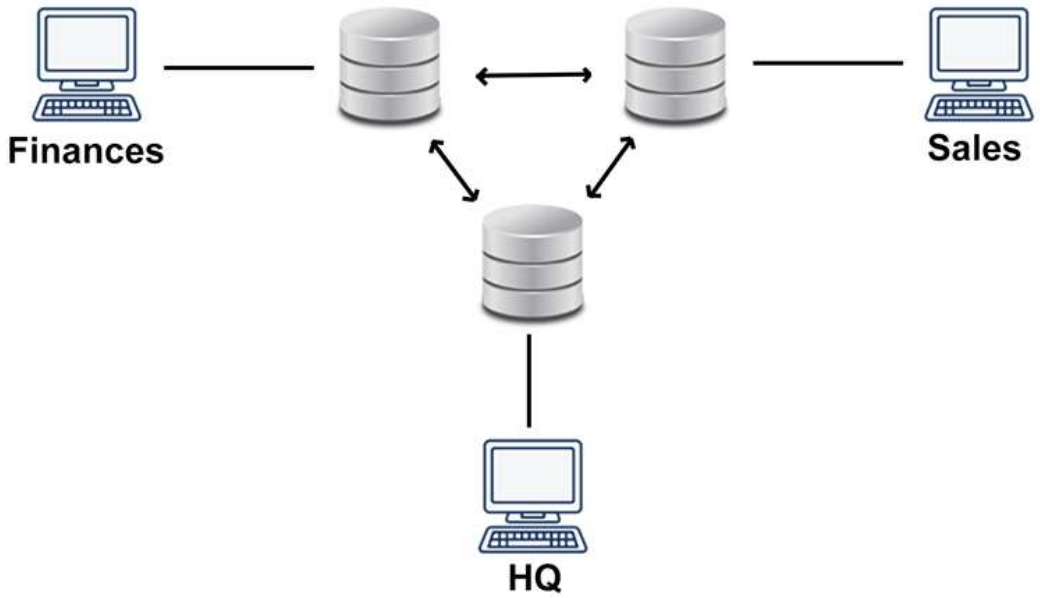
- A DDBS is **not a "collection of files**" that can be individually stored at each node of a computer network. To form a DDBS, ***files should not only be logically related, but there should be structured among the files, and access should be via a common interface.***

- It is important to make a distinction between a DDBS where this requirement is met, and more general distributed data management systems that provide a **"DBMS-like" access to data.**

DDBS Environment



Example-1



Example-2

Distributed Data Storage

There are 2 ways in which data can be stored on different sites. These are:

1. **Replication-** In this approach, the entire relationship is stored redundantly at 2 or more sites. If the entire database is available at all sites, it is a fully redundant database. Hence, in replication, systems maintain copies of data.

This is **advantageous** as it increases the **availability** of data at different sites.

It has certain **disadvantages** as well. Data needs to be constantly updated. Any **change made at one site needs to be recorded at every site** that relation is stored or else it may lead to inconsistency.

Distributed Data Storage

2. Fragmentation- In this approach, the relations are **fragmented (i.e., they're divided into smaller parts)** and each of the fragments is stored in different sites where they're required. It must be made sure that the fragments are such that they can be used to reconstruct the **original relation** (i.e, there isn't any loss of data).

Fragmentation is **advantageous** as it doesn't create copies of data, **consistency is not a problem.**

Fragmentation of relations can be done in two ways:

Horizontal fragmentation – Splitting by rows –

- The relation is fragmented into groups of tuples so that each tuple is assigned to at least one fragment.

Vertical fragmentation – Splitting by columns –

- The schema of the relation is divided into smaller schemas. Each fragment must contain a common candidate key so as to ensure a lossless join.

Distributed DBMS Promises

Distributed Database Management Systems (DDBMS) offer several promises or advantages that make them an attractive solution for organizations, particularly those with **distributed operations or large-scale data management** needs. Here are the key promises of DDBMS:

1. Improved Performance

Parallel Query Execution: Queries can be executed in parallel across multiple nodes, reducing response time and increasing throughput.

Local Processing: Data is processed locally at the site where it resides, minimizing the need for expensive data transfers.

2. Scalability

Horizontal Scalability: DDBMS can easily scale by adding new nodes to the system without significant changes to the architecture.

Growth Handling: The system can handle increased data volumes and user requests effectively.

Distributed DBMS Promises

3. High Availability and Reliability

- **Fault Tolerance:** Even if a node fails, the system can continue to function using replicated data or by rerouting queries to other operational nodes.
- **Data Redundancy:** Replicating data across multiple sites ensures data availability even during failures.
- **Continuous Operations:** With distributed nodes, the system can be designed to allow maintenance or updates without downtime.

4. Data Sharing and Integration

- **Collaborative Access:** Multiple sites can share and access data seamlessly, enabling collaborative operations.
- **Integration of Heterogeneous Databases:** DDBMS can unify diverse databases under a single system, enabling seamless access and management.

Distributed DBMS Promises

5. Cost-Effectiveness

Resource Optimization: Distributed systems allow organizations to utilize resources at multiple sites efficiently.

Cost Distribution: Hardware and operational costs are spread across various locations instead of relying on a single, expensive centralized system.

6. Modularity

- **Ease of Expansion:** New nodes or data can be added to the system without significant disruption.
- **Flexible Design:** The modular architecture allows for changes and updates to individual nodes without affecting the entire system.

Distributed DBMS Promises

7. Improved Security

- **Local Access Control:** Data at each site can have specific security measures tailored to local requirements.
- **Distributed Risk:** Data is not stored in a single location, reducing the risk of a catastrophic loss.

8. Support for Heterogeneous Environments

Integration of Diverse Systems: DDBMS can work across different hardware, software, and database platforms, making it suitable for organizations with diverse IT ecosystems.

Problem areas Distributed DBMS

Distributed Database Management Systems (DDBMS) are powerful but come with several problem areas or challenges due to their distributed nature. Here's an overview of key problem areas in DDBMS architecture:

1. Data Distribution and Fragmentation

Fragmentation Design: Determining how to split the database into fragments (horizontal, vertical, or hybrid) for optimal performance is complex.

Data Placement: Deciding where to store each fragment can significantly affect query performance and communication costs.

Replication Issues: Managing replicated data across multiple sites is challenging, especially ensuring consistency.

2. Transparency

Access Transparency: Users and applications should be unaware of whether the database is centralized or distributed, which requires complex abstraction.

Location Transparency: Users should not need to know where the data resides.

Replication Transparency: Applications should not need to manage multiple copies of data explicitly.

Failure Transparency: Ensuring operations continue seamlessly even in case of node or communication failures.

Problem areas Distributed DBMS

3. Concurrency Control

Synchronization: Ensuring transactions across multiple nodes remain consistent.

Distributed Deadlocks: Detecting and resolving deadlocks in a distributed environment is harder than in a centralized system.

Latency: Distributed locking mechanisms introduce latency due to communication overhead.

4. Distributed Query Processing and Optimization

Query Decomposition: Breaking queries into sub-queries for execution across nodes.

Cost Estimation: Estimating costs of distributed queries, which depend on data location, network latency, and system load.

Coordination: Combining results from multiple nodes while minimizing data transfer.

Problem areas Distributed DBMS

5. Data Consistency and Integrity

Eventual Consistency: Balancing strong consistency guarantees with performance in distributed systems.

Transaction Management: Implementing ACID (Atomicity, Consistency, Isolation, Durability) properties in distributed environments requires protocols like 2PC (Two-Phase Commit) or Paxos, which introduce overhead.

Schema Evolution: Keeping schema consistent across distributed sites is non-trivial.

6. Fault Tolerance and Recovery

Node Failures: Handling node or site failures gracefully without data loss or inconsistent states.

Network Failures: Ensuring communication failures do not lead to data inconsistencies or transaction failures.

Recovery Protocols: Developing mechanisms for restoring data and transactions after failures.

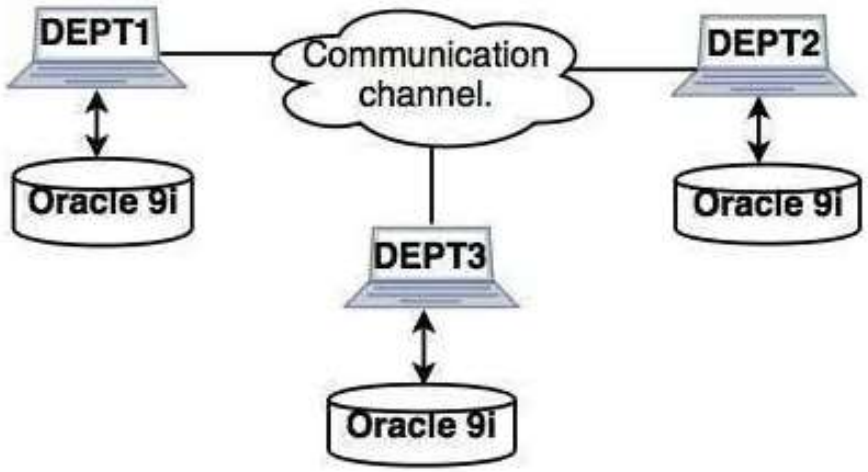
Architecture Models of Distributed Database Systems:

Types of Distributed Databases

Distributed databases can be broadly classified into **homogeneous and heterogeneous** distributed database environments.

Homogeneous – Same DBMS at each Node

1. Homogeneous Database: In a homogeneous database, all different sites store database identically. The operating system, database management system, and the data structures used – all are the same at all sites. Hence, they're easy to manage.

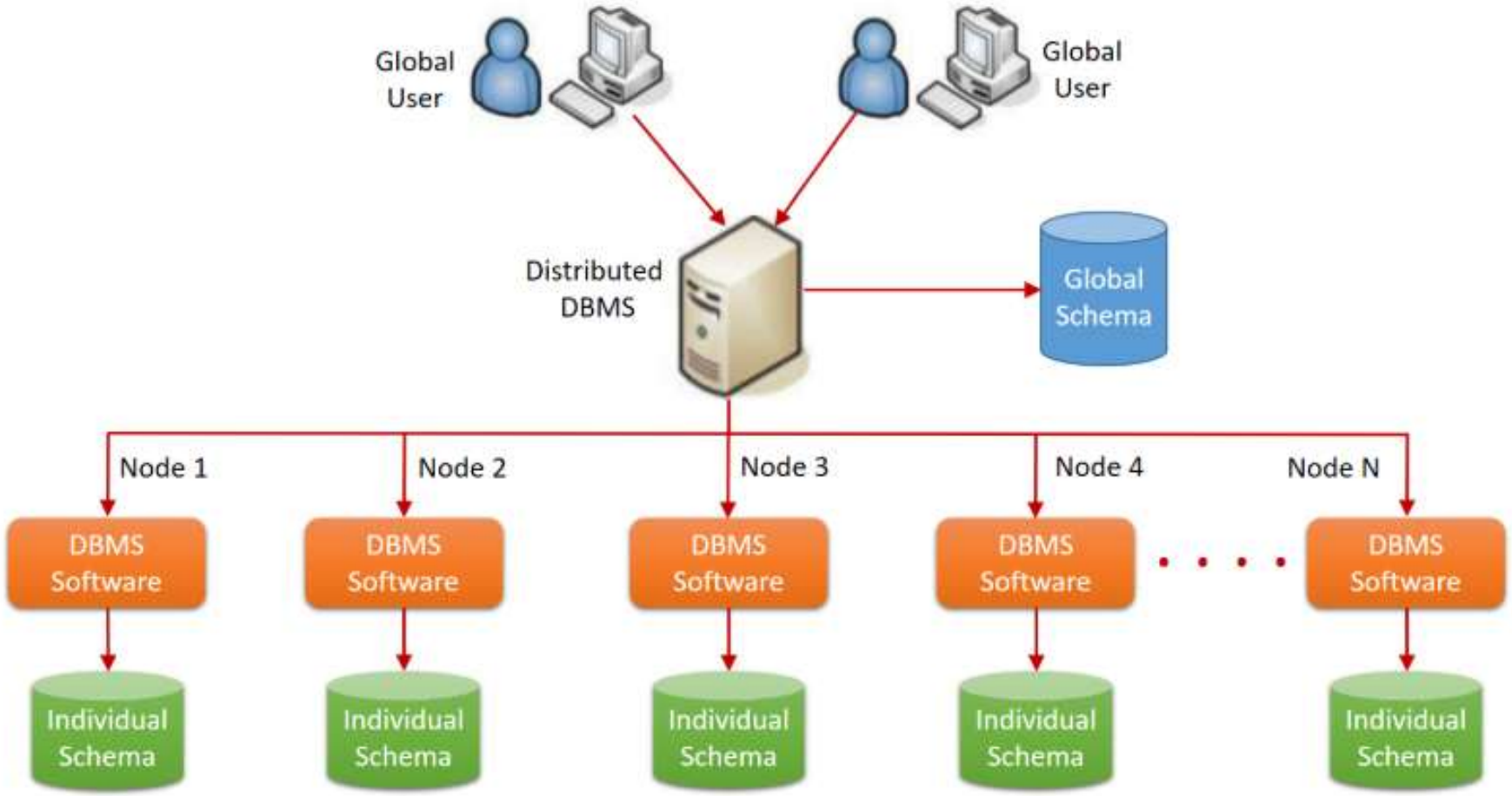


Homogeneous distributed system

Homogeneous Distributed Databases

In a homogeneous distributed database, all the sites use identical DBMS and operating systems. Its properties are

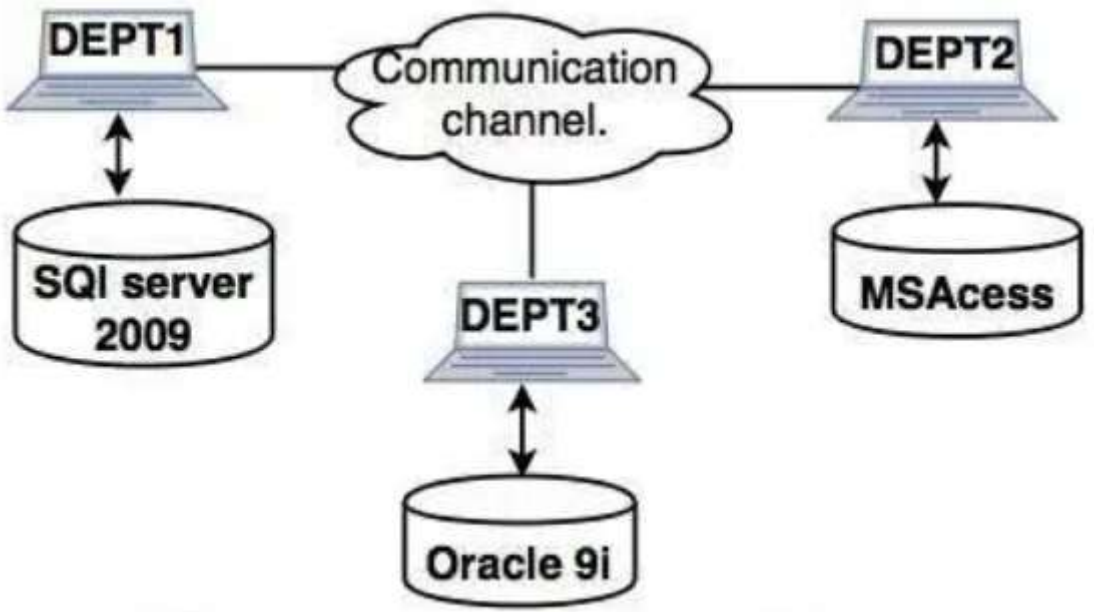
- The sites use very similar software.
- The sites use identical DBMS or DBMS from the same vendor.
- Each site is aware of all other sites and cooperates with other sites to process user requests.
- The database is accessed through a single interface as if it is a single database.



A homogeneous distributed database is a network of identical databases that use the same operating systems and DBMS across multiple sites

Heterogeneous Database: Different sites can use different schema and software that can lead to problems in query processing and transactions. Also, a particular site might be completely unaware of the other sites. Different computers may use a different operating system, different database application.

Heterogeneous- Different DBMS at different Nodes

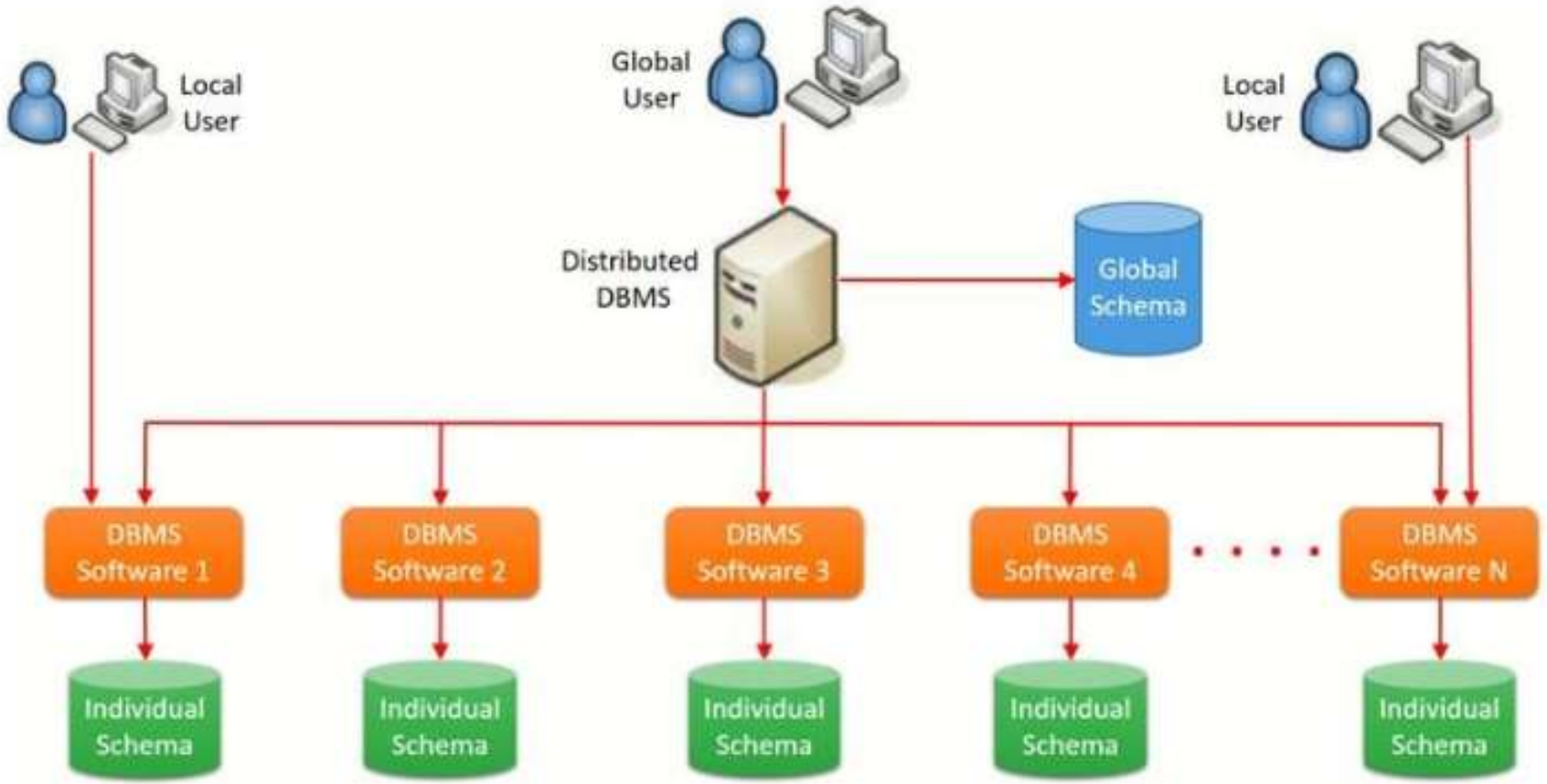


Heterogeneous distributed system

Heterogeneous Distributed Databases

In a heterogeneous distributed database, different sites have different operating systems, DBMS products and data models. Its properties are −

- Different sites use dissimilar schemas and software.
- The system may be composed of a variety of DBMSs like relational, network, hierarchical or object oriented.
- Query processing is complex due to dissimilar schemas.
- Transaction processing is complex due to dissimilar software.
- A site may not be aware of other sites and so there is limited co-operation in processing user requests.



A heterogeneous distributed database is a system that combines multiple different systems, such as data sets, operating systems, data models, and DBMSs

Distributed DBMS Architectures

DDBMS architectures are generally developed depending on three parameters −

Distribution − It states the physical **distribution of data** across the different sites.

Autonomy − It indicates the distribution of control of the database system and the degree to which each constituent DBMS can **operate independently**.

Heterogeneity − It refers to the uniformity or **dissimilarity of the data models**, system components and databases.

Architectural Models

Some of the common architectural models are −

- Client - Server Architecture for DDBMS
- Peer - to - Peer Architecture for DDBMS
- Multi - DBMS Architecture

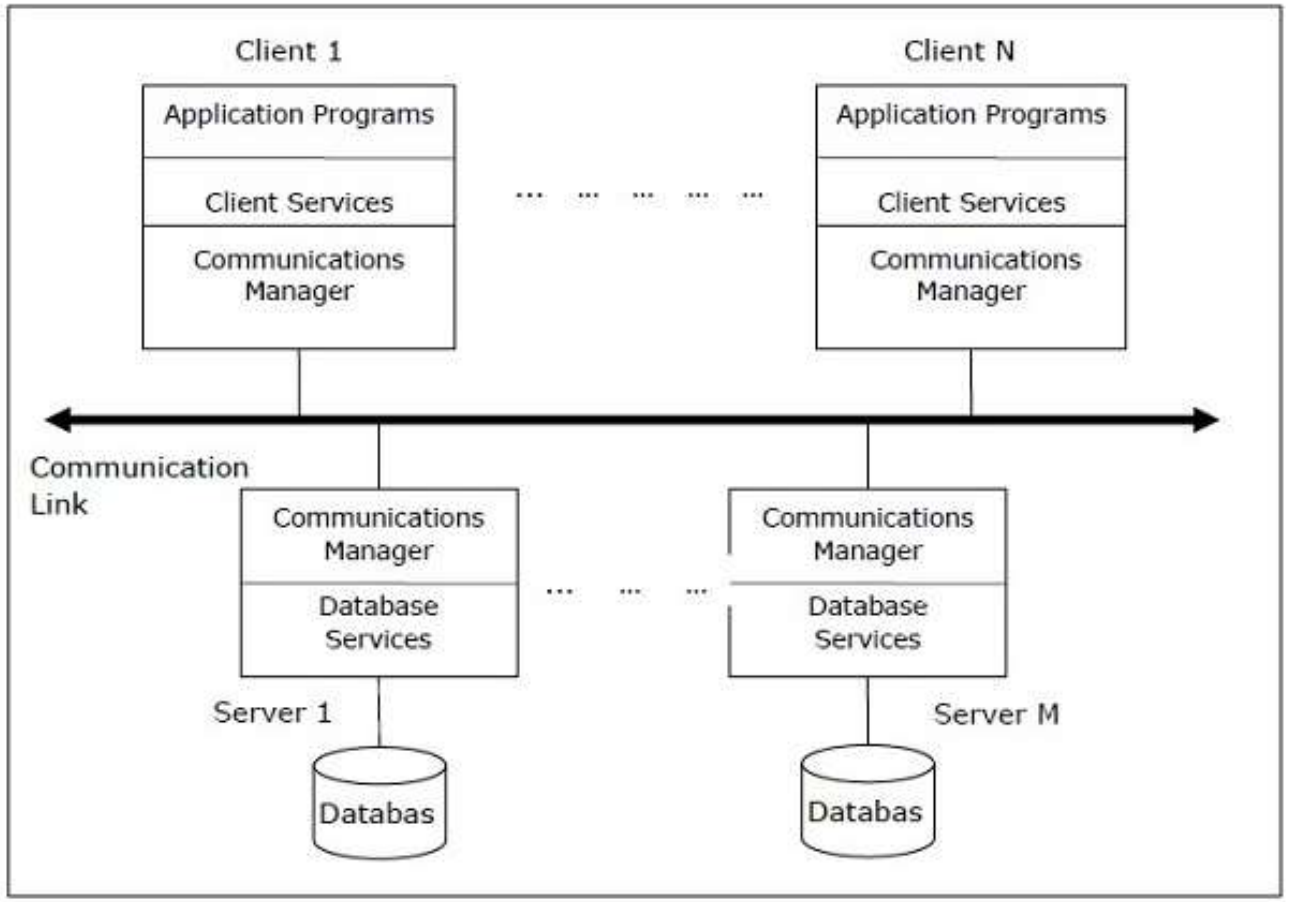
Client - Server Architecture for DDBMS

This is a two-level architecture where the functionality is divided into servers and clients. The server functions primarily encompass data management, query processing, optimization and transaction management. Client functions include mainly user interface. However, they have some functions like consistency checking and transaction management.

The two different client - server architecture are −

- Single Server Multiple Client
- Multiple Server Multiple Client

(shown in the following diagram)



Peer- to-Peer Architecture for DDBMS

In these systems**, each peer acts both as a client and a server** for imparting database services. The peers share their resource with other peers and co-ordinate their activities.

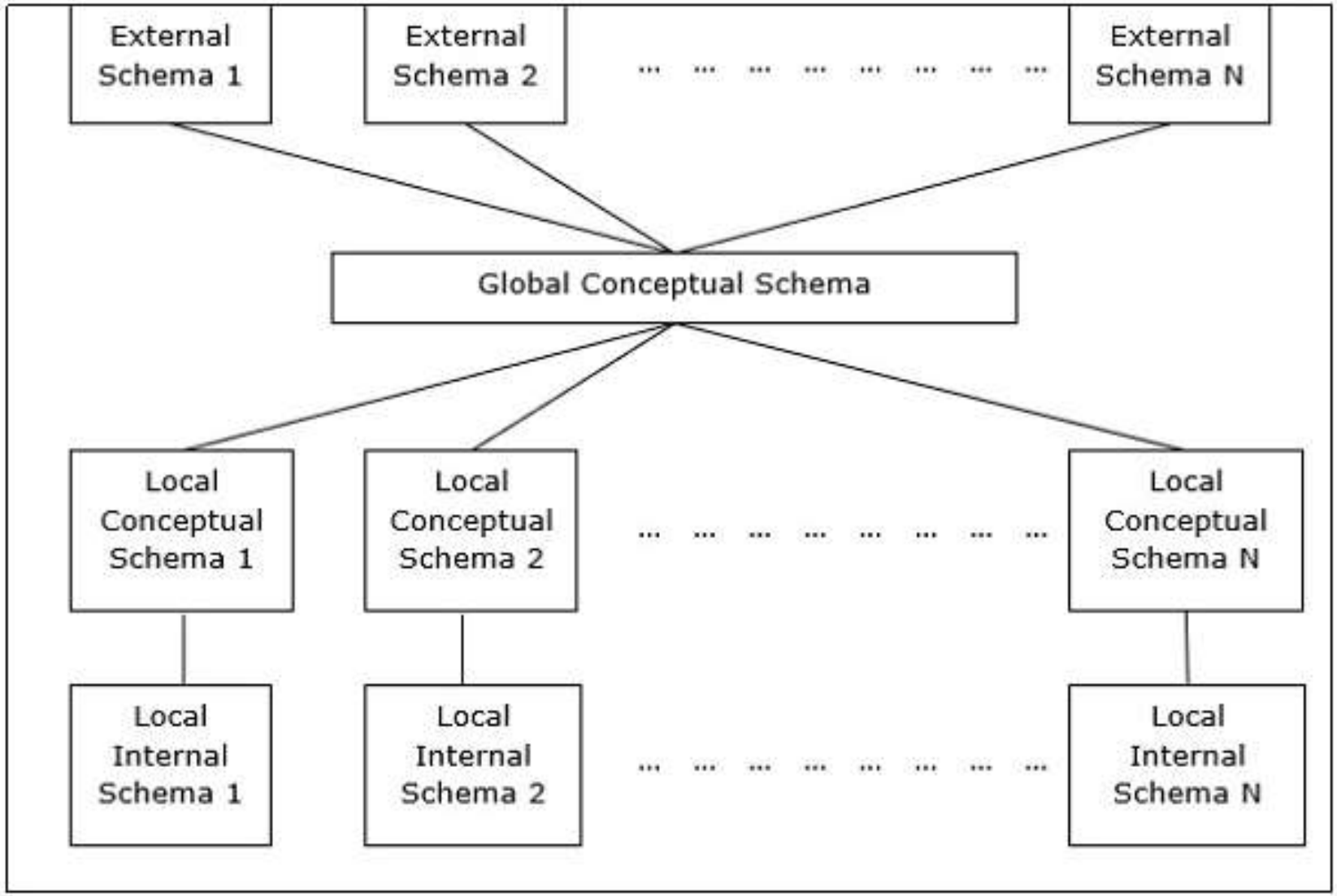
This architecture generally has four levels of schemas −

Global Conceptual Schema − Depicts the global logical view of data.

Local Conceptual Schema − Depicts logical data organization at each site.

Local Internal Schema − Depicts physical data organization at each site.

External Schema − Depicts user view of data.


External Schema 1
External Schema 2
External Schema N
Global Conceptual Schema
Local Conceptual Schema 1
Local Conceptual Schema 2
Local Conceptual Schema N
Local Internal Schema 1
Local Internal Schema 2
Local Internal Schema N

Multi - DBMS Architectures

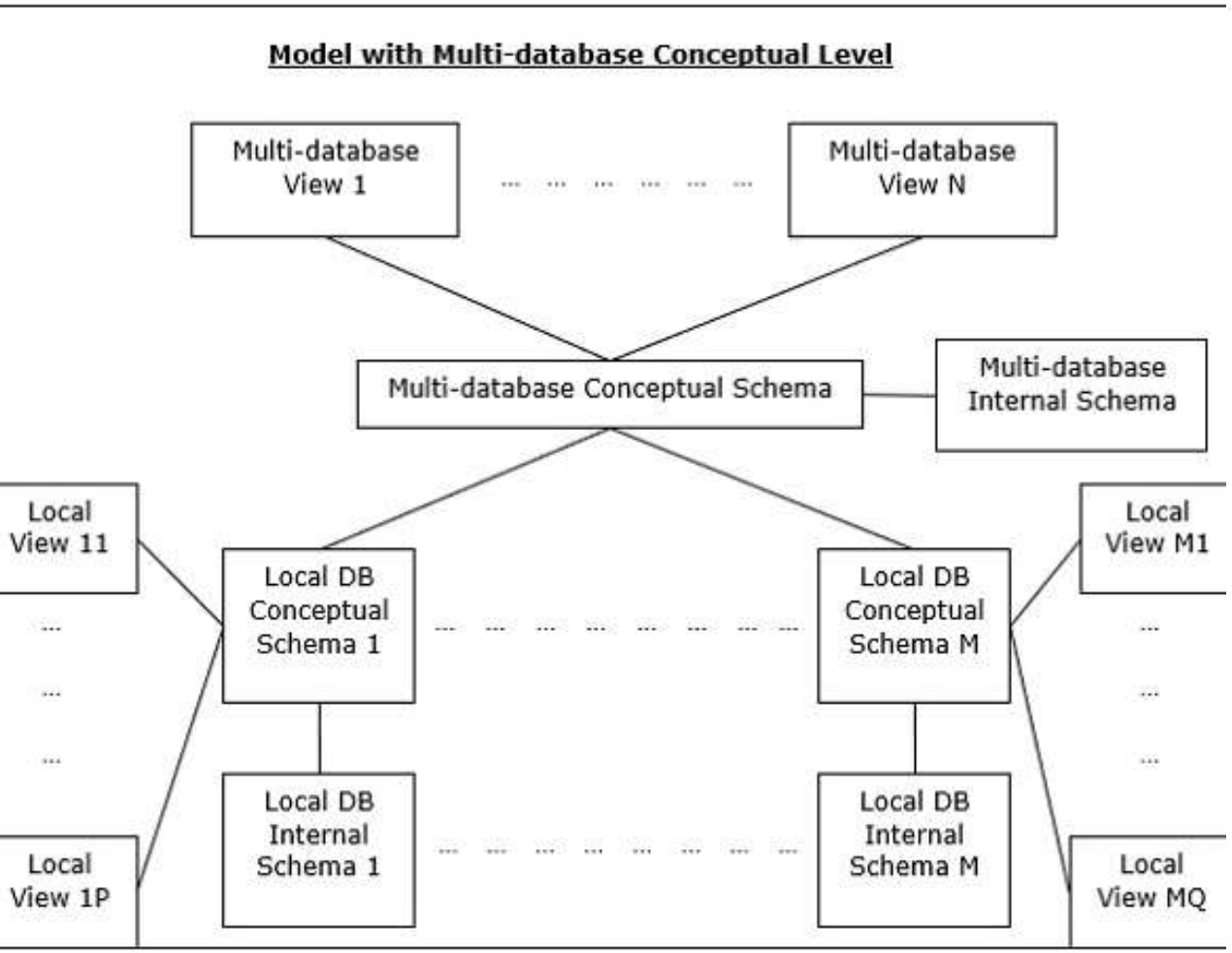
This is an integrated database system formed by a collection of two or more autonomous database systems.

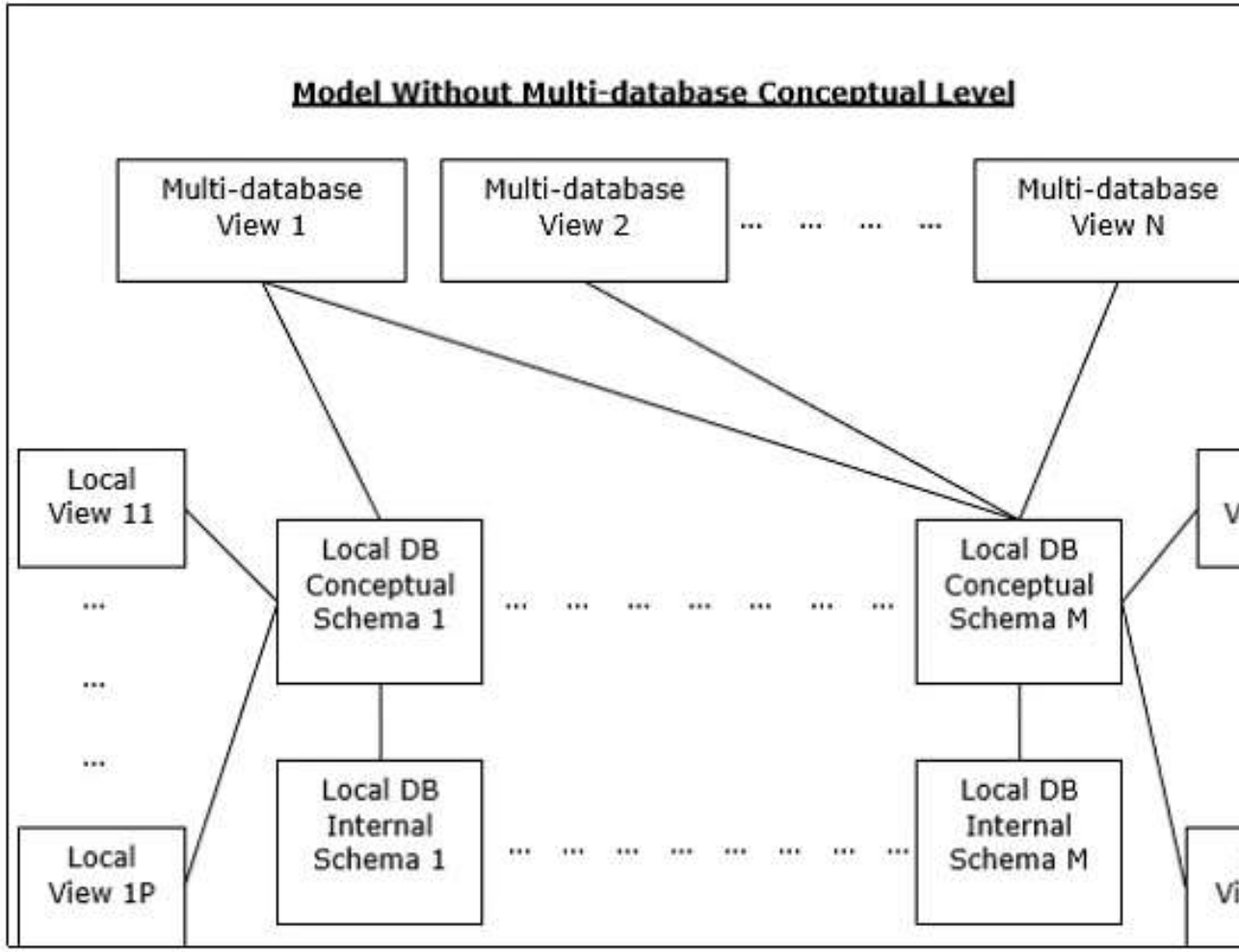
Multi-DBMS can be expressed through six levels of schemas −

- **Multi-database View Level** − Depicts multiple **user views** comprising of subsets of the integrated distributed database.
- **Multi-database Conceptual Level** − Depicts integrated multi-database that comprises of global logical **multi-database structure** definitions.
- **Multi-database Internal Level** − Depicts the data distribution across different sites and **multi-database to local data** mapping.
- **Local database View Level** − Depicts public view of local data.
- **Local database Conceptual Level** − Depicts local data organization at each site.
- **Local database Internal Level** − Depicts physical data organization at each site.

There are two design alternatives for multi-DBMS −

- **Model with multi-database conceptual level.**
- **Model without multi-database conceptual level.**





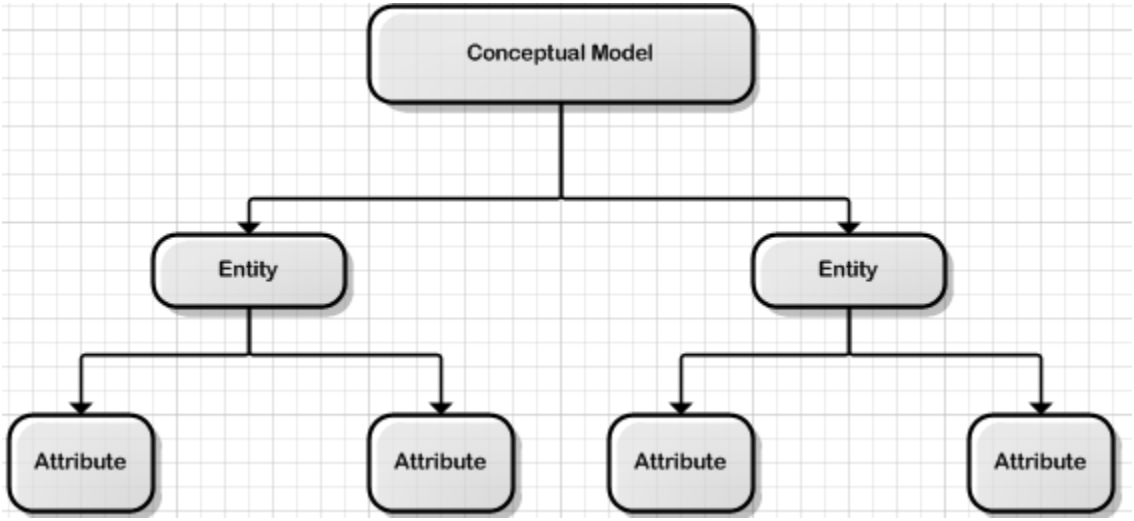
- A "**model with multi-database**" refers to an application or system that **interacts with multiple separate databases**, allowing it to store and access data from different sources

- While a "**model without multi-database**" only uses a **single database to manage all its data**, meaning it only interacts with one data source at a time

- **The key difference is the ability to access and manage data across various independent databases in a multi-database model.**

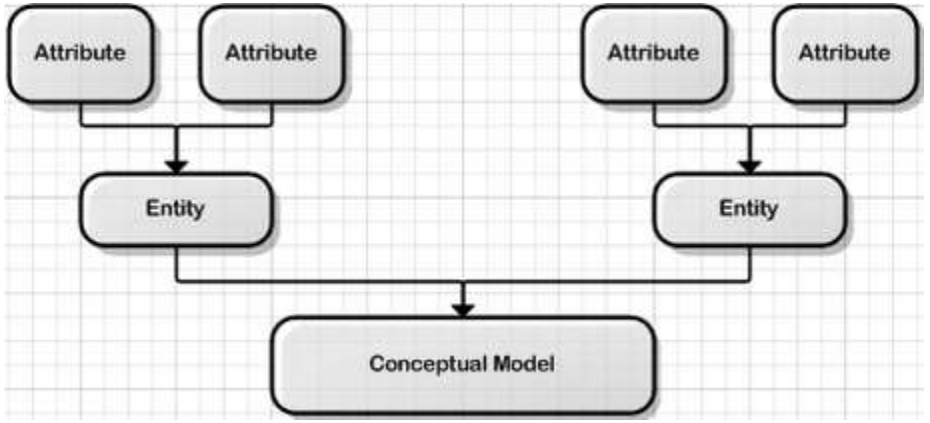
Distributed Database Design

The top-down design method starts from the general and moves to the specific. This process involves the identification of different entity types and the definition of each entity's attributes



Distributed Database Design

The bottom-up approach begins with the specific details and moves up to the general. This is done by first identifying the data elements (items) and then grouping them together in data sets. In other words, this method first identifies the attributes, and then groups them to form entities.



Alternate Design Strategies

Design Alternatives

The distribution design alternatives for the tables in a DDBMS are as follows −

- Non-replicated and non-fragmented
- Fully replicated
- Partially replicated and Fragmented
- Mixed

Non-replicated & Non-fragmented

- **Non-Replicated**: This means that **the data exists in only one location and is not copied to other nodes or servers for redundancy or availability**. If the single data source becomes unavailable, the data cannot be accessed until the issue is resolved. This approach has low overhead in terms of storage but sacrifices fault tolerance and availability.
- **Non-Fragmented**: This indicates that the **data is stored in its entirety as a single entity rather than being divided into smaller pieces** (fragments) distributed across multiple locations. Non-fragmented data simplifies access since all related data is in one place, but it may limit scalability and performance in distributed systems.
- **Example Use Case**
- A standalone database used for simple applications where high availability and distributed access are not critical might adopt this model.

Alternate Design Strategies

Fully Replicated

Fully Replicated refers to a data storage strategy where the **entire dataset is copied and stored across multiple nodes or servers in a system**. Each node has a complete copy of the data. This approach is commonly used in distributed systems to improve availability, fault tolerance, and performance.

Characteristics of Fully Replicated Systems

- **Data Redundancy**: Each node contains the same complete dataset, so data is never lost as long as one node remains available.
- **High Availability**: If one node fails, other nodes can continue serving requests without interruption.
- **Read Optimization**: Read operations are fast because data can be fetched from any available node, potentially reducing latency.
- **Write Complexity**: Writes are more complex because updates must be propagated to all replicas to maintain consistency.

Alternate Design Strategies

Fully Replicated

Use Cases

- **Content Delivery Networks (CDNs)**: Distributing the same data (e.g., website content, media) globally for fast access.
- **Distributed Databases**: Systems like CouchDB or Cassandra in scenarios requiring high availability.
- **Fault-Tolerant Systems**: Applications needing uninterrupted access even during node failures.

Partially Replicated and Fragmented

- **Partially Replicated**: **Only some parts of the data are replicated across multiple nodes,** while other parts are stored on specific nodes. Replication is selective and based on factors like access patterns, criticality of the data, or geographic proximity to users.
- **Fragmented**: The dataset is **divided into smaller pieces (fragments or shards), and these fragments are distributed across nodes in the system**. Each fragment typically contains a subset of the overall data.

Characteristics of Partially Replicated and Fragmented Systems

- **Selective Replication**: Certain fragments are replicated on multiple nodes for fault tolerance or performance, while others might exist on a single node.
- **Efficient Storage**: By fragmenting the data and replicating only what's necessary, the system uses storage more efficiently than a fully replicated approach.
- **Scalability**: Fragmentation allows the system to scale horizontally by adding more nodes to store additional fragments.

Partially Replicated and Fragmented

Use Cases

- **Distributed Databases**: Many NoSQL databases like MongoDB, Cassandra, or CockroachDB use fragmentation (sharding) combined with partial replication for scalability and fault tolerance.
- **Geo-Distributed Applications**: Replicating only certain fragments to regions where they are most needed (e.g., user data near their location).
- **Hybrid Data Models**: Systems where frequently accessed "hot" data is replicated more extensively, while "cold" data is stored in fragments without replication.

Mixed

Mixed refers to a data distribution strategy that combines multiple techniques—such as replication and fragmentation—within the same system. In a **mixed** system, some data is **fully replicated**, some is **partially replicated**, and some is **non-replicated**. Similarly, data can be fragmented (sharded) or stored as a single entity, depending on the use case or operational requirements.

Characteristics of a Mixed System

- **Combination of Replication Levels**:

  Fully Replicated: Critical or frequently accessed data is stored on all nodes for high availability and quick reads.

  Partially Replicated: Some fragments are replicated only to a subset of nodes to balance fault tolerance with resource efficiency.

  Non-Replicated: Non-critical data is stored on a single node to save resources.

- **Combination of Fragmentation**:

  Fragmented (Sharded): Large datasets are divided into fragments and distributed across nodes for scalability.

  Non-Fragmented: Smaller datasets or critical pieces of data may be stored in full at specific locations.

Mixed

Use Cases for Mixed Systems

Distributed Databases:

Many distributed databases (e.g., MongoDB, Cassandra) use a mixed approach for scalability and fault tolerance.

For instance, user profiles might be fully replicated, while logs or less-accessed historical data are sharded and partially replicated.

Geo-Distributed Applications:

Replicating frequently used data (like user settings or preferences) across regions, while storing large datasets (e.g., logs or archives) in fragments closer to where they are generated.

Hybrid Workloads:

E-commerce platforms, where product catalogs are fragmented and partially replicated, but transaction data is fully replicated for high availability.

IoT Systems:

Frequently accessed sensor data might be fully replicated near decision-making systems, while raw historical data is fragmented and stored in a cost-effective manner.

Distribution Design issues

Several key issues need to be addressed to ensure **optimal performance, scalability, and reliability.** These are collectively referred to as distribution design issues. Below is an overview of the primary concerns and considerations:

1. Data Fragmentation

- **What it is**: Dividing the database into smaller pieces (fragments or shards) that can be distributed across multiple nodes.
- **Types of Fragmentation**:
  - **Horizontal Fragmentation**: Divides data into subsets of rows (tuples) based on a condition (e.g., by region or department).
  - **Vertical Fragmentation**: Divides data into subsets of columns (attributes), often based on access patterns.
  - **Mixed Fragmentation**: Combines horizontal and vertical fragmentation.
- **Challenges**:
  - Identifying which fragments are accessed together.
  - Balancing fragmentation granularity (too fine vs. too coarse).

2. Data Replication

What it is: Making copies of data (replicas) and storing them on multiple nodes for availability and fault tolerance.

Types of Replication:

Full Replication: Entire database is replicated on all nodes.

Partial Replication: Only some fragments are replicated on select nodes.

No Replication: Each fragment is stored on only one node.

Challenges:

- Deciding which data to replicate and where.
- Managing consistency between replicas (e.g., eventual consistency vs. strong consistency).
- Balancing the trade-offs between replication overhead and availability.

3. Data Allocation

What it is: Determining which fragments or replicas are stored on which nodes in the system.

Strategies:

Centralized: All data is stored on a single node (rare in distributed systems).

Decentralized: Data is distributed across multiple nodes based on predefined rules.

Dynamic Allocation: Data placement changes dynamically based on workload or access patterns.

Challenges:

- Minimizing data movement between nodes during queries.
- Balancing load across nodes to avoid bottlenecks.
- Considering geographic location for latency optimization.

4. Query Processing and Optimization

What it is: Executing queries efficiently in a distributed environment.

Challenges:

- Identifying the location of the required data (fragments and replicas).
- Minimizing data transfer between nodes.
- Optimizing query execution plans to reduce latency and improve performance.
- Handling distributed joins and aggregations efficiently.

5. Transaction Management

What it is: Ensuring ACID (Atomicity, Consistency, Isolation, Durability) properties for transactions in a distributed environment.

Challenges:

- Managing distributed transactions across multiple nodes.
- Handling failures during transactions to maintain atomicity and consistency.
- Deciding on a concurrency control mechanism (e.g., locking, timestamp ordering).
- Using distributed commit protocols like Two-Phase Commit (2PC) or Three-Phase Commit (3PC).

UNIT-II: Query processing and decomposition: Query processing objectives, characterization of query processors, layers of query processing, query decomposition. Distributed query Optimization: Query optimization, distributed query optimization algorithms.

Query processing and decomposition

Query Processing:

The process of translating a user's high-level SQL query into a sequence of low-level operations that the system can execute.

High-Level Query (SQL)

SELECT name, age FROM students WHERE age > 18;

Low-Level Query (Execution Plan)

The DBMS converts the above query into a low-level query that includes steps like:

1. Sequential Scan (or Index Scan):Scan **the students table row by row** (or use an index if available) to identify rows where the condition age > 18 is true.
2. Projection: Extract only the **columns name and age** from the selected rows.
3. Return Result: Output the **final filtered and projected result** to the user.

An execution plan for the query could look like this (example):

1. Scan Table: students
2. Filter Rows: Keep only rows where age > 18
3. Project Columns: Select only "name" and "age"
4. Output Result

Key Difference:

- High-Level Query focuses on *what data you want*.
- Low-Level Query focuses on *how the data is fetched* or processed internally. It’s usually handled by the DBMS automatically and isn’t visible to the user unless explicitly requested (e.g., using the EXPLAIN or EXPLAIN ANALYZE commands in SQL).

Query processing and decomposition

Query Decomposition:

Breaking a high-level query into smaller, manageable sub-queries based on fragmentation and distribution of data across the system.

Challenges in Distributed Query Processing:

Data fragmentation (horizontal, vertical, or hybrid).

Data replication and allocation across multiple nodes.

Communication costs (e.g., transferring data between sites).

Ensuring consistency and correctness of results.

Key Components:

Query Optimizer: Determines the most efficient execution plan.

Query Executor: Executes the optimized query across nodes.

Cost Model: Estimates the cost of various query execution plans.

Query processing objectives

Query processing objectives refer to the specific goals and tasks that systems **aim to achieve when handling a user's query**. These objectives are typically associated with **search engines, databases, or natural language processing systems**. Here are some common query processing objectives:

1. Understanding the Query

Intent Identification: Determine what the user is looking for (e.g., informational, navigational, transactional).

Disambiguation: Resolve ambiguities in the query (e.g., "Apple" as a fruit or a company).

Entity Recognition: Identify entities like names, dates, or locations within the query.

2. Retrieving Relevant Information

Precision: Return results that are highly relevant to the query.

Recall: Ensure that all relevant results are retrieved, minimizing omissions.

Context Awareness: Use context (e.g., user location, search history) to improve result relevance.

3. Optimizing Query Execution (in databases)

Efficiency: Process the query with minimal computational resources.

Speed: Minimize the response time for retrieving results.

Index Utilization: Use pre-built indexes to speed up searches.

Query processing objectives

4. Improving User Experience

Suggestions and Autocomplete: Offer query suggestions or complete the user's input.

Spell Correction: Handle typos or spelling errors.

Ranking and Personalization: Prioritize results based on user preferences or behavior.

5. Handling Complex Queries

Natural Language Understanding (NLU): Interpret conversational or detailed queries.

Multi-Intent Processing: Address queries with multiple intents (e.g., "Find a restaurant nearby and book a table").

Structured Query Transformation: Convert natural language into structured database queries.

6. Ensuring Quality and Accuracy

Relevance Scoring: Use algorithms to score and rank the results.

Fact-Checking: Ensure returned information is accurate and up-to-date.

Filtering: Remove irrelevant, unsafe, or misleading content.

7. Scalability and Adaptability

Handle Large-Scale Data: Manage queries on vast datasets.

Adaptation to New Queries: Handle evolving language patterns and new terms.

Multilingual Support: Process queries in various languages.

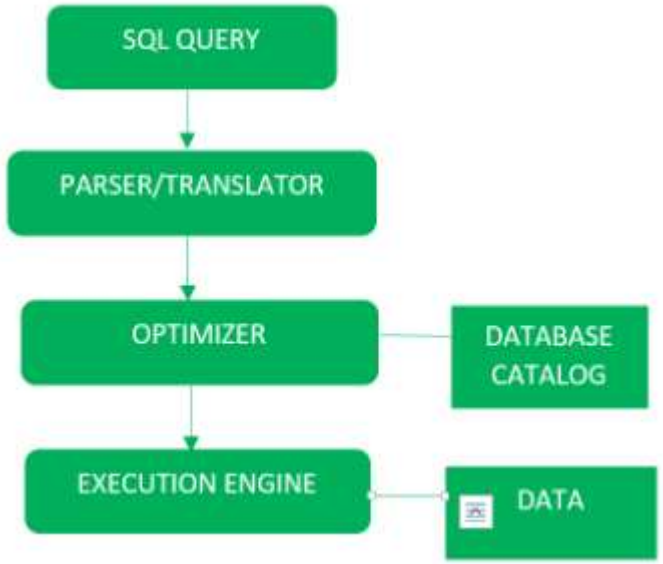
Query Processing in SQL

The process of extracting data from a database is called query processing.

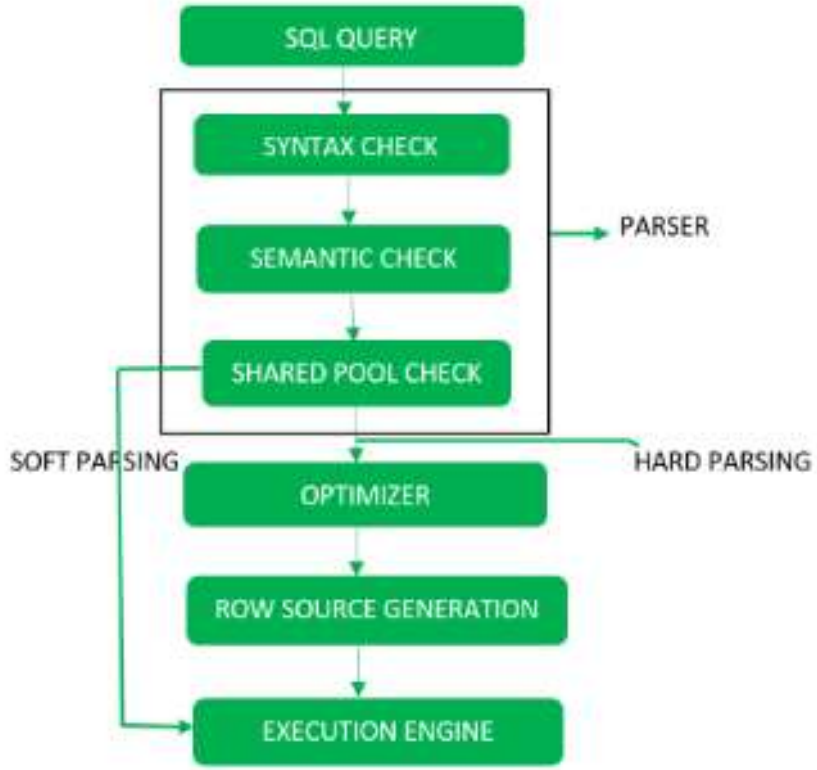
It requires several steps to retrieve the data from the database during query processing.

The actions involved actions are:

- **Parsing and translation**
- **Optimization**
- **Evaluation**



The Block Diagram of Query Processing



A detailed Diagram

Step-1 : Parsing

This step ensures that the query is syntactically and semantically correct.

Lexical Analysis:

- Breaks the query into tokens (keywords, identifiers, operators, etc.).
- For example, in SELECT name FROM students, tokens are SELECT, name, FROM, and students.

Syntactic Analysis :

- Validates whether the query follows the syntax rules of the query language.
- Example: If the user writes SELECT FROM name students, a syntax error will be thrown.

Semantic Analysis:

- Ensures the query makes sense in terms of the database schema.
- For instance:
  - Does the table students exist?
  - Does the column name exist in the students table?
  - Is the user authorized to access the specified data?

Shared Pool Check

Every **query possesses a hash code** during its execution. So, this check determines the **existence of written hash code in the shared pool if the code exists in the shared pool then the database will not take additional steps for optimization and execution**

Example of a Hash Code in Databases

Suppose you execute the SQL query: SELECT * FROM students WHERE age > 18;

The database generates a hash code for the query string (e.g., abcd1234) and checks if this hash code already exists in the shared pool.

How Hash Codes Work

Input Data:

The input could be **a string, a file, a database** query, etc.

Hash Function:

A mathematical function (e.g., MD5, SHA-256, or a database-specific algorithm) processes the input and produces a hash code.

Output (Hash Code):

A fixed-length numerical or alphanumeric representation of the input data.

Hash Code Example
For a string like "Hello, World!":
1. Input: "Hello, World!"
2. Hash Function: MD5
3. Hash Code: ***fc3ff98e8c6a0d3087d515c0473f8677***

How Does a Shared Pool Check Work?

When a query is submitted to the database, the shared pool is checked for the following:

Parsing Stage:

- The SQL query is hashed and its hash value is used to determine if it already exists in the **library cache**.
- If a matching query exists:
  - The database skips hard parsing (which is resource-intensive) and reuses the previously parsed version (this is called a **soft parse**).
- If no match is found:
  - The query undergoes **hard parsing**, which involves full syntax checking, semantic validation, and optimization.

Basic SQL Relational Algebra Operations

Relational algebra consists of a certain **set of rules or operations** that are widely used to manipulate and query data from a relational database. It can be facilitated by utilizing SQL language and helps users interact with database tables based on querying data from the database more efficiently and effectively.

Unary Relational Operations

SELECT (symbol: σ)

PROJECT (symbol: π)

RENAME (symbol: ρ)

Relational Algebra Operations From Set Theory

UNION (υ)

INTERSECTION ( ),

DIFFERENCE (-)

CARTESIAN PRODUCT (x)

Binary Relational Operations

JOIN

DIVISION

Basic SQL Relational Algebra Operations

SELECT (σ)

The SELECT operation is used for selecting a subset of the tuples according to a given selection condition. Sigma(σ)Symbol denotes it. It is used as an expression to choose tuples which meet the selection condition. Select operator selects tuples that satisfy a given predicate.

σp(r)

σ is the predicate

r stands for relation which is the name of the table

p is prepositional logic

Basic SQL Relational Algebra Operations

Example 1

σ topic = "Database" (Tutorials)

Output – Selects tuples from Tutorials where topic = ‘Database’.

Example 2

σ topic = "Database" and author = "guru99"(Tutorials)

Output – Selects tuples from Tutorials where the topic is ‘Database’ and ‘author’ is guru99.

Example 3

σ sales > 50000 (Customers)

Output – Selects tuples from Customers where sales is greater than 50000

Basic SQL Relational Algebra Operations

Projection(π)

The projection eliminates all attributes of the input relation but those mentioned in the projection list. The projection method defines a relation that contains a vertical subset of Relation.

This helps to extract the values of specified attributes to eliminates duplicate values. (pi) symbol is used to choose attributes from a relation. This operator helps you to keep specific columns from a relation and discards the other columns.

Example of Projection:

Consider the following table

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive
4	Alibaba	Active

Projection of CustomerName and status will give

Π CustomerName, Status (Customers)

CustomerName	Status
Google	Active
Amazon	Active
Apple	Inactive
Alibaba	Active

Relational Algebra Representation:

The query may also be translated into relational algebra, a mathematical notation used for database operations.
For example:

SQL: SELECT name FROM students WHERE age > 18

Relational Algebra: π_name(σ_age>18(students))

Projection (SELECT name)
Selection (age > 18)
students (table)

Tuple Relational Calculus (TRC) is a non-procedural query language used in relational database management systems (RDBMS) to retrieve data from tables. TRC is based on the **concept of tuples**, which are ordered sets of attribute values that represent a single row or record in a database table.

{ t | P(t) }

where **t is a tuple variable** and **P(t) is a logical formula that describes the conditions that the tuples** in the result must satisfy. The **curly braces {} are used to indicate that the expression is a set of tuples.**

For example, let's say we have a table called "**Employees**" with the following attributes:

Employee ID
Name
Salary
Department ID

To retrieve the names of all employees who earn more than $50,000 per year, we can use the following TRC query:

{ t | Employees(t) ∧ t.Salary > 50000 }

Domain Relational Calculus is a non-procedural query language equivalent in power to Tuple Relational Calculus.

Domain Relational Calculus provides only the description of the query but it does not provide the methods to solve it. In Domain Relational Calculus, a query is expressed as

$$\{ < x_1, x_2, x_3, \ldots, x_n > \mid P (x_1, x_2, x_3, \ldots, x_n) \}$$

< x1, x2, x3, …, xn> represents resulting domains variables

P (x1, x2, x3, …, xn) represents the condition or formula equivalent to the Predicate calculus.

Table-1 Customer

Customer name	Street	City
Debomit	Kadamtala	Alipurduar
Sayantan	Udaypur	Balurghat
Soumya	Nutanchati	Bankura
Ritu	Juhu	Mumbai

Table-2 Loan

Loan number	Branch name	Amount
L01	Main	200
L03	Main	150
L10	Sub	90
L08	Main	60

Table-3 Borrower

Customer name	Loan number
Ritu	L01
Debomit	L08
Soumya	L03

Query-1: Find the loan number, branch, amount of loans of greater than or equal to 100 amount.

$$\{\langle l, b, a\rangle \mid \langle l, b, a\rangle \in loan \wedge (a \geq 100)\}$$

Query-2: Find the loan number for each loan of an amount greater or equal to 150.

$$\{\langle l\rangle \mid \exists\, b, a\, (\langle l, b, a\rangle \in loan \wedge (a \geq 150))\}$$

Translation

Once the query passes parsing, it is translated into an internal, intermediate representation that the database can work with.

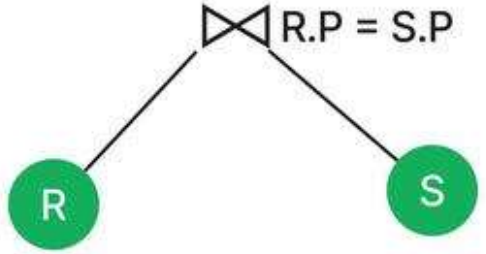
- **Query Tree:**
  - The query is often transformed into a tree-like structure called a parse tree or query tree.
  - Each node in the tree represents an operation (e.g., selection, projection, JOIN (⋈)).
- **Example:**
- For the query SELECT name FROM students WHERE age > 18, the tree might look like this:
  - Root: Projection (SELECT name)
  - Child: Selection (age > 18)
  - Leaf: students (table)

Example 1: Consider a relational algebra expression:

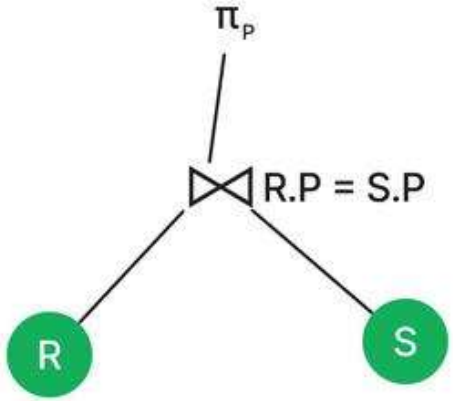
$$\pi p\ (R \bowtie R.P = S.P\ S)$$

Step 1: Write the relations you want to execute as the tree's Leaf nodes. Here R and S are the relations.

Step 2: Add the condition (here R.P = S.P) with the relational algebra operator as an internal node (or parent node of these two leaf nodes).



Step 3: Now add the root node that on execution gives the output of the query.



Example 2: For every project located at ‘Stanford’, list the project number (Pnumber), the controlling department number (Dnum), and the department manager’s last name (Lname), address (Address), and birth date (Bdate).

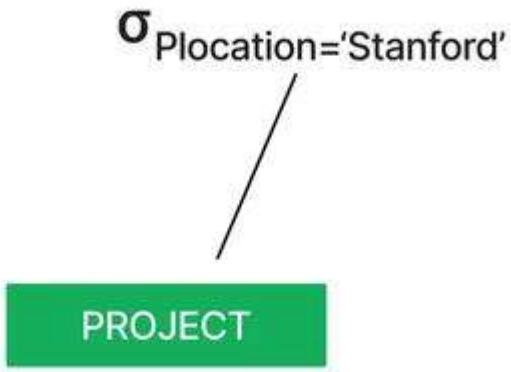
The relational algebra expression corresponding to this query:

πPnumber, Dnum, Lname, Address, Bdate

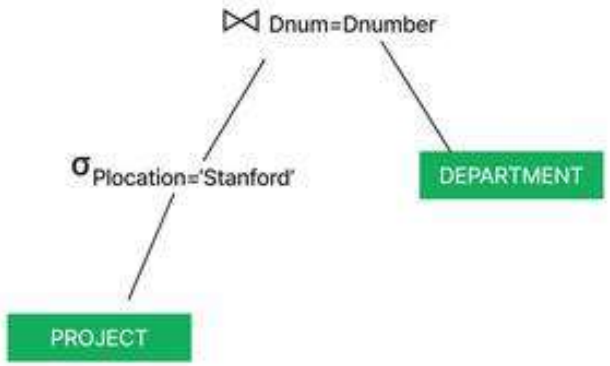
(((σPlocation = ‘Stanford’(PROJECT)) ⋈

Dnum=Dnumber(DEPARTMENT)) ⋈ Mgr_ssn=Ssn(EMPLOYEE))

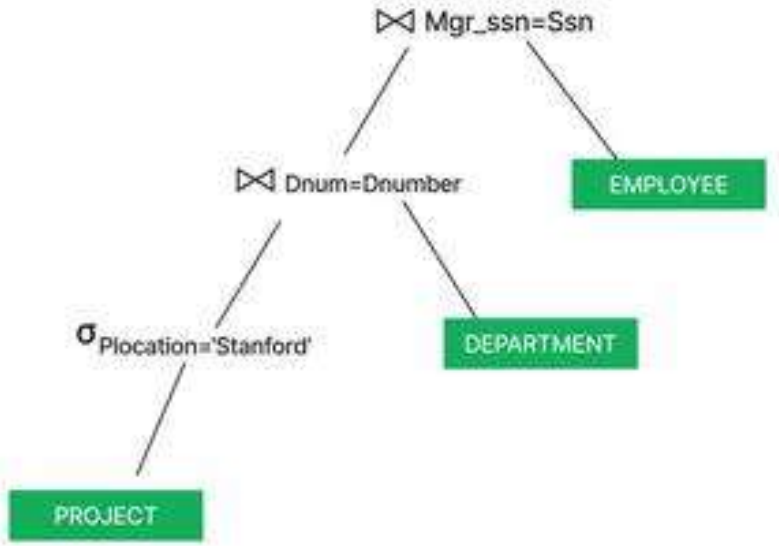
Step 1: We will begin with executing the first leaf node **PROJECT**, and the corresponding internal node $\sigma_{\text{Plocation = ‘Stanford’}}$ as we need these resulting tuples to execute the next operation.



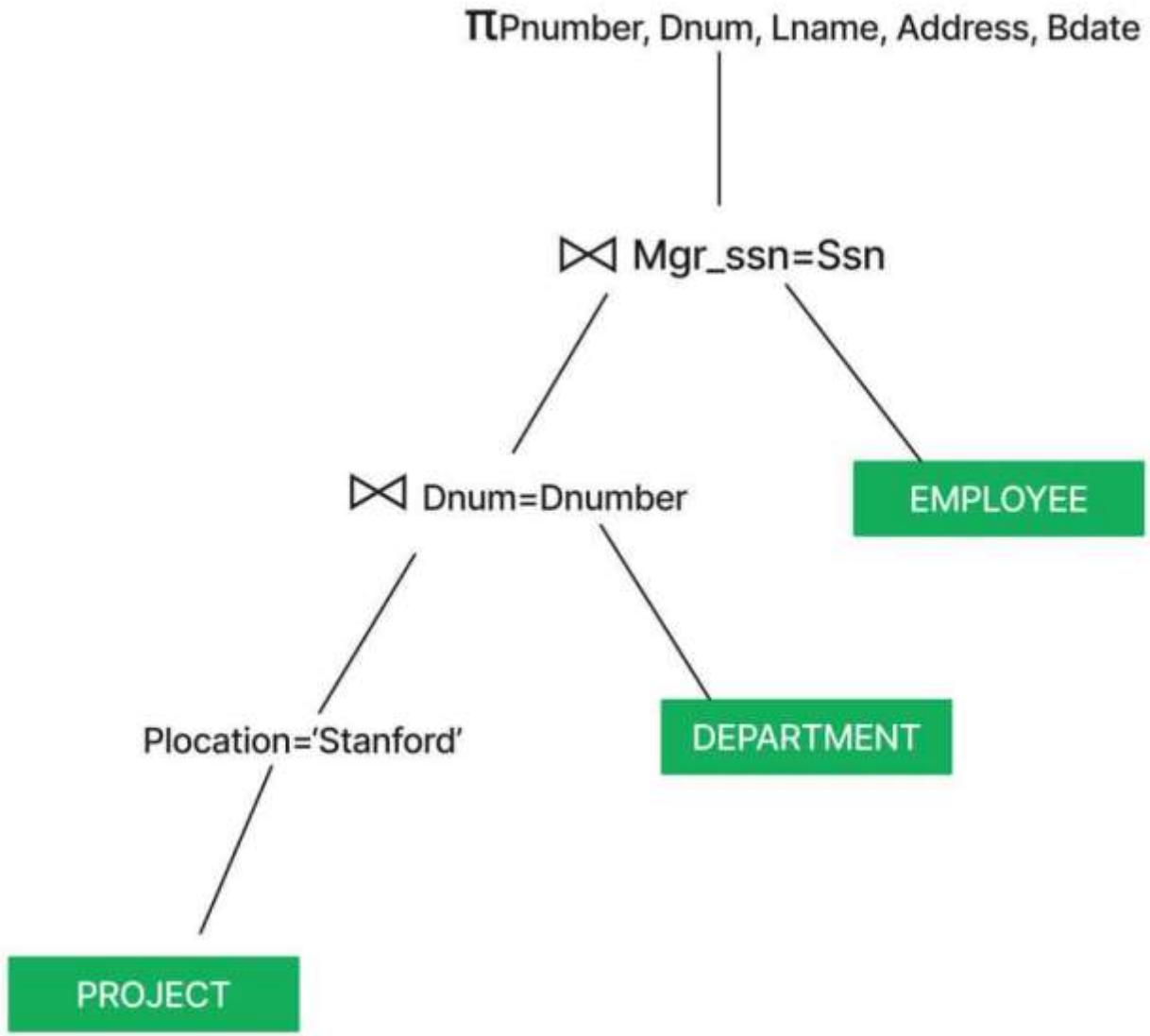
Step 2: Similarly, we will execute the leaf node **DEPARTMENT** and the intermediate/internal node $\bowtie_{\textbf{Dnum=Dnumber}}$ so that we can move to the next operation.



Step 3: We execute the next operation with the leaf node **EMPLOYEE** and intermediate node $\bowtie_{\textbf{Mgr\_ssn=Ssn}}$.



Step 4: Now add the root node i.e., $\pi_{\text{Pnumber, Dnum, Lname, Address, Bdate}}$ to get the output of the query on execution.



Step-2

Optimization

- Query optimization is the process of enhancing the **performance of a database** query by determining the most efficient way to execute it.

Why Query Optimization is Important

Faster Query Execution:

Optimized queries run faster, improving user experience.

Efficient Resource Utilization:

Reduces CPU, memory, and I/O usage.

Handles Large Datasets:

Critical for handling large-scale systems and high transaction loads.

Supports Scalability:

Ensures performance remains consistent as the database grows.

Query Optimizer

The **query optimizer** is a component of the database management system (DBMS) that automates the query optimization process.

It evaluates multiple execution plans and selects the one with the lowest cost.

Key Tasks of the Optimizer:

Plan Enumeration:

Considers different ways to execute the query, such as join orders and methods.

Cost Estimation:

Estimates the cost of each plan based on factors like I/O, CPU usage, and memory.

Plan Selection:

Chooses the optimal plan based on cost metrics.

Techniques for Query Optimization

1. Indexing

Indexes speed up data retrieval by reducing the number of rows scanned.

Example

SELECT name FROM students WHERE age > 18;

Adding an index on age improves performance.

2. Query Rewriting

Transform queries into equivalent, more efficient forms.

Example:

SELECT * FROM employees WHERE department_id = 10 AND salary > 5000;

Rewritten to:

SELECT * FROM employees WHERE salary > 5000 AND department_id = 10;

This ensures filters are applied in an order that reduces intermediate results.

** *3. Avoiding SELECT ** **

Retrieve only required columns instead of using SELECT *.

Example:

SELECT name, age FROM students WHERE age > 18;

This reduces I/O and memory usage compared to fetching all columns.

4. Predicate Pushdown

Apply filters as early as possible in the query execution plan.

Example: SELECT name FROM employees WHERE age > 30;

Without Predicate Pushdown:

- The query engine fetches all rows from the **employees** table.
- The filter **age > 30** is applied in the query engine after retrieving all rows.
- This requires more data to be read, transferred, and processed.

With Predicate Pushdown:

- The filter age > 30 is pushed down to the storage engine.
- The storage engine retrieves only the rows where age > 30.
- Only the relevant rows are sent to the query engine for further processing.

5. Join Optimization

Optimize the join order and join type.

Join Types:

Nested Loop Join: Efficient for small datasets.
Hash Join: Best for large, unsorted datasets.
Merge Join: Ideal for pre-sorted datasets.

Example: For:

SELECT * FROM orders o JOIN customers c ON o.customer_id = c.customer_id;

Ensure an index exists on customer_id in both tables.

6. Use Bind Variables

Prevent repeated hard parsing and encourage execution plan reuse.

Example:

```
SELECT * FROM students WHERE student_id = :id;
```

7. Partitioning

Divide large tables into smaller partitions to improve query performance.

Example: Partition a sales table by year:

```
SELECT * FROM sales WHERE sale_date >= '2023-01-01';
```

8. Collect and Update Statistics

Ensure the optimizer has accurate metadata about the data distribution.

Example:

```
EXEC DBMS_STATS.GATHER_TABLE_STATS('HR', 'EMPLOYEES');
```

Row Source Generation

Row Source Generation is software that receives an **optimal execution plan** from the optimizer and produces an **iterative execution plan** that is usable by the rest of the database.

Row Source Operations

Some **common row sources** in an execution plan include:

1. **Table Scan:**
   Reads all rows from a table.
   Example: TABLE ACCESS FULL.
2. **Index Scan:**
   Reads rows based on an index.
   Example: INDEX RANGE SCAN.
3. **Join Operations:**
   Nested Loop Join: Iterates through rows and matches with the joined table.
   Hash Join: Uses a hash table to match rows.
   Merge Join: Combines sorted datasets.

4. Sorting:

Sorts rows based on specified columns.

Example: SORT ORDER BY.

5. Filter:

Filters rows based on conditions.

Example: FILTER (age > 18).

6. Aggregation:

Performs operations like SUM, COUNT, or GROUP BY.

Example: SORT GROUP BY.

Step-3

Evaluation – Execution Engine

The **execution engine** is the component of a database management system (DBMS) that executes a query after it has been **parsed, optimized, and transformed** into an **execution plan**.

The execution engine interprets and runs the physical operations specified in the execution plan to retrieve the desired results from the database.

Steps in Query Execution

Query Submission:

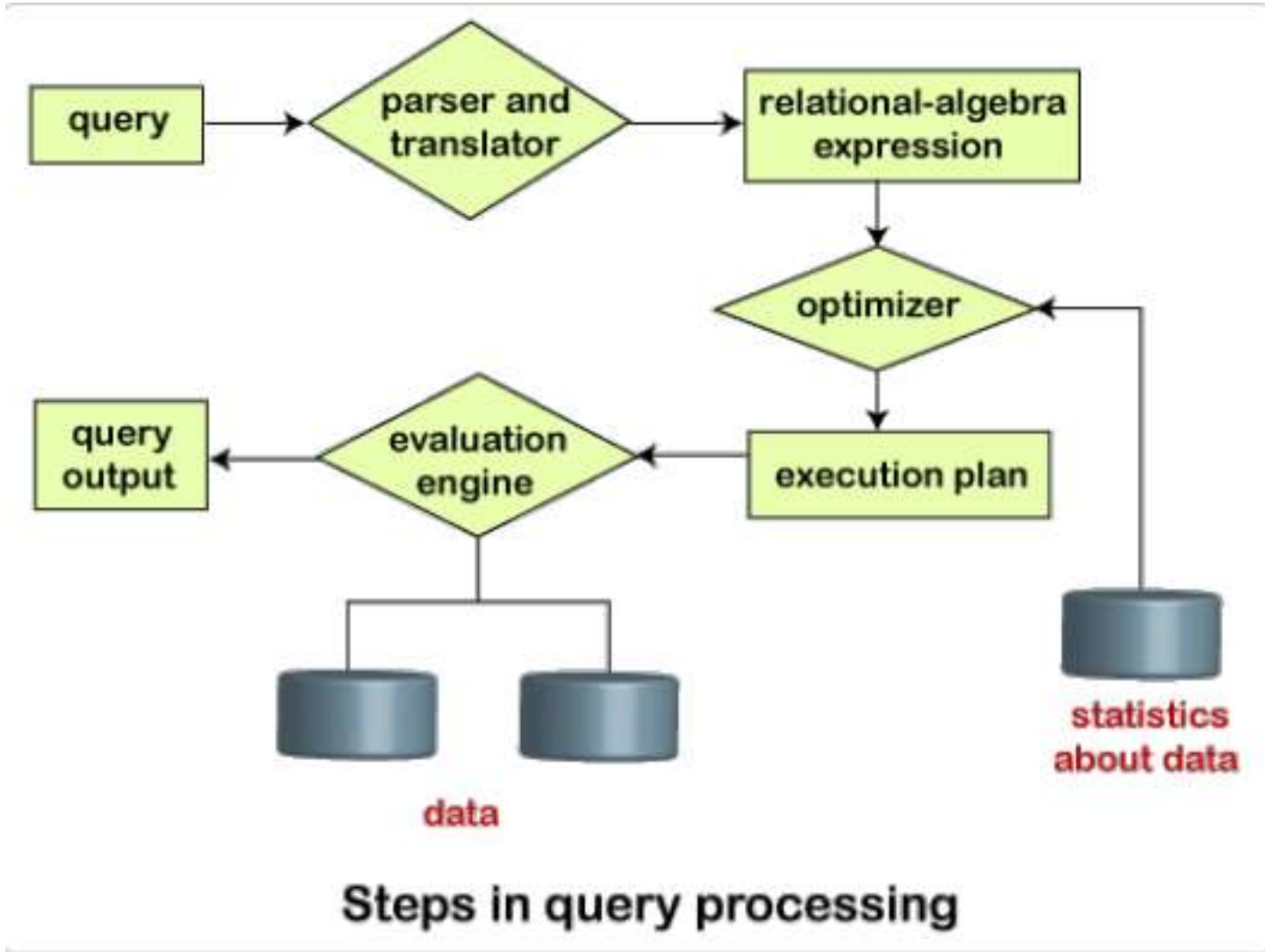
The user submits an SQL query.

Query Compilation:

The query is parsed, validated, and optimized into a query execution plan.

Plan Execution by the Execution Engine:

The execution engine processes the physical operators (e.g., table scan, join, sort) in the execution plan and retrieves the results.



Steps in query processing

Characterization of query processors

The characterization of query processors refers to how query processors are **classified, structured, and evaluated based on their functionality, architecture, and performance.**

Query processors are essential components of systems like **search engines, database management systems (DBMS), or natural language processing (NLP) platforms**.

Below are some key aspects of how query processors can be characterized:

1. Based on Query Processing Functionality

Query Parsing:
Converts the user query into a structured format for analysis. For example:

- In databases: Converts SQL queries into execution plans.
- In search engines: Translates natural language into tokens or search expressions.

Query Optimization:
Improves the efficiency of query execution by determining the most efficient strategy to retrieve data. Examples include:

- Using indexes in DBMS.
- Optimizing query terms for better search results in search engines.

Query Execution:
Handles the actual retrieval of data or execution of the query plan. This involves:

- Accessing relevant databases or indexes.
- Applying ranking or sorting algorithms to results.

2. Based on Input Type

Structured Queries:

- Process queries written in structured languages like SQL or SPARQL.
- Often used in relational databases or knowledge graphs.

Unstructured Queries:

- Handle free-form text or natural language input.
- Common in search engines or NLP-based systems.

Hybrid Queries:

- Combine structured and unstructured query processing.
- For example: "Find all articles written by John after 2020" combines structured filters with unstructured text search.

3. Based on Architecture

Centralized Query Processors:

- Operate on a single database or a local system.
- Simpler architecture but limited scalability.

Distributed Query Processors:

- Process queries across multiple nodes or distributed databases.
- Used in large-scale systems like Hadoop or Spark.

Federated Query Processors:

- Query across heterogeneous systems without central data integration.
- Common in scenarios involving multiple, independent databases.

Characterization of query processors

4. Based on Processing Paradigm

Batch Query Processors:

Process queries in bulk, often in a time-insensitive manner.

Examples: Data warehousing, analytics systems.

Real-Time Query Processors:

Process queries with low latency, often in real-time or near real-time.

Examples: Online search engines, recommendation systems.

5. Based on Optimization Techniques

Cost-Based Optimizers:

Use cost metrics (e.g., disk I/O, CPU time) to choose the best execution plan.

Common in relational database systems.

Heuristic-Based Optimizers:

Rely on **predefined rules or heuristics** to optimize queries quickly.

Often used for simpler systems where cost models are unnecessary.

AI/ML-Based Optimizers:

Leverage machine learning models to improve query optimization.

Used in advanced or experimental database systems.

6. Based on Use Case

Database Query Processors:

- Designed to interact with structured databases like MySQL, PostgreSQL, or MongoDB.
- Focus on data retrieval, manipulation, and transaction management.

Search Query Processors:

- Handle text-based or semantic search queries in systems like Google, Elasticsearch, or Solr.
- Emphasize relevance ranking, indexing, and content matching.

Natural Language Query Processors:

- Interpret and process queries written in human language.
- Found in virtual assistants, chatbots, and NLP systems.

Big Data Query Processors:

- Handle large-scale queries in distributed frameworks like Hadoop, Hive, or Spark SQL.
- Focus on parallelism and scalability.

7. Based on Performance Metrics

Response Time:
How quickly the processor returns results.

Throughput:
Number of queries the system can handle per unit of time.

Scalability:
Ability to handle increased data size or query volume.

Accuracy and Relevance:
For search systems, how well results match the user's intent.

Resource Utilization:
Efficiency of CPU, memory, and storage use during query processing.

8. Based on Query Complexity

Simple Query Processors:

Handle straightforward queries (e.g., single-table lookups or keyword searches).

Complex Query Processors:

Handle multi-table joins, nested queries, or advanced filters.

Examples include graph queries, machine learning model predictions, or deep NLP queries.

Layers of query processing

Understanding Query processing in **distributed database environments is very difficult instead of centralized database**, because there are many elements involved.

Main layers of Query Processing

Query processing involves 4 main layers:

- Query Decomposition
- Data Localization
- Global Query Optimization
- Distributed Execution

Main layers of Query Processing

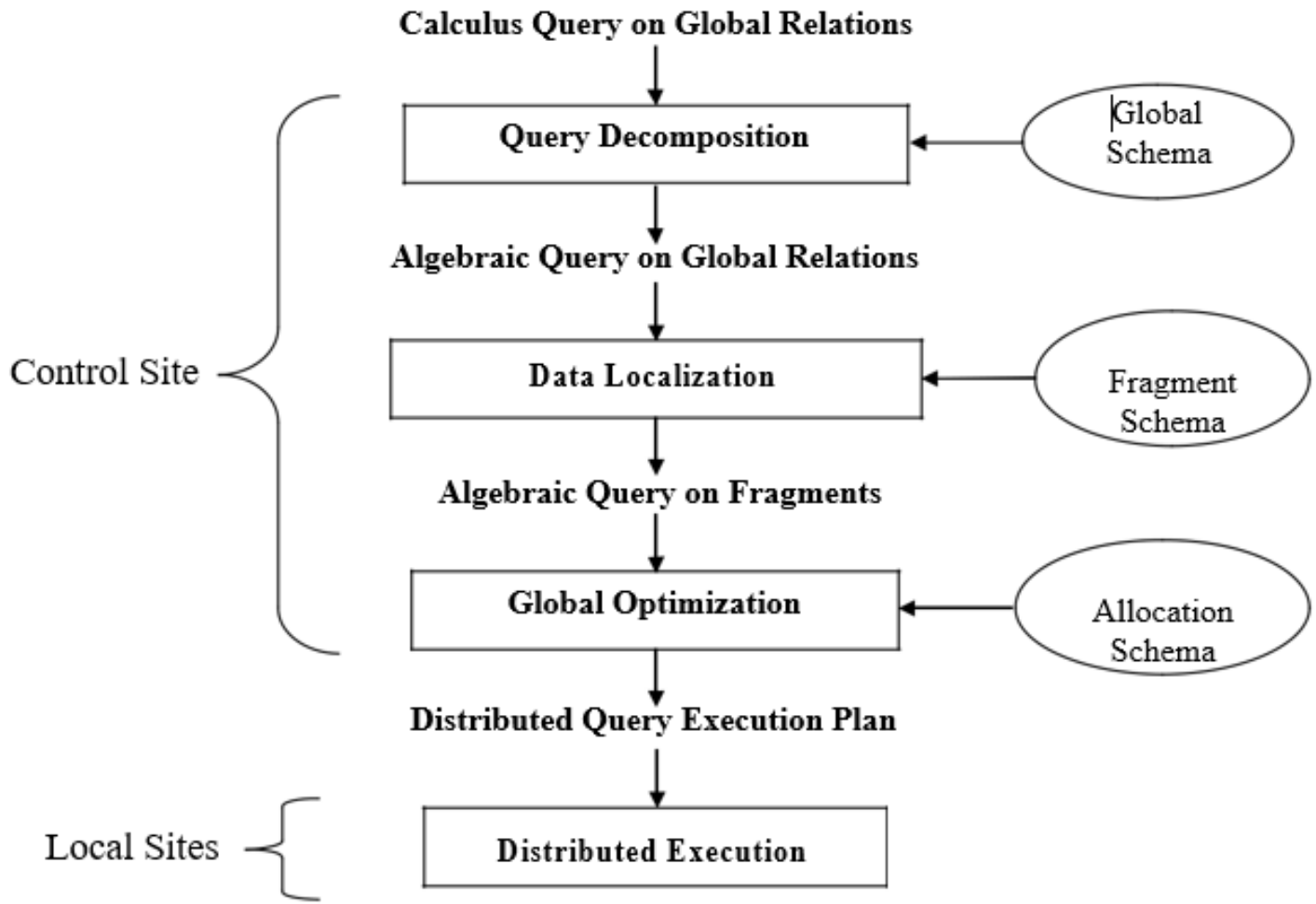


Fig. Generic Layering Scheme for Distributed Query Processing

Main layers of Query Processing

- The **input is a query on global data expressed in relational calculus**. This query is posed on **global (distributed) relations**, meaning that data distribution is hidden.

- The first three layers map the **input query** into an optimized distributed query execution plan. **They perform the functions of query decomposition, data localization, and global query optimization.**

- Query decomposition and data localization correspond to **query rewriting.**

- The **first three layers are performed by a central control site** and use schema information stored in the global directory.

- The **fourth layer performs distributed query execution by executing the plan and returns the answer to the query.** It is done by the local sites and the control site.

Query Decomposition

First, the **calculus query is rewritten in a normalized form** that is suitable for subsequent manipulation.

Second, **the normalized query is analyzed semantically so that incorrect queries are detected and rejected as early as possible**.

Third, the correct query is simplified. One way to simplify a query is to eliminate redundant predicates.

Fourth, the calculus query is restructured as an algebraic query. Several algebraic queries can be derived from the same calculus query, and that some algebraic queries are "better" than others. The quality of an algebraic query is defined in terms of expected performance.

Localization of Distributed Data

- Output of the first layer is an algebraic query on distributed relations which is input to the second layer.
- The main role of this layer is to **localize the query's data** using data distribution information.
- We know that relations are fragmented and stored in disjoint subsets, called fragments where each fragment is stored at different site.
- This layer **determines which fragments are involved in the query and transforms the distributed query into a fragment query**.
- Fourth, the calculus query is restructured as an algebraic query. **Several algebraic queries can be derived from the same calculus query,** and that some algebraic queries are "better" than others. The quality of an algebraic query is defined in terms of expected performance.

Global Query Optimization

The input to the third layer is a **fragment algebraic query**.

The goal of this layer is to **find an execution strategy** for the algebraic fragment query which is **close to optimal**.

Query optimization consists of

- Finding the best **ordering of operations** in the fragment query,
- Finding the **communication operations** which minimize a cost function.

  The cost function refers to computing resources such as **disk space, disk I/Os, buffer space, CPU cost, communication cost,** and so on.

  Query optimization is achieved through the **semi-join** operator instead of join operators.

Local Query Optimization(Distributed Execution)

- The last layer is performed by all the sites having fragments involved in the query.
- Each subquery, called a local query, is executing at one site. It is then optimized using the local schema of the site.

Examples of Query Decomposition

Example 1: SQL Query

- Query:

  SELECT name, department FROM employees WHERE department = 'HR' AND salary > 5000
- Decomposition:
  - Subquery 1: SELECT name, department FROM employees WHERE department = 'HR'
  - Subquery 2: SELECT name, department FROM employees WHERE salary > 5000
  - Final Step: Intersect results of Subquery 1 and Subquery 2.

Example 2: Natural Language Query

- Query:
  "Find all employees in the HR department who earn more than $5000."
- Decomposition:
  - Subtask 1: Identify employees in the HR department.
  - Subtask 2: Identify employees earning more than $5000.
  - Final Step: Combine both sets to identify matching employees.

Examples of Query Decomposition

Example 3: Distributed Query

- Query:
  SELECT COUNT(*) FROM orders WHERE region = 'North' AND order_date > '2024-01-01'
- Decomposition:
  - Subquery 1 (Node 1): Process data for region = 'North'.
  - Subquery 2 (Node 2): Filter data by order_date > '2024-01-01'.
  - Final Step: Aggregate results across nodes to compute the final count.

Distributed Query Optimization (DQO)

Distributed Query Optimization (DQO) refers to the process of optimizing queries that involve data **distributed across multiple locations, systems, or nodes in a distributed database environment**.

The **goal** is to minimize the cost of executing the query, which could involve

- **Reducing communication overhead**
- **Data transfer**
- **Disk I/O, or computation time.**

Optimization Techniques in Distributed Query Optimization

- **Data Fragmentation**:
  - Use **horizontal fragmentation** (dividing rows) or **vertical fragmentation** (dividing columns) to reduce data movement.
  - Example: Process "employees from HR" on the node storing the relevant fragment.
- **Join Optimization**:
  - Minimize data transfer by choosing the best join strategy:
    - **Semi-Joins**: Transfer only the **required attributes** for join conditions instead of the full tables.
    - **Bloom Filters**: Send a **compact representation of one table to reduce unnecessary data transfer**.

  Example: If joining tables Orders and Customers, move only the relevant parts of Orders to the node with Customers.
- **Query Shipping vs. Data Shipping**:
  - **Query Shipping**: **Move the query** to the node where the data resides, reducing data transfer.
  - **Data Shipping**: **Move data** to the query processing node, often used when processing requires combining data from multiple nodes.
- **Replicated Data**:
  - Use replicated copies of data to process queries locally and avoid remote data fetching.
- **Parallelism**:
  - Exploit the distributed system’s ability to process subqueries in parallel across multiple nodes.
- **Dynamic Optimization**:
  - Adapt query execution plans during runtime based on real-time data and resource availability.

Distributed Query Optimization Algorithms

Algorithms aim to minimize costs (e.g., communication, computation, and storage costs) while ensuring timely and accurate results. Below are the main algorithms and strategies used in distributed query optimization:

1. Dynamic Programming-Based Algorithms

Description:
These algorithms systematically evaluate all possible query execution plans and **select the one with the lowest cost.**

- Typically used for optimizing **joins**.
- Explores join orders (e.g., left-deep, right-deep, bushy trees).

Algorithm Steps:

- Generate all possible query plans.
- Estimate the cost of each plan (e.g., communication, I/O, CPU).
- Choose the plan with the minimum cost.

Advantages:

- Produces an optimal plan for small queries.

Disadvantages:

- Computationally expensive for large queries (exponential complexity).

Example: Optimize SELECT * FROM A, B, C WHERE A.id = B.id AND B.id = C.id by evaluating all possible join orders.

2. Heuristic-Based Algorithms

Description:
These algorithms use rules or heuristics to quickly generate a query execution plan without exploring all possible options.

Focuses on simplicity and speed over optimality.

Common Heuristics:

Reduce Data Early: Push selection and projection operations closer to the data source to minimize data transfer.

Minimize Communication Cost: Avoid transferring large datasets.

Join Ordering: Perform joins with smaller datasets first.

Advantages:

Faster execution and less overhead compared to dynamic programming.

Disadvantages:

May not always produce the optimal plan.

Example: Apply filters (WHERE clauses) before performing joins to reduce intermediate data size

ORDER	CLAUSE	FUNCTION
1	from	Choose and join tables to get base data.
2	where	Filters the base data.
3	group by	Aggregates the base data.
4	having	Filters the aggregated data.
5	select	Returns the final data.
6	order by	Sorts the final data.
7	limit	Limits the returned data to a row count.

3. Greedy Algorithms

Description:
These algorithms iteratively build a query execution plan by choosing the best option at each step (locally optimal choices).

- Unlike dynamic programming, greedy algorithms don't backtrack to reconsider earlier decisions.

Algorithm Steps:

- Start with the smallest subquery or data fragment.
- Choose the next operation or join that has the lowest incremental cost.
- Repeat until the full query is processed.

Advantages:

- Simple and efficient for queries with many fragments.

Disadvantages:

- May result in suboptimal plans due to lack of global optimization.

Example: Join the two smallest tables first and proceed iteratively.

4. Genetic Algorithms

Description:
These are evolutionary algorithms that mimic natural selection to generate and optimize query execution plans.

- Useful for complex queries with large search spaces.

Algorithm Steps:

- Generate an initial population of query plans.
- Evaluate the fitness (cost) of each plan.
- Apply crossover (combine parts of two plans) and mutation (random changes) to create new plans.
- Repeat until a sufficiently good plan is found.

Advantages:

- Can handle large, complex queries with multiple joins and constraints.

Disadvantages:

- Computationally expensive; may not guarantee the optimal plan.

Example: Optimize a query with 10+ joins across multiple nodes using a genetic algorithm to explore possible execution plans.

5. Simulated Annealing

Description:
This is a probabilistic algorithm inspired by the annealing process in metallurgy. It explores the search space of query plans by allowing occasional moves to higher-cost plans to escape local minima.

Algorithm Steps:

Start with an initial query plan and cost.

Generate a new plan by making small changes to the current plan.

Accept the new plan if it has a lower cost or with a probability that decreases over time (cooling schedule).

Repeat until convergence.

Advantages:

Avoids getting stuck in local minima.

Disadvantages:

May require tuning of parameters (e.g., cooling rate).

Example: Optimize a distributed query involving complex joins and filters by iteratively exploring new plans.

6. Iterative Improvement Algorithms

Description:
These algorithms start with an initial query execution plan and iteratively refine it by making incremental changes to reduce the cost.

Algorithm Steps:

Generate an initial plan (e.g., using heuristics).
Modify the plan (e.g., reorder joins, change data distribution).
If the modified plan has a lower cost, update the current plan.
Repeat until no further improvements can be made.

Advantages:

Simple to implement.

Disadvantages:

May converge to a local minimum rather than a global minimum.

Example: Start with a left-deep join plan and refine it to a bushy tree plan to reduce execution costs.

Example Transformation

Assume four tables: R1, R2, R3, R4.

- **Left-Deep Plan:**

(((R1 ⋈ R2) ⋈ R3) ⋈ R4)

Sequential joins limit flexibility, and intermediate results can grow large.

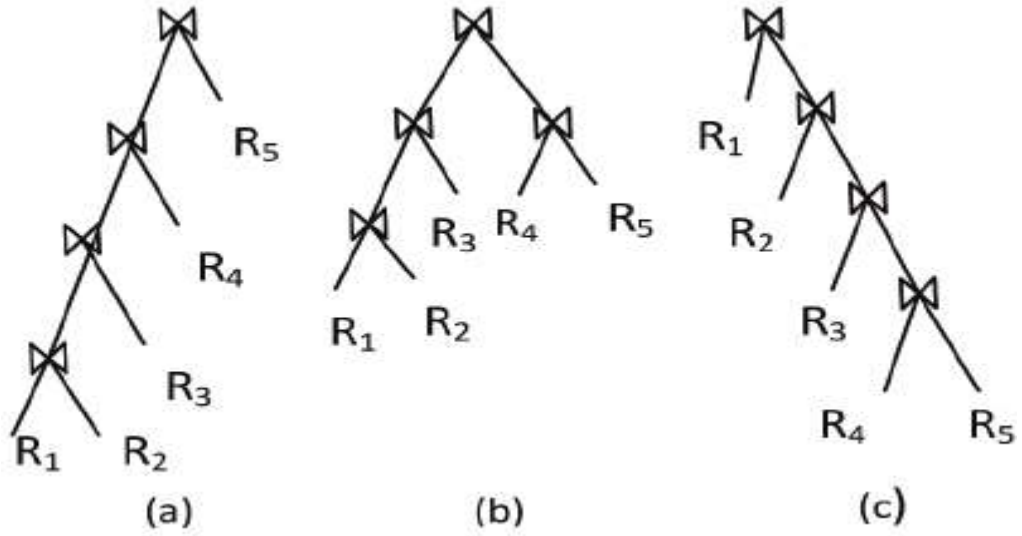
- **Bushy Tree Plan:**

First compute (R1 ⋈ R3) and (R2 ⋈ R4) in parallel, then join the results:

((R1 ⋈ R3) ⋈ (R2 ⋈ R4))

This approach reduces intermediate result sizes and allows for parallel processing.

a. Left Deep b. bush c. Right Deep



Starting with a Left-Deep Plan

A left-deep plan joins relations sequentially from left to right:

1. Join `R1` with `R2` → intermediate result: `T1 = R1 ⋈ R2`
2. Join `T1` with `R3` → intermediate result: `T2 = T1 ⋈ R3`
3. Join `T2` with `R4` → final result: `T2 ⋈ R4`

Visual representation (left-deep):

markdown

```
      ⋈
     / \
    ⋈   R4
   / \
  ⋈   R3
 / \
R1  R2
```

Right-Deep Plan Transformation

In a right-deep plan, the joins start from the rightmost relations, working upward. Here's how it would look:

1. Join `R3` with `R4` → intermediate result: `T1 = R3 ⋈ R4`
2. Join `R2` with `T1` → intermediate result: `T2 = R2 ⋈ T1`
3. Join `R1` with `T2` → final result: `R1 ⋈ T2`

Visual representation (right-deep):

markdown Copy

```
    ⋈
   / \
  R1  ⋈
     / \
    R2  ⋈
       / \
      R3  R4
```

Bushy Tree Plan Transformation

Instead of sequentially joining from one side, we can divide the relations into **independent subtrees** and join the results of those subtrees. This allows **parallel processing** and better control over intermediate result sizes.

Transformation Steps:

1. Divide the relations into **two groups** (subtrees). For example:
   - Subtree 1: `(R1 ⋈ R2)`
   - Subtree 2: `(R3 ⋈ R4)`
2. Compute these subtrees **independently and in parallel**:
   - `T1 = R1 ⋈ R2`
   - `T2 = R3 ⋈ R4`
3. Join the results of the subtrees:
   - `Final Result = T1 ⋈ T2`

Visual Representation (Bushy Tree):

markdown Copy

```
      ⋈
     / \
    ⋈   ⋈
   / \ / \
  R1 R2 R3 R4
```

7. Cost-Based Algorithms

Description:
These algorithms evaluate the cost of different execution plans based on a cost model that considers:

- Communication cost (data transfer between nodes).
- Computation cost (CPU time for joins, filters, etc.).
- Disk I/O cost (reading/writing data to/from disk).

Algorithm Steps:

- Estimate the cost of executing each operation on each node.
- Generate alternative plans for query execution.
- Select the plan with the lowest estimated cost.

Advantages:

- Produces high-quality plans for small to medium queries.

Disadvantages:

- Requires accurate cost models, which can be challenging in dynamic environments.

Example: Use a cost model to decide whether to ship data from Node A to Node B for a join or to process it locally.

8. Parallel Processing Algorithms

Description:
These algorithms optimize queries for parallel execution across multiple nodes, aiming to maximize throughput and minimize query latency.

Key Techniques:

- **Partitioned Parallelism**: Divide data into partitions and process them independently across nodes.
- **Pipelined Parallelism**: Overlap different stages of query execution across nodes.

Advantages:

- Exploits the full computational power of distributed systems.

Disadvantages:

- Requires careful coordination and scheduling.

Example: Execute a query with a `GROUP BY` clause by partitioning the data based on the grouping key and processing each partition in parallel.

9. Machine Learning-Based Algorithms

Description:
Use machine learning models to predict the cost of query execution plans and select the optimal one. These algorithms learn from past query executions to improve future optimizations.

Advantages:

- Adapts to dynamic environments and changing workloads.

Disadvantages:

- Requires a large dataset of past query execution plans for training.

Example: Use a neural network to predict the execution time of query plans and choose the best one.

Summary of Distributed Query Optimization Algorithms:

Algorithm Type	Strengths	Weaknesses
Dynamic Programming	Produces optimal plans	High computational cost
Heuristic-Based	Fast and simple	Suboptimal results in some cases
Greedy	Quick, locally optimal decisions	May not be globally optimal
Genetic Algorithms	Handles complex queries	Computationally expensive
Simulated Annealing	Escapes local minima	Requires parameter tuning
Iterative Improvement	Simple to implement	Risk of local minima
Cost-Based	Accurate cost estimation	Requires detailed cost models
Parallel Processing	Maximizes system throughput	High coordination overhead
ML-Based	Learns and adapts to workloads	Training data required

UNIT-III

Introduction to SQL: DDL commands (Create, Alter, Drop, Truncate and Rename), DML commands (insert, update, delete and select), TCL commands (Commit and Rollback), DCL commands (Grant and Revoke).
Integrity constraints: Primary key, Unique key, Not null, Check condition and Foreign key

SQL Functions: Number Functions, Character Functions and Date Functions.

Introduction to SQL

Structured Query Language (SQL) is used to interact with databases. It allows users to define, manipulate, control, and retrieve data. Below are the key SQL components:

---

1. SQL Commands

SQL commands are categorized into four main types:

a) Data Definition Language (DDL) Commands

DDL commands define and modify database structures.

- **CREATE** – Creates a new table, database, or object.

```
CREATE TABLE Employees (
    ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    Salary DECIMAL(10,2)
);
```

- **ALTER** – Modifies an existing table structure.

```
ALTER TABLE Employees ADD COLUMN Department VARCHAR(50);
```

- **DROP** – Deletes a table or database permanently.

```
DROP TABLE Employees;
```

- **TRUNCATE** – Deletes all rows in a table but keeps the structure.

```
TRUNCATE TABLE Employees;
```

- **RENAME** – Renames a table or column.

```
RENAME TABLE Employees TO Staff;
```

---

b) Data Manipulation Language (DML) Commands

DML commands handle data manipulation.

- **INSERT** – Adds new records.

  ```
  INSERT INTO Employees (ID, Name, Age, Salary) VALUES (1, 'Alice', 30, 50000);
  ```

- **UPDATE** – Modifies existing records.

  ```
  UPDATE Employees SET Salary = 55000 WHERE ID = 1;
  ```

- **DELETE** – Removes records from a table.

  ```
  DELETE FROM Employees WHERE ID = 1;
  ```

- **SELECT** – Retrieves records.

  ```
  SELECT * FROM Employees;
  ```

---

c) Transaction Control Language (TCL) Commands

TCL commands manage database transactions.

- **COMMIT** – Saves all changes permanently.

  ```
  COMMIT;
  ```

- **ROLLBACK** – Reverts changes to the last COMMIT.

  ```
  ROLLBACK;
  ```

---

d) Data Control Language (DCL) Commands

DCL commands control access permissions.

- **GRANT** – Gives access rights to users.

  ```
  GRANT SELECT, INSERT ON Employees TO User1;
  ```

- **REVOKE** – Removes access rights.

  ```
  REVOKE INSERT ON Employees FROM User1;
  ```

---

2. Integrity Constraints

Integrity constraints ensure data accuracy and consistency.

- **Primary Key** – Uniquely identifies each record.

```
CREATE TABLE Employees (
   ID INT PRIMARY KEY,
   Name VARCHAR(50)
);
```

- **Unique Key** – Ensures unique values in a column.

```
CREATE TABLE Employees (
   Email VARCHAR(100) UNIQUE
);
```

- **Not Null** – Prevents NULL values.

```
CREATE TABLE Employees (
   Name VARCHAR(50) NOT NULL
);
```

- **Check Condition** – Ensures specific conditions are met.

```
CREATE TABLE Employees (
   Age INT CHECK (Age >= 18)
);
```

- **Foreign Key** – Links tables together.

```
CREATE TABLE Orders (
   OrderID INT PRIMARY KEY,
   EmployeeID INT,
   FOREIGN KEY (EmployeeID) REFERENCES Employees(ID)
);
```

Syntax for defining **column-level integrity constraints** in Oracle SQL:

```
CREATE TABLE table_name (
   column_name DATA_TYPE [CONSTRAINT constraint_name] PRIMARY KEY,
   column_name DATA_TYPE [CONSTRAINT constraint_name] UNIQUE,
   column_name DATA_TYPE NOT NULL,
   column_name DATA_TYPE [CONSTRAINT constraint_name] CHECK (condition),
   column_name DATA_TYPE [CONSTRAINT constraint_name] REFERENCES
   parent_table(column_name)
);
```

Example:

```
CREATE TABLE employees (
   emp_id NUMBER CONSTRAINT emp_pk PRIMARY KEY,
   email VARCHAR2(100) CONSTRAINT emp_email_uk UNIQUE,
   emp_name VARCHAR2(50) NOT NULL,
   salary NUMBER CONSTRAINT emp_salary_ck CHECK (salary > 0),
   dept_id NUMBER CONSTRAINT emp_dept_fk REFERENCES departments(dept_id)
);
```

This ensures:

- emp_id is the **Primary Key**.
- email must be **unique**.
- emp_name **cannot be NULL**.
- salary must be **greater than 0**.
- dept_id **references** dept_id from the departments table.

Syntax for Defining Table-Level Integrity Constraints in Oracle SQL

In Oracle SQL, **table-level constraints** are defined **outside** the column definitions, typically at the end of the CREATE TABLE statement. These constraints can be named explicitly and are useful when applying constraints to multiple columns.

General Syntax:

```
CREATE TABLE table_name (
   column1 DATA_TYPE,
   column2 DATA_TYPE,
   ...,
   [CONSTRAINT constraint_name] PRIMARY KEY (column_name),
   [CONSTRAINT constraint_name] UNIQUE (column_name),
   [CONSTRAINT constraint_name] CHECK (condition),
   [CONSTRAINT constraint_name] FOREIGN KEY (column_name) REFERENCES
   parent_table (parent_column)
);
```

Example:

```
CREATE TABLE employees (
   emp_id NUMBER,
   emp_name VARCHAR2(50) NOT NULL,
   email VARCHAR2(100),
   salary NUMBER,
   dept_id NUMBER,

   -- Table-Level Constraints
   CONSTRAINT emp_pk PRIMARY KEY (emp_id),
   CONSTRAINT emp_email_uk UNIQUE (email),
   CONSTRAINT emp_salary_ck CHECK (salary > 0),
   CONSTRAINT emp_dept_fk FOREIGN KEY (dept_id) REFERENCES
   departments(dept_id)
);
```

Example of a Composite Primary Key Using Table-Level Constraint

```
CREATE TABLE order_details (
   order_id NUMBER,
   product_id NUMBER,
   quantity NUMBER,

   -- Composite Primary Key
   CONSTRAINT order_details_pk PRIMARY KEY (order_id, product_id)
);
```

Understanding ON DELETE CASCADE in Oracle SQL

The ON DELETE CASCADE clause is used in **foreign key constraints** to automatically delete child records when the referenced parent record is deleted. This helps maintain referential integrity.

Syntax for ON DELETE CASCADE (Table-Level)

```
CREATE TABLE child_table (
   child_id NUMBER PRIMARY KEY,
   parent_id NUMBER,
   CONSTRAINT fk_parent FOREIGN KEY (parent_id)
      REFERENCES parent_table(parent_id)
      ON DELETE CASCADE
);
```

Example: Parent-Child Relationship with ON DELETE CASCADE

1. Create Parent Table

```
CREATE TABLE departments (
   dept_id NUMBER PRIMARY KEY,
   dept_name VARCHAR2(100)
);
```

2. Create Child Table with **ON DELETE CASCADE**

```
CREATE TABLE employees (
   emp_id NUMBER PRIMARY KEY,
   emp_name VARCHAR2(50),
   dept_id NUMBER,
   CONSTRAINT emp_dept_fk FOREIGN KEY (dept_id)
      REFERENCES departments(dept_id)
      ON DELETE CASCADE
);
```

How ON DELETE CASCADE Works

1. Insert data into **departments** (Parent Table):

   ```
   INSERT INTO departments VALUES (1, 'HR');
   INSERT INTO departments VALUES (2, 'Finance');
   ```

2. Insert data into **employees** (Child Table):

   ```
   INSERT INTO employees VALUES (101, 'Alice', 1);
   INSERT INTO employees VALUES (102, 'Bob', 2);
   ```

3. Delete a department:

   DELETE FROM departments WHERE dept_id = 1;

   - This will **automatically delete** employees who belong to department **1** (Alice will be deleted).

```
-- the table-level primary key
CREATE TABLE SAMP (
   CLASS_CODE CHAR(7),
   DAY NUMBER(9),
   STARTING_TIME TIMESTAMP,
   ENDING_TIME TIMESTAMP,
   PRIMARY KEY (CLASS_CODE, DAY)
)


INSERT ALL
   INTO SAMP (CLASS_CODE, DAY, STARTING_TIME, ENDING_TIME)
   VALUES ('MATH101', 1, TIMESTAMP '2024-02-05 08:00:00', TIMESTAMP '2024-02-05 09:30:00')

   INTO SAMP (CLASS_CODE, DAY, STARTING_TIME, ENDING_TIME)
   VALUES ('PHY202', 2, TIMESTAMP '2024-02-06 10:00:00', TIMESTAMP '2024-02-06 11:30:00')

   INTO SAMP (CLASS_CODE, DAY, STARTING_TIME, ENDING_TIME)
   VALUES ('CS303', 3, TIMESTAMP '2024-02-07 13:00:00', TIMESTAMP '2024-02-07 14:30:00')
SELECT 1 FROM DUAL


-- column-level primary key constraint named OUT_TRAY_PK:

CREATE TABLE OUT_TRAY
            (
            SENT TIMESTAMP,
            DESTINATION CHAR(8),
            SUBJECT CHAR(64) CONSTRAINT OUT_TRAY_PK PRIMARY KEY,
            NOTE_TEXT VARCHAR2(3000)
  )

-- Use a column-level constraint for an arithmetic check
-- Use a table-level constraint


CREATE TABLE EMP
            (
            EMPNO CHAR(6) CONSTRAINT EMP_PK PRIMARY KEY,
            FIRSTNME CHAR(12) NOT NULL,
            MIDINIT vARCHAR(12) NOT NULL,
            LASTNAME VARCHAR(15) NOT NULL,
            SALARY DECIMAL(9,2) CONSTRAINT SAL_CK CHECK (SALARY >= 10000),
            BONUS DECIMAL(9,2),
            TAX DECIMAL(9,2),
            CONSTRAINT BONUS_CK CHECK (BONUS > TAX)
            )
```

```
-- use a check constraint to allow only appropriate
CREATE TABLE FLIGHTS
            (
            FLIGHT_ID CHAR(6) NOT NULL ,
            SEGMENT_NUMBER NUMBER(9) NOT NULL ,
            ORIG_AIRPORT CHAR(3),
            DEPART_TIME TIMESTAMP,
            DEST_AIRPORT CHAR(3),
            ARRIVE_TIME TIMESTAMP,
            MEAL CHAR(1) CONSTRAINT MEAL_CONSTRAINT
      CHECK (MEAL IN ('B','L', 'D', 'S')),
            PRIMARY KEY (FLIGHT_ID, SEGMENT_NUMBER)
            )

INSERT ALL
   INTO FLIGHTS (FLIGHT_ID, SEGMENT_NUMBER, ORIG_AIRPORT, DEPART_TIME,
DEST_AIRPORT, ARRIVE_TIME, MEAL)
   VALUES ('FL1001', 1, 'JFK', TIMESTAMP '2024-02-10 08:00:00', 'LAX', TIMESTAMP '2024-02-10
11:00:00', 'B')

   INTO FLIGHTS (FLIGHT_ID, SEGMENT_NUMBER, ORIG_AIRPORT, DEPART_TIME,
DEST_AIRPORT, ARRIVE_TIME, MEAL)
   VALUES ('FL1001', 2, 'LAX', TIMESTAMP '2024-02-10 13:00:00', 'SFO', TIMESTAMP '2024-02-10
14:30:00', 'L')

   INTO FLIGHTS (FLIGHT_ID, SEGMENT_NUMBER, ORIG_AIRPORT, DEPART_TIME,
DEST_AIRPORT, ARRIVE_TIME, MEAL)
   VALUES ('FL2002', 1, 'ORD', TIMESTAMP '2024-02-11 09:15:00', 'DFW', TIMESTAMP '2024-02-11
11:45:00', 'D')

   INTO FLIGHTS (FLIGHT_ID, SEGMENT_NUMBER, ORIG_AIRPORT, DEPART_TIME,
DEST_AIRPORT, ARRIVE_TIME, MEAL)
   VALUES ('FL3003', 1, 'ATL', TIMESTAMP '2024-02-12 16:30:00', 'MIA', TIMESTAMP '2024-02-12
18:30:00', 'S')
SELECT 1 FROM DUAL
```

-- create a table with a table-level primary key constraint

```
CREATE TABLE FLIGHTS
            (
            FLIGHT_ID CHAR(6) NOT NULL ,
            SEGMENT_NUMBER NUMBER(9) NOT NULL ,
            ORIG_AIRPORT CHAR(3),
            DEPART_TIME TIMESTAMP,
            DEST_AIRPORT CHAR(3),
            ARRIVE_TIME TIMESTAMP,
            MEAL CHAR(1) CONSTRAINT MEAL_CONSTRAINT CHECK (MEAL IN ('B',
    'L', 'D', 'S')),
            PRIMARY KEY (FLIGHT_ID, SEGMENT_NUMBER)
            )
```

-- Table-level foreign key constraint

```
CREATE TABLE FLTAVAIL
            (
            FLIGHT_ID CHAR(6) NOT NULL,
            SEGMENT_NUMBER NUMBER(9) NOT NULL,
            FLIGHT_DATE DATE NOT NULL,
            ECONOMY_SEATS_TAKEN NUMBER(9),
            BUSINESS_SEATS_TAKEN NUMBER(9),
            FIRSTCLASS_SEATS_TAKEN NUMBER(9),
            CONSTRAINT FLTS_FK FOREIGN KEY (FLIGHT_ID, SEGMENT_NUMBER)
            REFERENCES Flights (FLIGHT_ID, SEGMENT_NUMBER)
            )


ON DELETE CASCADE

CREATE TABLE CITIES (
   CITY_ID NUMBER(9) CONSTRAINT CITIES_PK PRIMARY KEY,
   CITY_NAME VARCHAR2(50) NOT NULL,
   COUNTRY VARCHAR2(50) NOT NULL
)


CREATE TABLE METROPOLITAN (
   HOTEL_ID NUMBER(9) CONSTRAINT HOTELS_PK PRIMARY KEY,
   HOTEL_NAME VARCHAR2(40) NOT NULL,
   CITY_ID NUMBER(9) CONSTRAINT METRO_FK REFERENCES CITIES(CITY_ID)
)
CREATE TABLE CONDOS (
   CONDO_ID NUMBER(9) CONSTRAINT condos_PK PRIMARY KEY,
   CONDO_NAME VARCHAR2(40) NOT NULL,
   CITY_ID NUMBER(9) CONSTRAINT city_foreign_key REFERENCES
   CITIES(CITY_ID) ON DELETE CASCADE
)
```

3. SQL Functions

SQL functions perform operations on data.

a) Number Functions

- **ABS()** – Returns absolute value.

  ```
  SELECT ABS(-10);
  ```

- **ROUND()** – Rounds a number.

  ```
  SELECT ROUND(123.456, 2);
  ```

- **CEIL() & FLOOR()** – Returns next or previous integer.

  ```
  SELECT CEIL(12.3), FLOOR(12.7);
  ```

b) Character Functions

- **UPPER() & LOWER()** – Converts case.

  ```
  SELECT UPPER('hello'), LOWER('WORLD');
  ```

- **LENGTH()** – Returns string length.

  ```
  SELECT LENGTH('SQL');
  ```

- **CONCAT()** – Joins strings.

  ```
  SELECT CONCAT('Hello', ' World');
  ```

c) Date Functions

- **NOW()** – Returns current timestamp.

  ```
  SELECT NOW();
  ```

- **DATEDIFF()** – Finds difference between dates.

  ```
  SELECT DATEDIFF('2025-01-01', '2024-01-01');
  ```

- **DATE_ADD() & DATE_SUB()** – Adds or subtracts days.

  ```
  SELECT DATE_ADD(NOW(), INTERVAL 7 DAY);
  ```

4. Aggregate/Group Functions

Used for calculations on data sets.

- **AVG()** – Returns average value.

  SELECT AVG(Salary) FROM Employees;

- **COUNT()** – Counts records.

  SELECT COUNT(*) FROM Employees;

- **MAX() & MIN()** – Returns maximum and minimum values.

  SELECT MAX(Salary), MIN(Salary) FROM Employees;

- **SUM()** – Returns sum of values.

  SELECT SUM(Salary) FROM Employees;

Common SQL Clauses for Group Functions

- **GROUP BY** – Groups records based on a column.

  SELECT Department, AVG(Salary) FROM Employees GROUP BY Department;

- **HAVING** – Filters grouped records.

  SELECT Department, AVG(Salary) FROM Employees GROUP BY Department HAVING AVG(Salary) > 50000;

- **ORDER BY** – Sorts records.

  SELECT * FROM Employees ORDER BY Salary DESC;

Number Functions

1. ROUND(n, d)

- Rounds a number **n** to **d** decimal places.
- If **d** is omitted, it rounds to the nearest integer.

Example:

```
SELECT ROUND(123.456, 2) FROM DUAL;  -- Output: 123.46
SELECT ROUND(123.456) FROM DUAL;     -- Output: 123
```

2. TRUNC(n, d)

- Truncates a number **n** to **d** decimal places without rounding.
- If **d** is omitted, it removes the decimal portion.

Example:

```
SELECT TRUNC(123.456, 2) FROM DUAL;  -- Output: 123.45
SELECT TRUNC(123.456) FROM DUAL;     -- Output: 123
```

---

3. CEIL(n)

- Returns the smallest integer **greater than or equal** to **n**.

Example:

```
SELECT CEIL(12.3) FROM DUAL;  -- Output: 13
SELECT CEIL(-12.3) FROM DUAL; -- Output: -12
```

---

4. FLOOR(n)

- Returns the largest integer **less than or equal** to **n**.

Example:

```
SELECT FLOOR(12.7) FROM DUAL;  -- Output: 12
SELECT FLOOR(-12.7) FROM DUAL; -- Output: -13
```

---

5. MOD(n, m)

- Returns the remainder of **n divided by m**.

Example:

```
SELECT MOD(10, 3) FROM DUAL;  -- Output: 1  (10 ÷ 3 = 3 remainder 1)
```

---

6. POWER(n, m)

- Returns **n** raised to the power of **m**.

Example:

```
SELECT POWER(2, 3) FROM DUAL;  -- Output: 8 (2³ = 8)
```

---

7. SQRT(n)

- Returns the square root of **n**.

Example:

```
SELECT SQRT(16) FROM DUAL;  -- Output: 4
```

---

8. ABS(n)

- Returns the **absolute value** of **n**.

Example:

```
SELECT ABS(-25) FROM DUAL;  -- Output: 25
```

---

9. SIGN(n)

- Returns:
  - **1** if **n > 0**
  - **0** if **n = 0**
  - **-1** if **n < 0**

Example:

```
SELECT SIGN(-10) FROM DUAL;  -- Output: -1
SELECT SIGN(0) FROM DUAL;    -- Output: 0
SELECT SIGN(10) FROM DUAL;   -- Output: 1
```

---

10. GREATEST(n1, n2, ..., nn)

- Returns the largest value from the given list of numbers.

Example:

```
SELECT GREATEST(10, 20, 30, 15) FROM DUAL;  -- Output: 30
```

11. LEAST(n1, n2, ..., nn)

- Returns the smallest value from the given list of numbers.

Example:

```
SELECT LEAST(10, 20, 30, 15) FROM DUAL;  -- Output: 10
```

---

12. EXP(n)

- Returns **e^n** (exponential function).

Example:

```
SELECT EXP(1) FROM DUAL;  -- Output: 2.7182818 (≈ e)
```

---

13. LN(n)

- Returns the **natural logarithm** (base **e**) of **n**.

Example:

```
SELECT LN(2.7182818) FROM DUAL;  -- Output: 1
```

---

14. LOG(m, n)

- Returns the **logarithm of n to base m**.

Example:

```
SELECT LOG(10, 1000) FROM DUAL;  -- Output: 3 (log10(1000) = 3)
```

---

15. COS(n), SIN(n), TAN(n)

- Returns the **cosine, sine, or tangent** of **n** (in radians).

Example:

```
SELECT COS(0) FROM DUAL;  -- Output: 1
SELECT SIN(PI()/2) FROM DUAL;  -- Output: 1
SELECT TAN(PI()/4) FROM DUAL;  -- Output: 1
```

---

16. PI()

- Returns the constant **π (pi ≈ 3.1415926535)**.

Example:

```
SELECT PI() FROM DUAL;  -- Output: 3.1415926535
```

Character Functions in Oracle

Oracle provides several **character functions** to manipulate and process string data. These functions help in formatting, modifying, and analyzing character strings.

1. UPPER()

- Converts a string to **uppercase**.

Example:

```
SELECT UPPER('oracle sql') FROM DUAL;  -- Output: ORACLE SQL
```

---

2. LOWER()

- Converts a string to **lowercase**.

Example:

```
SELECT LOWER('ORACLE SQL') FROM DUAL;  -- Output: oracle sql
```

---

3. INITCAP()

- Converts the first letter of each word to **uppercase**, and the rest to lowercase.

Example:

```
SELECT INITCAP('oracle sql database') FROM DUAL;  -- Output: Oracle Sql Database
```

---

4. LENGTH()

- Returns the **length of a string** (number of characters, including spaces).

Example:

```
SELECT LENGTH('Oracle') FROM DUAL;  -- Output: 6
```

---

5. CONCAT()

- Joins two strings together (same as || operator).

Example:

```
SELECT CONCAT('Oracle ', 'SQL') FROM DUAL;  -- Output: Oracle SQL
-- OR
SELECT 'Oracle ' || 'SQL' FROM DUAL;  -- Output: Oracle SQL
```

---

6. SUBSTR()

- Extracts a substring from a string.
- **Syntax:** SUBSTR(string, start_position, length)

Example:

```
SELECT SUBSTR('Oracle SQL', 1, 6) FROM DUAL;  -- Output: Oracle
SELECT SUBSTR('Database', 5, 3) FROM DUAL;    -- Output: bas
```

Note: start_position is **1-based** (starts at 1).

---

7. INSTR()

- Returns the position of a **substring** within a string.
- **Syntax:** INSTR(string, substring, start_position, occurrence)

Example:

```
SELECT INSTR('Oracle SQL', 'SQL') FROM DUAL;  -- Output: 8
SELECT INSTR('hello world', 'o', 1, 2) FROM DUAL;  -- Output: 8 (2nd occurrence of 'o')
```

---

8. LPAD()

- Left-pads a string with a specified character up to a certain length.

Example:

```
SELECT LPAD('SQL', 6, '*') FROM DUAL;  -- Output: ***SQL
```

---

9. RPAD()

- Right-pads a string with a specified character up to a certain length.

Example:

```
SELECT RPAD('SQL', 6, '*') FROM DUAL;  -- Output: SQL***
```

---

10. TRIM()

- Removes **leading and trailing spaces** (or a specific character).

Example:

```
SELECT TRIM('  Oracle  ') FROM DUAL;  -- Output: Oracle
SELECT TRIM('O' FROM 'Oracle') FROM DUAL;  -- Output: racle
```

---

11. LTRIM()

- Removes **leading spaces** or a specified character.

Example:

```
SELECT LTRIM('   Oracle') FROM DUAL;  -- Output: Oracle
SELECT LTRIM('000123', '0') FROM DUAL;  -- Output: 123
```

---

12. RTRIM()

- Removes **trailing spaces** or a specified character.

Example:

```
SELECT RTRIM('Oracle   ') FROM DUAL;  -- Output: Oracle
SELECT RTRIM('123000', '0') FROM DUAL;  -- Output: 123
```

---

13. REPLACE()

- Replaces all occurrences of a substring with another substring.

Example:

```
SELECT REPLACE('Oracle SQL', 'SQL', 'Database') FROM DUAL;
-- Output: Oracle Database
```

---

14. TRANSLATE()

- Replaces **multiple characters** in a string.
- **Syntax:** TRANSLATE(string, from_chars, to_chars)

Example:

```
SELECT TRANSLATE('SQL123', '123', 'ABC') FROM DUAL;
-- Output: SQLABC
```

Note: It replaces **character-by-character**, not substrings.

---

15. ASCII()

- Returns the **ASCII code** of a character.

Example:

```
SELECT ASCII('A') FROM DUAL;  -- Output: 65
```

---

16. CHR()

- Converts an **ASCII code** to a character.

Example:

```
SELECT CHR(65) FROM DUAL;  -- Output: A
```

---

17. SOUNDEX()

- Returns a phonetic representation of a string.

Example:

```
SELECT SOUNDEX('Smith') FROM DUAL;  -- Output: S530
SELECT SOUNDEX('Smyth') FROM DUAL;  -- Output: S530
```

Useful for finding similar-sounding words.

---

18. REGEXP_LIKE()

- Checks if a string matches a **regular expression pattern**.

Example:

```
SELECT REGEXP_LIKE('hello123', '[0-9]') FROM DUAL;  -- Output: TRUE
```

---

19. REGEXP_SUBSTR()

- Extracts a substring using **regular expressions**.

Example:

```
SELECT REGEXP_SUBSTR('Email: john.doe@example.com', '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}') FROM DUAL;
-- Output: john.doe@example.com
```

---

20. REGEXP_REPLACE()

- Replaces substrings using **regular expressions**.

Example:

```
SELECT REGEXP_REPLACE('Phone: (123) 456-7890', '\D', '') FROM DUAL;
-- Output: 1234567890 (removes non-digits)
```

Date Functions in Oracle

Oracle provides several **date functions** to handle **date and time** values efficiently. These functions allow you to manipulate, format, and extract information from date/time data.

---

1. SYSDATE

- Returns the **current date and time** of the database system.

Example:

```
SELECT SYSDATE FROM DUAL;
-- Output: 24-FEB-2025 10:30:15
```

---

2. CURRENT_DATE

- Returns the **current date and time** based on the **session's time zone**.

Example:

```
SELECT CURRENT_DATE FROM DUAL;
-- Output: 24-FEB-2025 10:30:15
```

---

3. SYSTIMESTAMP

- Returns the **current timestamp** including fractional seconds and time zone.

Example:

```
SELECT SYSTIMESTAMP FROM DUAL;
-- Output: 24-FEB-2025 10:30:15.123456 AM +05:30
```

---

4. ADD_MONTHS(date, n)

- Adds **n months** to a given date.

Example:

```
SELECT ADD_MONTHS(SYSDATE, 3) FROM DUAL;
-- Output: 24-MAY-2025 (3 months added)
```

---

5. MONTHS_BETWEEN(date1, date2)

- Returns the number of **months** between **date1 and date2**.

Example:

```
SELECT MONTHS_BETWEEN('24-DEC-2024', '24-JUN-2024') FROM DUAL;
-- Output: 6 (months between)
```

---

6. LAST_DAY(date)

- Returns the **last day of the month** for the given date.

Example:

```
SELECT LAST_DAY(SYSDATE) FROM DUAL;
-- Output: 29-FEB-2025 (since 2025 is a leap year)
```

---

7. NEXT_DAY(date, 'day')

- Returns the **next occurrence** of a specified weekday.

Example:

```
SELECT NEXT_DAY(SYSDATE, 'SUNDAY') FROM DUAL;
-- Output: 02-MAR-2025 (next Sunday)
```

---

8. TRUNC(date, format)

- Truncates the date to the specified format.

Example:

```
SELECT TRUNC(SYSDATE, 'MONTH') FROM DUAL;
-- Output: 01-FEB-2025 (truncated to first day of the month)

SELECT TRUNC(SYSDATE, 'YEAR') FROM DUAL;
-- Output: 01-JAN-2025 (truncated to first day of the year)
```

---

9. ROUND(date, format)

- Rounds the date to the nearest format unit.

Example:

```
SELECT ROUND(TO_DATE('15-FEB-2025'), 'MONTH') FROM DUAL;
-- Output: 01-MAR-2025 (rounded to nearest month)

SELECT ROUND(TO_DATE('15-JUL-2025'), 'YEAR') FROM DUAL;
-- Output: 01-JAN-2026 (rounded to next year)
```

---

10. EXTRACT()

- Extracts a specific part (**year, month, day, hour, minute, second**) from a date.

Example:

```
SELECT EXTRACT(YEAR FROM SYSDATE) FROM DUAL;
-- Output: 2025

SELECT EXTRACT(MONTH FROM SYSDATE) FROM DUAL;
-- Output: 2 (February)
```

---

11. TO_DATE()

- Converts a string into a **date**.

Example:

```
SELECT TO_DATE('24-FEB-2025', 'DD-MON-YYYY') FROM DUAL;
-- Output: 24-FEB-25
```

---

12. TO_CHAR(date, format)

- Converts a date into a formatted string.

Example:

```
SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY HH24:MI:SS') FROM DUAL;
-- Output: 24-FEB-2025 10:30:15

SELECT TO_CHAR(SYSDATE, 'DAY') FROM DUAL;
-- Output: MONDAY
```

Common Date Formats:

Format	Description	Example
YYYY	4-digit year	2025
YY	Last 2 digits of year	25
MM	Month (01-12)	02
MON	Abbreviated month	FEB
MONTH	Full month name	FEBRUARY
DD	Day of month (01-31)	24
D	Day of the week (1-7)	2 (Monday)
DAY	Full day name	MONDAY
HH24	Hour (24-hour format)	14
MI	Minutes	45
SS	Seconds	30

---

13. CURRENT_TIMESTAMP

- Returns the **current timestamp** including time zone.

Example:

```
SELECT CURRENT_TIMESTAMP FROM DUAL;
-- Output: 24-FEB-2025 10:30:15.123 AM +05:30
```

---

14. LOCALTIMESTAMP

- Returns the current timestamp **without** the time zone.

Example:

```
SELECT LOCALTIMESTAMP FROM DUAL;
-- Output: 24-FEB-2025 10:30:15.123 AM
```

---

15. DBTIMEZONE

- Returns the **time zone of the database**.

Example:

```
SELECT DBTIMEZONE FROM DUAL;
-- Output: +00:00
```

---

16. SESSIONTIMEZONE

- Returns the **time zone of the session**.

Example:

```
SELECT SESSIONTIMEZONE FROM DUAL;
-- Output: +05:30
```

---

17. INTERVAL Arithmetic

- You can perform arithmetic operations using **INTERVAL**.

Example:

```
SELECT SYSTIMESTAMP + INTERVAL '10' DAY FROM DUAL;
-- Output: 06-MAR-2025 10:30:15
```

Conversion Functions in Oracle

Oracle provides several **conversion functions** that allow you to convert data from one type to another, such as **string to date, number to string, etc.** These functions ensure data consistency and facilitate data manipulation.

---

1. TO_CHAR()

- Converts **dates** or **numbers** into a **formatted string**.

For Dates:

```
SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY HH24:MI:SS') FROM DUAL;
-- Output: 24-FEB-2025 14:45:30
```

For Numbers:

```
SELECT TO_CHAR(12345.67, '99999.99') FROM DUAL;
-- Output: 12345.67
```

Common Formats:

Format	Description	Example
YYYY	4-digit year	2025
MM	Month (01-12)	02
MON	Abbreviated month	FEB
DD	Day (01-31)	24
HH24:MI:SS	24-hour time	14:45:30
9999.99	Number format	1234.56

---

2. TO_NUMBER()

- Converts a **string** into a **number**.

Example:

```
SELECT TO_NUMBER('12345.67', '99999.99') FROM DUAL;
-- Output: 12345.67
```

Example with Currency Symbols:

```
SELECT TO_NUMBER('$1,234.56', '$9,999.99') FROM DUAL;
-- Output: 1234.56
```

---

3. TO_DATE()

- Converts a **string** into a **date**.

Example:

```
SELECT TO_DATE('24-FEB-2025', 'DD-MON-YYYY') FROM DUAL;
-- Output: 24-FEB-25
```

---

4. CAST()

- Converts one **data type** into another.

Example:

```
SELECT CAST('2025-02-24' AS DATE) FROM DUAL;
-- Output: 24-FEB-25

SELECT CAST(12345 AS VARCHAR2(10)) FROM DUAL;
-- Output: '12345'
```

---

5. CONVERT()

- Converts a string from one **character set** to another.

Example:

```
SELECT CONVERT('HELLO', 'AL32UTF8', 'WE8MSWIN1252') FROM DUAL;
```

Converts HELLO from WE8MSWIN1252 (Western European) to AL32UTF8 (Unicode).

---

6. NVL()

- Replaces **NULL** with a specified value.

Example:

```
SELECT NVL(NULL, 'Default Value') FROM DUAL;
-- Output: Default Value
```

---

7. NVL2()

- Checks if a value is **NULL**:
  - If **NOT NULL**, returns the first value.
  - If **NULL**, returns the second value.

Example:

```
SELECT NVL2(NULL, 'Value Present', 'Value Missing') FROM DUAL;
-- Output: Value Missing
```

---

8. NULLIF()

- Compares two values:
  - If **equal**, returns **NULL**.
  - If **not equal**, returns the first value.

Example:

```
SELECT NULLIF(10, 10) FROM DUAL;
-- Output: NULL

SELECT NULLIF(10, 20) FROM DUAL;
-- Output: 10
```

---

9. COALESCE()

- Returns the **first non-null value** from a list.

Example:

```
SELECT COALESCE(NULL, NULL, 'Oracle', 'SQL') FROM DUAL;
-- Output: Oracle
```

---

10. HEXTORAW()

- Converts a **hexadecimal string** to **raw data**.

Example:

```
SELECT HEXTORAW('48656C6C6F') FROM DUAL;
-- Output: Hello
```

---

11. RAWTOHEX()

- Converts **raw data** to a **hexadecimal string**.

Example:

```
SELECT RAWTOHEX('Hello') FROM DUAL;
-- Output: 48656C6C6F
```

---

5. Joins

Joins combine records from multiple tables.

- **INNER JOIN** – Returns matching records.

  ```
  SELECT Employees.Name, Orders.OrderID
  FROM Employees
  INNER JOIN Orders ON Employees.ID = Orders.EmployeeID;
  ```

- **LEFT JOIN (Outer Join)** – Returns all records from left table and matching records from right table.

  ```
  SELECT Employees.Name, Orders.OrderID
  FROM Employees
  LEFT JOIN Orders ON Employees.ID = Orders.EmployeeID;
  ```

- **RIGHT JOIN (Outer Join)** – Returns all records from right table and matching records from left table.

  ```
  SELECT Employees.Name, Orders.OrderID
  FROM Employees
  RIGHT JOIN Orders ON Employees.ID = Orders.EmployeeID;
  ```

- **SELF JOIN** – Joins a table to itself.

  ```
  SELECT A.Name, B.Name AS Manager
  FROM Employees A, Employees B
  WHERE A.ManagerID = B.ID;
  ```

---

6. Subqueries / Nested Queries

Queries inside another query.

- **Single-Row Subquery** – Returns one row.

  ```
  SELECT Name FROM Employees WHERE Salary = (SELECT MAX(Salary) FROM Employees);
  ```

- **Multi-Row Subquery** – Returns multiple rows.

  ```
  SELECT Name FROM Employees WHERE Salary IN (SELECT Salary FROM Employees WHERE Department = 'IT');
  ```

---

7. Case Studies

Case Study 1: Employee Database

Tables:

- **Employees(ID, Name, Age, Salary, DepartmentID)**
- **Departments(DepartmentID, DepartmentName)**

Query Examples:

- Find employees with a salary above the average.

  ```
  SELECT Name FROM Employees WHERE Salary > (SELECT AVG(Salary) FROM Employees);
  ```

- Count employees in each department.

  ```
  SELECT DepartmentID, COUNT(*) FROM Employees GROUP BY DepartmentID;
  ```

Case Study 2: Sales Database

Tables:

- **Customers(CustomerID, Name)**
- **Orders(OrderID, CustomerID, Amount)**

Query Examples:

- Retrieve customers who placed more than 3 orders.

  ```
  SELECT CustomerID FROM Orders GROUP BY CustomerID HAVING COUNT(OrderID) > 3;
  ```

- Get total sales per customer.

  ```
  SELECT CustomerID, SUM(Amount) FROM Orders GROUP BY CustomerID;
  ```

SQL Case Studies with Real-World Scenarios

Here are some case studies demonstrating practical use of SQL queries in various scenarios:

Case Study 1: Employee Management System

Scenario:
A company maintains employee records, including personal details, salary, and department information.

Tables:

1. **Employees**
   - EmployeeID (Primary Key)
   - Name
   - Age
   - Salary
   - DepartmentID (Foreign Key references Departments table)
   - JoiningDate
2. **Departments**
   - DepartmentID (Primary Key)
   - DepartmentName

Queries:

1. Retrieve Employees Earning More than the Average Salary

```
SELECT Name, Salary FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

2. Find the Number of Employees in Each Department

```
SELECT Departments.DepartmentName, COUNT(Employees.EmployeeID) AS EmployeeCount
```

```
FROM Employees
JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID
GROUP BY Departments.DepartmentName;
```

3. List Employees Who Have Been with the Company for More than 5 Years

```
SELECT Name, JoiningDate FROM Employees
WHERE DATEDIFF(CURDATE(), JoiningDate) > 1825;
```

4. Retrieve the Highest Paid Employee in Each Department

```
SELECT Name, Salary, DepartmentID FROM Employees e
WHERE Salary = (SELECT MAX(Salary) FROM Employees WHERE DepartmentID = e.DepartmentID);
```

Case Study 2: E-Commerce Database

Scenario:
An online store wants to analyze its customers and order data.

Tables:

1. **Customers**
   - CustomerID (Primary Key)
   - Name
   - Email
   - City
2. **Orders**
   - OrderID (Primary Key)
   - CustomerID (Foreign Key references Customers table)
   - OrderDate
   - Amount

Queries:

1. Retrieve Customers Who Have Placed More than 3 Orders

```
SELECT CustomerID, Name, COUNT(OrderID) AS OrderCount
FROM Customers
JOIN Orders ON Customers.CustomerID = Orders.CustomerID
GROUP BY CustomerID, Name
HAVING COUNT(OrderID) > 3;
```

2. Find Total Revenue for Each Customer

```
SELECT Customers.Name, SUM(Orders.Amount) AS TotalSpent
FROM Customers
JOIN Orders ON Customers.CustomerID = Orders.CustomerID
GROUP BY Customers.Name;
```

3. Retrieve the Most Recent Order Placed by Each Customer

```
SELECT Customers.Name, Orders.OrderID, Orders.OrderDate
FROM Orders
JOIN Customers ON Orders.CustomerID = Customers.CustomerID
WHERE OrderDate = (SELECT MAX(OrderDate) FROM Orders WHERE CustomerID =
Customers.CustomerID);
```

4. Find the City with the Highest Number of Customers

```
SELECT City, COUNT(CustomerID) AS CustomerCount
FROM Customers
GROUP BY City
ORDER BY CustomerCount DESC
LIMIT 1;
```

Case Study 3: Library Management System

Scenario:
A library maintains records of books and members who borrow them.

Tables:

1. **Books**
    - BookID (Primary Key)
    - Title
    - Author
    - Genre
    - AvailableCopies
2. **Members**
    - MemberID (Primary Key)
    - Name
    - MembershipDate
3. **BorrowedBooks**
    - BorrowID (Primary Key)
    - BookID (Foreign Key references Books)
    - MemberID (Foreign Key references Members)
    - BorrowDate
    - ReturnDate

Queries:

1. Retrieve the Most Borrowed Book

```
SELECT Books.Title, COUNT(BorrowedBooks.BookID) AS TimesBorrowed
FROM BorrowedBooks
JOIN Books ON BorrowedBooks.BookID = Books.BookID
GROUP BY Books.Title
ORDER BY TimesBorrowed DESC
```

```
LIMIT 1;
```

2. Find Members Who Have Borrowed More Than 5 Books

```
SELECT Members.Name, COUNT(BorrowedBooks.BookID) AS BooksBorrowed
FROM Members
JOIN BorrowedBooks ON Members.MemberID = BorrowedBooks.MemberID
GROUP BY Members.Name
HAVING COUNT(BorrowedBooks.BookID) > 5;
```

3. Retrieve Books That Are Currently Available

```
SELECT Title FROM Books
WHERE AvailableCopies > 0;
```

4. Find Members Who Have Not Returned Books on Time

```
SELECT Members.Name, Books.Title, BorrowedBooks.ReturnDate
FROM BorrowedBooks
JOIN Members ON BorrowedBooks.MemberID = Members.MemberID
JOIN Books ON BorrowedBooks.BookID = Books.BookID
WHERE BorrowedBooks.ReturnDate < CURDATE();
```

Case Study 4: Hospital Management System

Scenario:
A hospital manages patient records, doctors, and appointments.

Tables:

1. **Patients**
   - PatientID (Primary Key)
   - Name
   - Age
   - Gender
   - Contact
2. **Doctors**
   - DoctorID (Primary Key)
   - Name
   - Specialty
3. **Appointments**
   - AppointmentID (Primary Key)
   - PatientID (Foreign Key references Patients)
   - DoctorID (Foreign Key references Doctors)
   - AppointmentDate
   - Status (Completed/Pending)

Queries:

1. Find the Number of Appointments for Each Doctor

```
SELECT Doctors.Name, COUNT(Appointments.AppointmentID) AS TotalAppointments
FROM Doctors
JOIN Appointments ON Doctors.DoctorID = Appointments.DoctorID
GROUP BY Doctors.Name;
```

2. Retrieve Patients Who Have an Appointment Today

```
SELECT Patients.Name, Doctors.Name AS Doctor, Appointments.AppointmentDate
FROM Appointments
JOIN Patients ON Appointments.PatientID = Patients.PatientID
JOIN Doctors ON Appointments.DoctorID = Doctors.DoctorID
WHERE AppointmentDate = CURDATE();
```

3. List Doctors with No Appointments

```
SELECT Name FROM Doctors
WHERE DoctorID NOT IN (SELECT DoctorID FROM Appointments);
```

4. Find Patients Who Have Had More than 3 Appointments

```
SELECT Patients.Name, COUNT(Appointments.AppointmentID) AS TotalAppointments
FROM Patients
JOIN Appointments ON Patients.PatientID = Appointments.PatientID
GROUP BY Patients.Name
HAVING COUNT(Appointments.AppointmentID) > 3;
```

Conclusion

These case studies cover real-world database management problems and show how SQL can be used for data retrieval, analytics, and management. Let me know if you need more complex queries or explanations!

UNIT-IV

Aggregate/Group Functions: AVG, COUNT, MAX, MIN, SUM
Common SQL Clauses for Group Functions: Group By, Having and Order By.
Joins: Inner Join, Outer Join and Self Join

Sub Queries/ Nested Queries: Single-Row Subqueries, Multi-Row Subqueries.

Aggregate/Group Functions in Oracle SQL

An aggregate function is a function that performs a calculation on a set of values, and returns a single value.

Aggregate functions are often used with the GROUP BY clause of the SELECT statement. The GROUP BY clause splits the result-set into groups of values and the aggregate function can be used to return a single value for each group.

SELECT AGGREGATE_FUNCTION(column_name_or_expression)
FROM table_name
[WHERE condition]
[GROUP BY column_name]
[HAVING condition];

Key Terms

- AGGREGATE_FUNCTION: The name of aggregate function for example SUM, COUNT, AVG, MIN, MAX.
- column_name_or_expression: The column or expression the function will be operate on.
- table_name: The name of table from which the data is selected.
- WHERE (optional): It is used to filter the rows before applying aggregation function.
- GROUP BY (optional): It is used to group the rows that have same values in the specified columns.
- HAVING (optional): It is used to filter groups after aggregation is applied.

The most commonly used SQL aggregate functions are:

- MIN() - returns the smallest value within the selected column
- MAX() - returns the largest value within the selected column
- COUNT() - returns the number of rows in a set
- SUM() - returns the total sum of a numerical column
- AVG() - returns the average value of a numerical column
- MEDIAN () – Returns the median (middle value).
- VARIANCE () – Returns the statistical variance of numeric values.
- STDDEV () – Returns the standard deviation.

Aggregate functions ignore null values (except for COUNT ()).

```
CREATE TABLE employees (
   employee_id   NUMBER(10)     PRIMARY KEY,  -- Unique identifier for each
employee
   first_name    VARCHAR2(20),              -- Employee's first name
   last_name     VARCHAR2(20),              -- Employee's last name
   department_id NUMBER(10),                -- Department ID the employee belongs to
   salary        NUMBER(10, 2),             -- Employee's salary with 2 decimal places
   hire_date     DATE                       -- The date when the employee was hired
);

-- Insert the rows into employees table
INSERT INTO employees (employee_id, first_name, last_name, department_id, salary,
hire_date)
VALUES (1, 'John', 'Doe', 10, 50000, TO_DATE('2020-05-10', 'YYYY-MM-DD'));

INSERT INTO employees (employee_id, first_name, last_name, department_id, salary,
hire_date)
VALUES (2, 'Jane', 'Smith', 20, 60000, TO_DATE('2019-03-20', 'YYYY-MM-DD'));

INSERT INTO employees (employee_id, first_name, last_name, department_id, salary,
hire_date)
VALUES (3, 'Alice', 'Johnson', 10, 70000, TO_DATE('2021-07-15', 'YYYY-MM-DD'));

INSERT INTO employees (employee_id, first_name, last_name, department_id, salary,
hire_date)
VALUES (4, 'Bob', 'Brown', 20, 90000, TO_DATE('2022-01-10', 'YYYY-MM-DD'));

INSERT INTO employees (employee_id, first_name, last_name, department_id, salary,
hire_date)
VALUES (5, 'Charlie', 'Davis', 30, 40000, TO_DATE('2020-12-05', 'YYYY-MM-DD'));

INSERT INTO employees (employee_id, first_name, last_name, department_id, salary,
hire_date)
VALUES (6, 'Eve', 'Wilson', 10, 55000, TO_DATE('2018-09-25', 'YYYY-MM-DD'));
```

Output

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	DEPARTMENT_ID	SALARY	HIRE_DATE
1	John	Doe	10	50000	10-MAY-20
2	Jane	Smith	20	60000	20-MAR-19
3	Alice	Johnson	10	70000	15-JUL-21
4	Bob	Brown	20	90000	10-JAN-22

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	DEPARTMENT_ID	SALARY	HIRE_DATE
5	Charlie	Davis	30	40000	05-DEC-20
6	Eve	Wilson	10	55000	25-SEP-18

Example 1: Using SUM and GROUP BY

The **SUM** function is used to calculate the **total salary** for each department. This query shows how much each **department spends** on salaries.

Query:

```
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id;
```

Output

DEPARTMENT_ID	TOTAL_SALARY
10	175000
20	150000
30	40000

Explanation:

- This query calculates the total salary for each department by summing the salary values for all employees within the department.
- The result will be shown with **department_id** and total salary paid to all the employees in that department.

Example 2: Using AVG with GROUP BY

The **AVG** function finds the average salary for each department, which is useful for analyzing salary trends across departments.

Query:

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id;
```

Output

DEPARTMENT_ID	AVG_SALARY
10	58333.33
20	75000
30	40000

Explanation:

- This query calculates the average salary for employees in each department by averaging the salary values for all employees in the department.
- The result will be shown with each **department_id** and average salary for the employees in that department.

Example 3: Using COUNT and MAX with GROUP BY

The **COUNT** function is combined with **MAX** to count the number of employees in each department and identify the **highest-paid** employee in each department.

Query:

```
SELECT department_id, COUNT(employee_id) AS num_employees, MAX(salary) AS highest_salary
FROM employees
GROUP BY department_id;
```

Output

DEPARTMENT_ID	NUM_EMPLOYEES	HIGHEST_SALARY
10	3	70000
20	2	90000
30	1	40000

Explanation:

- In this example, **COUNT** and **MAX** functions are used.
- This query counts number of employees **(COUNT(employee_id))** and it find the highest salary **(MAX(salary))** for each department.
- The result shows each **department_id**, the number of the employees and the highest salary in the each department.

Using HAVING with Aggregate Functions

Query:

```
CREATE TABLE Employee (
  Id INT PRIMARY KEY,
  Name CHAR(1),  -- Adjust data type and length if names can be longer than a single character
  Salary DECIMAL(10,2)  -- Adjust precision and scale if needed for salaries
);

INSERT INTO Employee (Id, Name, Salary)
VALUES (1, 'A', 802),
       (2, 'B', 403),
       (3, 'C', 604),
       (4, 'D', 705),
       (5, 'E', 606),
       (6, 'F', NULL);
```

Output:

Id	Name	Salary
1	A	802
2	B	403
3	C	604
4	D	705
5	E	606
6	F	NULL

The **HAVING clause** is used to filter results after applying aggregate functions. Unlike WHERE, which filters rows before aggregation, HAVING filters groups after aggregation.

Example: Find Employees with Salary Greater Than 600

Query:

```
SELECT Name, SUM(Salary) AS TotalSalary
FROM Employee
GROUP BY Name
HAVING SUM(Salary) > 600;
```

Output:

Name	TotalSalary
A	802
C	604
D	705
E	606

Aggregate functions in SQL perform calculations on a set of values and return a single value. Here are some common aggregate functions using a single table:

Example Table: `Sales`

SaleID	Product	Quantity	Price
1	Laptop	2	800
2	Phone	5	500
3	Tablet	3	300
4	Laptop	1	800
5	Phone	4	500

Aggregate Function Examples

1. **SUM()** – Total revenue from sales

   ```
   SELECT SUM(Quantity * Price) AS TotalRevenue FROM Sales;
   ```

   Output: `TotalRevenue = 8500`

2. **AVG()** – Average price of products

   ```
   SELECT AVG(Price) AS AveragePrice FROM Sales;
   ```

   Output: `AveragePrice = 580`

3. **COUNT()** – Total number of sales transactions

   ```
   SELECT COUNT(*) AS TotalSales FROM Sales;
   ```

   Output: `TotalSales = 5`

4. **MAX()** – Highest price of a product

   ```
   SELECT MAX(Price) AS MaxPrice FROM Sales;
   ```

   Output: `MaxPrice = 800`

5. **MIN()** – Lowest price of a product

   ```
   SELECT MIN(Price) AS MinPrice FROM Sales;
   ```

   Output: `MinPrice = 300`

Using GROUP BY

To calculate aggregates per product:

```
SELECT Product, SUM(Quantity) AS TotalQuantity, AVG(Price) AS AvgPrice
FROM Sales
GROUP BY Product;
```

This returns total quantity sold and average price per product.

---

1. Using `HAVING` with Aggregate Functions

The `HAVING` clause filters grouped data based on aggregate function results.

Example: Filter products with total quantity sold > 3

```
SELECT Product, SUM(Quantity) AS TotalQuantity
FROM Sales
GROUP BY Product
HAVING SUM(Quantity) > 3;
```

Output

Product	TotalQuantity
Phone	9

Explanation:

- `GROUP BY Product` groups sales by product.
- `SUM(Quantity)` calculates total quantity sold per product.
- `HAVING SUM(Quantity) > 3` filters products where the total quantity exceeds 3.

---

2. Using Aggregate Functions with `JOIN`

We often join multiple tables and apply aggregate functions to analyze data.

Example Tables

Orders Table

OrderID	CustomerID	OrderDate
1	101	2024-01-01
2	102	2024-01-05
3	103	2024-02-10

OrderDetails Table

OrderID	Product	Quantity	Price
1	Laptop	2	800
1	Phone	1	500
2	Tablet	3	300
3	Laptop	1	800
3	Phone	4	500

Example: Total Sales Revenue Per Customer

```
SELECT O.CustomerID, SUM(OD.Quantity * OD.Price) AS TotalSpent
FROM Orders O
JOIN OrderDetails OD ON O.OrderID = OD.OrderID
GROUP BY O.CustomerID
HAVING SUM(OD.Quantity * OD.Price) > 1000;
```

Output

CustomerID	TotalSpent
101	2100
103	2800

Explanation:

- `JOIN` connects `Orders` and `OrderDetails` using `OrderID`.
- `SUM(Quantity * Price)` calculates total spending per customer.
- `GROUP BY CustomerID` groups results by customer.
- `HAVING SUM(OD.Quantity * OD.Price) > 1000` filters customers who spent more than 1000.

---

3. Using Multiple Aggregate Functions Together

Example: Get total orders, average order value, and max order value per customer

```
SELECT O.CustomerID,
       COUNT(O.OrderID) AS TotalOrders,
       AVG(OD.Quantity * OD.Price) AS AvgOrderValue,
       MAX(OD.Quantity * OD.Price) AS MaxOrderValue
FROM Orders O
JOIN OrderDetails OD ON O.OrderID = OD.OrderID
GROUP BY O.CustomerID;
```

Output

CustomerID	TotalOrders	AvgOrderValue	MaxOrderValue
101	1	1050	1600
102	1	900	900
103	1	1400	2000

Explanation:

- `COUNT(O.OrderID)`: Total number of orders per customer.
- `AVG(Quantity * Price)`: Average order value per customer.
- `MAX(Quantity * Price)`: Highest single order value per customer.

---

joins(inner join , outer join and self Join) examples

Great! Let's explore **JOINs** in SQL with practical examples. We'll cover:

1. **INNER JOIN**
2. **LEFT (OUTER) JOIN**
3. **RIGHT (OUTER) JOIN**
4. **FULL (OUTER) JOIN**
5. **SELF JOIN**

Sample Tables

1. Customers Table

CustomerID	Name	Country
1	Alice	USA
2	Bob	Canada
3	Charlie	UK

CustomerID	Name	Country
4	David	USA

2. Orders Table

OrderID	CustomerID	Product	Amount
101	1	Laptop	1000
102	2	Phone	500
103	1	Tablet	300
104	3	Laptop	800
105	5	Headphones	200

---

1. INNER JOIN (Matches only common records)

Returns rows where there is a match in both tables.

```
SELECT C.CustomerID, C.Name, O.OrderID, O.Product, O.Amount
FROM Customers C
INNER JOIN Orders O ON C.CustomerID = O.CustomerID;
```

Output

CustomerID	Name	OrderID	Product	Amount
1	Alice	101	Laptop	1000
1	Alice	103	Tablet	300
2	Bob	102	Phone	500
3	Charlie	104	Laptop	800

✅ **Notice:** Customer **ID = 5** is in the `Orders` table but not in `Customers`, so it is excluded.

---

2. LEFT (OUTER) JOIN (All from Left + Matching from Right)

Returns all records from the **left table (`Customers`)** and matches from the **right table (`Orders`)**. If no match, `NULL` is returned.

```
SELECT C.CustomerID, C.Name, O.OrderID, O.Product, O.Amount
FROM Customers C
LEFT JOIN Orders O ON C.CustomerID = O.CustomerID;
```

Output

CustomerID	Name	OrderID	Product	Amount
1	Alice	101	Laptop	1000
1	Alice	103	Tablet	300
2	Bob	102	Phone	500
3	Charlie	104	Laptop	800
4	David	NULL	NULL	NULL

✅ **Notice:**

- `David` (CustomerID = 4) has no order, so the columns from `Orders` are **NULL**.

---

3. RIGHT (OUTER) JOIN (All from Right + Matching from Left)

Returns all records from the **right table (`Orders`)** and matches from the **left table (`Customers`)**.

```
SELECT C.CustomerID, C.Name, O.OrderID, O.Product, O.Amount
FROM Customers C
RIGHT JOIN Orders O ON C.CustomerID = O.CustomerID;
```

Output

CustomerID	Name	OrderID	Product	Amount
1	Alice	101	Laptop	1000
1	Alice	103	Tablet	300
2	Bob	102	Phone	500
3	Charlie	104	Laptop	800
NULL	NULL	105	Headphones	200

✅ **Notice:**

- Order **ID = 105** has a **CustomerID = 5**, but **CustomerID = 5** is missing from the `Customers` table. So, `NULL` appears in the `Customers` columns.

---

4. FULL (OUTER) JOIN (All Records from Both)

Returns **all** records from both tables. If no match, `NULL` is used.

```
SELECT C.CustomerID, C.Name, O.OrderID, O.Product, O.Amount
FROM Customers C
FULL JOIN Orders O ON C.CustomerID = O.CustomerID;
```

Output

CustomerID	Name	OrderID	Product	Amount
1	Alice	101	Laptop	1000
1	Alice	103	Tablet	300
2	Bob	102	Phone	500
3	Charlie	104	Laptop	800
4	David	NULL	NULL	NULL
NULL	NULL	105	Headphones	200

✅ **Notice:**

- `David` (CustomerID = 4) has **no orders** (NULL in `Orders`).
- Order **ID = 105** has **no customer** (NULL in `Customers`).

---

5. SELF JOIN (Joins a Table to Itself)

Used when a table has hierarchical or relational data.

Example: Employee-Manager Relationship

Employees Table

EmpID	Name	ManagerID
1	John	NULL
2	Sarah	1
3	Mike	1
4	Emma	2

Query: Get Employee and Their Manager

```
SELECT E1.Name AS Employee, E2.Name AS Manager
FROM Employees E1
LEFT JOIN Employees E2 ON E1.ManagerID = E2.EmpID;
```

Output

Employee	Manager
John	NULL
Sarah	John

Employee	Manager
Mike	John
Emma	Sarah

✅ **Notice:**

- `John` has `NULL` because he has no manager.
- `Sarah` and `Mike` report to `John`, and `Emma` reports to `Sarah`.

Summary of Joins

Join Type	Description
INNER JOIN	Returns matching records from both tables.
LEFT JOIN	Returns all records from the left table and matching records from the right. If no match, NULL.
RIGHT JOIN	Returns all records from the right table and matching records from the left. If no match, NULL.
FULL JOIN	Returns all records from both tables. If no match, NULL is used.
SELF JOIN	Joins a table with itself (used for hierarchical data).

SQL Joins (Inner, Left, Right and Full Join)

SQL **joins** are the foundation of **database management systems**, enabling the combination of data from multiple tables based on relationships between columns. Joins allow **efficient data retrieval**, which is essential for generating meaningful observations and solving **complex business queries**.
Understanding SQL join types, such as **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, **FULL JOIN**, is critical for working with relational databases.
In this article, we will cover the **different types of SQL joins,** including **INNER JOIN**, **LEFT OUTER JOIN**, **RIGHT JOIN**, **FULL JOIN**. Each join type will be explained with examples, **syntax**, and practical use cases to help us understand when and how to use these joins effectively.

What is SQL Join?
SQL JOIN clause is used to **query** and **access data** from multiple tables by establishing **logical relationships** between them. It can access data from multiple tables simultaneously using common key values shared across different tables. We can use **SQL JOIN** with **multiple tables**. It can also be paired with other clauses, the most popular use will be using JOIN with **WHERE clause** to filter data retrieval.

Example of SQL JOINS

Consider the two tables, **Student** and **StudentCourse**, which share a common column **ROLL_NO**. Using SQL JOINS, we can combine data from these tables based on their **relationship**, allowing us to retrieve meaningful information like student details along with their **enrolled courses**.

Student Table

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

Student Course Table

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

Both these tables are connected by one common key (column) i.e **ROLL_NO**. We can perform a JOIN operation using the given SQL query:

Query:

```
SELECT s.roll_no, s.name, s.address, s.phone, s.age, sc.course_id
FROM Student s
JOIN StudentCourse sc ON s.roll_no = sc.roll_no;
```

Output

ROLL_NO	NAME	ADDRESS	PHONE	AGE	COURSE_ID
1	HARSH	DELHI	XXXXXXXXXX	18	1
2	PRATIK	BIHAR	XXXXXXXXXX	19	2
3	RIYANKA	SILGURI	XXXXXXXXXX	20	2
4	DEEP	RAMNAGAR	XXXXXXXXXX	18	3

ROLL_NO	NAME	ADDRESS	PHONE	AGE	COURSE_ID
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19	1

Types of JOIN in SQL

There are many types of Joins in SQL. Depending on the use case, we can use different type of **SQL JOIN** clause. Below, we explain the most commonly used join types with syntax and examples:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- Natural Join

1. SQL INNER JOIN

The **INNER JOIN** keyword selects all rows from both the tables as long as the condition is satisfied. This keyword will create the **result-set** by combining all rows from both the tables where the **condition satisfies** i.e value of the common field will be the same.

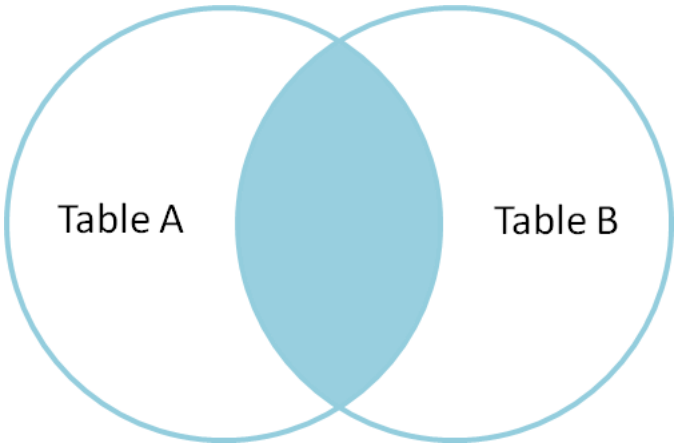
Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
INNER JOIN table2
ON  table1.matching_column = table2.matching_column;
```

Key Terms

- **table1**: First table.
- **table2**: Second table
- **matching_column**: Column common to both the tables.

Note: We can also write JOIN instead of INNER JOIN. JOIN is same as INNER JOIN.



INNER JOIN Example

Let's look at the example of **INNER JOIN** clause, and understand it's working. This query will show the names and age of students enrolled in different courses.

Query:

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE FROM Student
INNER JOIN StudentCourse
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

Output

COURSE_ID	NAME	Age
1	HARSH	18
2	PRATIK	19
2	RIYANKA	20
3	DEEP	18
1	SAPTARHI	19

2. SQL LEFT JOIN

LEFT JOIN returns all the rows of the table on the left side of the join and matches rows for the table on the right side of the join. For the rows for which there is **no matching row** on the right side, the result-set will contain **null**. LEFT JOIN is also known as **LEFT OUTER JOIN**.

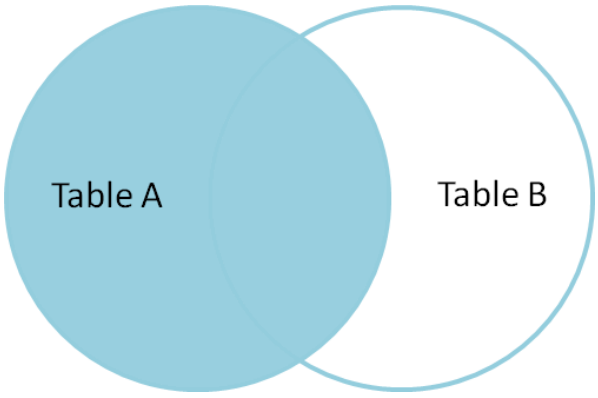
Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
```

Key Terms

- **table1:** First table.
- **table2**: Second table
- **matching_column**: Column common to both the tables.

Note: We can also use LEFT OUTER JOIN instead of LEFT JOIN, both are the same.



LEFT JOIN Example

In this example, the **LEFT JOIN** retrieves all rows from the **Student** table and the matching rows from the **StudentCourse** table based on the **ROLL_NO** column.

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
DHANRAJ	*NULL*
ROHIT	*NULL*
NIRAJ	*NULL*

3. SQL RIGHT JOIN

RIGHT JOIN returns all the rows of the table on the **right side of the join** and matching rows for the table on the left side of the join. It is very similar to **LEFT JOIN f**or the rows for which there is no matching row on the left side, the result-set will contain **null**. **RIGHT JOIN** is also known as **RIGHT OUTER JOIN**.

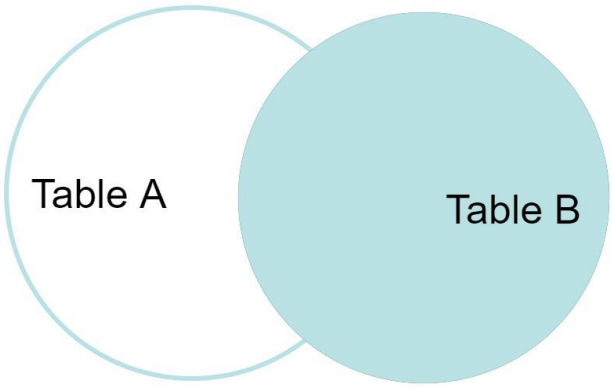
Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```

Key Terms

- **table1**: First table.
- **table2**: Second table
- **matching_column**: Column common to both the tables.

Note: We can also use **RIGHT OUTER JOIN** instead of RIGHT JOIN, both are the same.



RIGHT JOIN Example

In this example, the **RIGHT JOIN** retrieves all rows from the **StudentCourse** table and the matching rows from the **Student** table based on the ROLL_NO column.

Query:

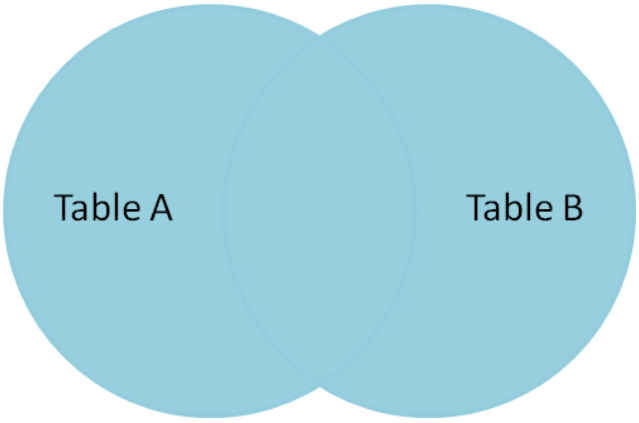
```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
RIGHT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
NULL	4
NULL	5
NULL	4

4. SQL FULL JOIN

FULL JOIN creates the result-set by combining results of both **LEFT JOIN** and **RIGHT JOIN**. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain *NULL* values.



Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```

Key Terms

- **table1**: First table.
- **table2**: Second table
- **matching_column**: Column common to both the tables.

FULL JOIN Example

This example demonstrates the use of a **FULL JOIN**, which combines the results of both **LEFT JOIN** and **RIGHT JOIN**. The query retrieves all rows from the **Student** and **StudentCourse** tables. If a record in one table does not have a matching record in the other table, the result set will include that record with **NULL values** for the missing fields

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
FULL JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1

NAME	COURSE_ID
DHANRAJ	NULL
ROHIT	NULL
NIRAJ	NULL
NULL	4
NULL	5
NULL	4

Subqueries / Nested Queries in Oracle SQL

S**ub queries** are powerful **SQL** features that allow for the nesting of one query inside another for dynamic data retrieval. They have extensive applications when complex problems are to be solved by reducing them into smaller, more **manageable queries**.
The inner query, usually called the **subquery**, returns results that the outer or main query uses for **filtering**, **comparison**, or further processing. **Subqueries** can be applied to every part of the **SQL statement**: **SELECT**, **FROM**, or even inside a **WHERE clause**.

Subqueries

Include one query inside another to make data retrieval more **flexible**. The inner query runs first and provides results that the **outer** query uses for **filtering** or **processing**.
This is helpful when solving **complex** problems because you can break them into smaller, easier steps. Subqueries can be used in different parts of an **SQL** statement, like in the SELECT or WHERE clauses, to help manage and analyze data more effectively.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT column_name FROM another_table WHERE
condition);
```

Key Terms:

- **SELECT column_name**: This part of the query selects data from a column in table_name.
- **FROM table_name:** Specifies the table from which the data will be selected.
- **WHERE column_name =:** This filters the rows in **table_name** based on the condition that follows.

Subqueries

PL/SQL **subqueries** are powerful tools that allow **queries** to be nested inside other queries, making data retrieval more **dynamic** and **flexible**. By breaking down **complex** operations into smaller, **manageable** parts, subqueries enhance **efficiency** and enable more detailed data analysis.

These **subqueries** can be used in various parts of an SQL statement, including the **SELECT**, **FROM**, and **WHERE** clauses, each offering unique **functionalities** to handle complex tasks.

Departments Table

The departments table is created with three columns: **department_id, department_name,** and **location_id.** The department_id is the primary key, and five departments with respective **locations** are inserted into this table.

Query:

```
CREATE TABLE departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50),
    location_id INT
);

INSERT INTO departments (department_id, department_name, location_id)
VALUES
    (1, 'HR', 100),
    (2, 'IT', 200),
    (3, 'Finance', 100),
    (4, 'Marketing', 300),
    (5, 'Operations', 200);
```

Output:

department_id	department_name	location_id
1	HR	100
2	IT	200
3	Finance	100
4	Marketing	300
5	Operations	200

Explanation:
The table contains information about five departments: HR, IT, Finance, Marketing, and Operations, each with a unique department_id and location_id.

Employees Table

The employees table is created with columns like employee_id, employee_name, department_id, and salary. The department_id is a foreign key that references the departments table. Nine employees are inserted into this table, each belonging to one of the five departments.Query:

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(50),
    department_id INT,
    salary DECIMAL(10, 2),
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

INSERT INTO employees (employee_id, employee_name, department_id, salary)
VALUES
    (1, 'John Doe', 1, 5000),
    (2, 'Jane Smith', 2, 7000),
    (3, 'Sam Brown', 3, 6000),
    (4, 'Lisa White', 1, 5500),
    (5, 'Tom Johnson', 4, 7500),
    (6, 'Nancy Allen', 2, 7200),
    (7, 'Steve Adams', 5, 6300),
    (8, 'Mike Taylor', 3, 5800),
    (9, 'Sophie King', 1, 5200);
```

Output:

employee_id	employee_name	department_id	salary
1	John Doe	1	5000
2	Jane Smith	2	7000
3	Sam Brown	3	6000
4	Lisa White	1	5500
5	Tom Johnson	4	7500
6	Nancy Allen	2	7200
7	Steve Adams	5	6300
8	Mike Taylor	3	5800
9	Sophie King	1	5200

Explanation:

The table now holds data for nine employees, including their employee_id, employee_name, associated department_id, and their salary. Each employee belongs to a department, and their salary varies based on their role.

Types of Subqueries

SQL Subqueries-One of the main types of subqueries, depending on what for and how they connect with the principal query, is shown below. The main types of subqueries are as follows:

1. Single-Row Subquery

The query retrieves the **employee_name** from the **employees** table where the department_id matches any of the department_id values from the subquery. The **subquery** finds all departments located at **location_id** = 100 from the departments table.

Query:

```
SELECT employee_name
FROM employees
WHERE department_id IN (SELECT department_id FROM departments WHERE
location_id = 100);
```

Output:

employee_name
Alice

Explanation:

The result displays "**Alice**" as the employee working in a **department** located at location_id = 100. The **subquery** first identifies the relevant department_id, and the outer query **filters** employees working in those **departments**.

2. Multiple-Row Subquery

- The query retrieves the **employee_id**, **employee_name**, and **salary** of employees who work in **departments** located at location_id = 200.
- The subquery first fetches the **department_id** values from the departments table for those located at **location_id = 200** (IT and Operations).
- The **outer** query then filters the employees table for employees working in these departments.

Query:

```
SELECT employee_id, employee_name, salary
FROM employees
WHERE department_id IN (
  SELECT department_id
  FROM departments
  WHERE location_id = 200
);
```

Output:

employee_id	employee_name	salary
2	Jane Smith	7000
6	Nancy Allen	7200
7	Steve Adams	6300

Explanation:

- This question lists **employees** who work in **departments** located at **location_id** = **200**. This query uses a subquery to identify first the **department_id** values of departments in that location by searching the **departments** table for **location_id** = **200**.
- This time the subquery identifies that the **IT** and **Operations departments** are in this location. The main query then uses this result to filter the employees table, returning the **employee_id**, **employee_name**, and salary for all employees who belong to those departments.
- As a result, the **employees** Jane Smith, **Nancy Allen**, and **Steve Adams** would appear in the output. This helps decompose complex conditions into smaller queries.

3. Correlated Subquery

In that case, the **subquery** will compute the average salary for each department coming from the outer query. It constitutes a correlated subquery because it draws its input from the department_id in the outer query.

Such **subqueries** are **dependent** upon values from the **main** query for their operation.

Query:

```
SELECT employee_name
FROM employees emp
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id =
emp.department_id);
```

Output:

employee_name
Bob
David

Explanation:

The employees "**Bob**" and "**David**" are displayed because their salaries are higher than the **average** salary of their respective departments. Each department's average salary is calculated dynamically based on the **department_id** from the outer query.

4. Nested Subquery

A **nested** subquery is a **subquery** included within another subquery. In other words, actually doing several layers of **querying**. In other words, it is a query inside a query **inside** another query.

Query:

```
SELECT employee_name
FROM employees
WHERE department_id = (SELECT department_id FROM departments WHERE location_id
= (SELECT location_id FROM locations WHERE city = 'New York');
```

Output:

employee_name
Alice

Explanation:

The employee "**Alice**" is displayed because she works in a department located in **New York**. The query runs through **multiple** nested layers to extract this information.

Scalar Subquery

A scalar subquery returns precisely one value-one row, and one column. It can be used in any part of a query where a single value is expected. This **subquery** retrieves the name of the department that each employee belongs to.

Query:

```
SELECT employee_name,
(SELECT department_name FROM departments WHERE department_id =
employees.department_id) AS department_name
FROM employees;
```

Output:

employee_name	department_name
Alice	HR
Bob	Finance

Explanation:

The result shows each employee's name along with their department. For example, "**Alice**" belongs to the HR department, and "**Bob**" works in the Finance department. Each employee has one corresponding department name returned by the **scalar subquery**.

Subqueries with SELECT, FROM, and WHERE Clauses

Subqueries can be used in various parts of an SQL statement: the **SELECT**, **FROM**, or **WHERE** clauses, each serving a unique purpose. When used in the **SELECT** clause, subqueries return computed or derived values to enhance the result

set. **Subqueries** in the **FROM** clause act as inline views, creating **temporary** tables for further operations.
In the **WHERE** clause, subqueries help filter the results based on conditions from another table or query. Each type of **subquery** allows for complex data retrieval and **manipulation**, enabling more dynamic and detailed analysis of data.

Subqueries in the SELECT Clause

The subqueries in the SELECT clause return some derived or calculated value that appears in the results of the main query. It will be generally used when you need to get some extra data requiring **computation** or a **lookup**.

Query:

```
SELECT employee_name,
      (SELECT department_name
       FROM departments
       WHERE department_id = employees.department_id) AS department_name
FROM employees;
```

Output:

employee_name	department_name
John	Sales
Jane	IT
Alice	Sales

Explanation:

The result displays each employee's name along with their department. For example, "**John**" and "**Alice**" are shown as being part of the "Sales" department, while "**Jane**" is in the "**IT**" department. The subquery retrieves the department name **dynamically** for each employee.

Subqueries in the FROM Clause

Subqueries that occur in the FROM clause are also called inline views. An inline view is a **temporary** result table produced by an **inner query** that is nested inside an outer query.

Lets use the **employees table** again. Now, we want to calculate the average salary per **department** and join back to the original table in order to get employees' names and the average salary that their corresponding department earns.

Query:

```
SELECT e.employee_name, avg_salary
FROM employees e,
    (SELECT department_id, AVG(salary) AS avg_salary
     FROM employees
     GROUP BY department_id) avg_dept
WHERE e.department_id = avg_dept.department_id;
```

Output:

employee_name	avg_salary
John	6000
Jane	6000
Alice	6000

Explanation:

The result shows the names of employees, such as "**John**," "**Jane**," and "**Alice**," along with the average salary for their department, which is 6000 in this case. The **subquery** groups salaries by **department_id** and calculates the **average** salary, joining it with employee data in the main query.

Subqueries with WHERE Clause

This query retrieves the **employee_name** from the employees table where the **department_id** matches the department_id of the "Sales" department, found using a **subquery** in the **WHERE** clause.

Query:

```
SELECT employee_name
FROM employees
WHERE department_id = (SELECT department_id
                       FROM departments
                       WHERE department_name = 'Sales');
```

Output:

employee_name
John
Alice

Explanation:

The result lists "**John**" and "**Alice**" as employees in the "**Sales**" department. The subquery finds the **department_id** for "Sales" in the departments table, and the main query uses this to filter the **employees** working in that department.

Single-Row Subqueries

Returns a single value.

```
SELECT employee_id, name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Multi-Row Subqueries

Returns multiple values using IN, ANY, or ALL.

- **Using IN**

  ```
  SELECT employee_id, name
  FROM employees
  WHERE department_id IN (SELECT department_id FROM departments WHERE location_id =
  1700);
  ```

- **Using ANY**

  ```
  SELECT employee_id, name, salary
  FROM employees
  WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 10);
  ```

- **Using ALL**

```
SELECT employee_id, name, salary
FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 10);
```

Single-Row Subqueries in SQL

A **single-row subquery** is a subquery that returns only **one row and one value**. It is used in `WHERE`, `HAVING`, and `SELECT` clauses and works with comparison operators like =, <, >, <=, >=, and <>.

Example Tables

1. Employees Table

EmpID	Name	Department	Salary
1	Alice	IT	70000
2	Bob	HR	50000
3	Charlie	IT	60000
4	David	Finance	55000

2. Departments Table

DeptID	DeptName	Location
1	IT	New York
2	HR	London
3	Finance	Sydney

---

1. Using Single-Row Subquery in `WHERE` Clause

Find the employees who earn the **highest salary**.

```
SELECT Name, Salary
FROM Employees
WHERE Salary = (SELECT MAX(Salary) FROM Employees);
```

Output

Name	Salary
Alice	70000

✅ **Explanation**

- The subquery `(SELECT MAX(Salary) FROM Employees)` returns `70000`.
- The main query filters employees where `Salary = 70000`.

---

2. Using Single-Row Subquery in `HAVING` Clause

Find the department with the **lowest-paid employee**.

```
SELECT Department, MIN(Salary) AS MinSalary
FROM Employees
GROUP BY Department
HAVING MIN(Salary) = (SELECT MIN(Salary) FROM Employees);
```

Output

Department	MinSalary
HR	50000

✅ Explanation

- The subquery `(SELECT MIN(Salary) FROM Employees)` returns `50000`.
- The main query filters departments where the minimum salary is `50000`.

---

3. Using Single-Row Subquery in `SELECT` Clause

Retrieve all employees along with the **average salary of the company**.

```
SELECT Name, Salary, (SELECT AVG(Salary) FROM Employees) AS
CompanyAvgSalary
FROM Employees;
```

Output

Name	Salary	CompanyAvgSalary
Alice	70000	58750
Bob	50000	58750
Charlie	60000	58750
David	55000	58750

✅ Explanation

- The subquery `(SELECT AVG(Salary) FROM Employees)` returns `58750`.
- It is added as an additional column in the result.

---

4. Using Single-Row Subquery with Comparison Operators

Find employees who earn **more than the average salary**.

```
SELECT Name, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

Output

Name	Salary
Alice	70000
Charlie	60000

✅ Explanation

- The subquery `(SELECT AVG(Salary) FROM Employees)` returns `58750`.
- The main query filters employees where `Salary > 58750`.

Common Errors with Single-Row Subqueries

💡 **Error:** If a subquery returns **multiple rows**, an error occurs:

```
SELECT Name FROM Employees
WHERE Salary = (SELECT Salary FROM Employees WHERE Department = 'IT');
```

🔴 **Error:** `Subquery returns more than 1 row`

✅ **Solution:** Use **multi-row operators** (`IN`, `ANY`, `ALL`), or adjust the subquery to return a single value.

Multi-Row Subqueries in SQL

A **multi-row subquery** returns **multiple values (rows)** and is used with **operators like `IN`, `ANY`, `ALL`, and `EXISTS`** in the main query.

Example Tables

1. Employees Table

EmpID	Name	Department	Salary
1	Alice	IT	70000
2	Bob	HR	50000
3	Charlie	IT	60000
4	David	Finance	55000
5	Emma	IT	65000

2. Departments Table

DeptID	DeptName	Location
1	IT	New York
2	HR	London
3	Finance	Sydney

---

1. Using `IN` with Multi-Row Subquery

Find employees working in departments **with IT or Finance**.

```
SELECT Name, Department
FROM Employees
WHERE Department IN (SELECT DeptName FROM Departments WHERE DeptName IN
('IT', 'Finance'));
```

Output

Name	Department
Alice	IT
Charlie	IT
David	Finance
Emma	IT

✅ **Explanation**

- The subquery returns multiple departments (`IT`, `Finance`).
- The main query selects employees whose department is in that list.

---

2. Using `ANY` with Multi-Row Subquery

Find employees **who earn more than the lowest-paid IT employee**.

```
SELECT Name, Salary
FROM Employees
WHERE Salary > ANY (SELECT Salary FROM Employees WHERE Department = 'IT');
```

Output

Name	Salary
Alice	70000
Emma	65000

✅ **Explanation**

- The subquery `(SELECT Salary FROM Employees WHERE Department = 'IT')` returns multiple values: `{70000, 60000, 65000}`.
- `ANY` checks if the **employee's salary is greater than at least one of those values**.
- The lowest IT salary is `60000`, so salaries above `60000` are selected.

---

3. Using `ALL` with Multi-Row Subquery

Find employees **who earn more than all IT employees**.

```
SELECT Name, Salary
FROM Employees
WHERE Salary > ALL (SELECT Salary FROM Employees WHERE Department = 'IT');
```

Output

Name	Salary
Alice	70000

✅ **Explanation**

- The subquery returns `{70000, 60000, 65000}` (salaries of IT employees).
- `ALL` checks if **the employee's salary is greater than all** values in the list.
- The highest IT salary is `70000`, so only Alice qualifies.

---

4. Using `EXISTS` with Multi-Row Subquery

Find employees **who belong to a department listed in the Departments table**.

```
SELECT Name, Department
FROM Employees E
WHERE EXISTS (SELECT 1 FROM Departments D WHERE E.Department = D.DeptName);
```

Output

Name	Department
Alice	IT
Bob	HR
Charlie	IT
David	Finance
Emma	IT

✅ Explanation

- The subquery checks if the employee's department exists in the `Departments` table.
- `EXISTS` returns `TRUE` for matching rows.

UNIT-V: Distributed object Database Management Systems: Fundamental object concepts and models, object distributed design, architectural issues, object management, distributed object storage.

Fundamental object concepts and models

Need of Object Oriented Data Model :

To represent the complex real world problems there was a need for a data model that is closely related to real world. Object Oriented Data Model represents the real world problems easily.

Object Oriented Data Model :

In Object Oriented Data Model, data and their relationships are contained in a single structure which is referred as object in this data model.

In this, real world problems are represented as objects with different attributes. All objects have multiple relationships between them.

Object Oriented Data Model

= Combination of Object Oriented Programming +

Relational database model

Distributed Object Systems

Distributed Object Database Management Systems (DO-DBMS) manages databases where **data is stored across multiple locations and accessed by various clients, utilizing object-oriented principles for data modeling and manipulation**

What are Distributed Object Databases?

Distributed:

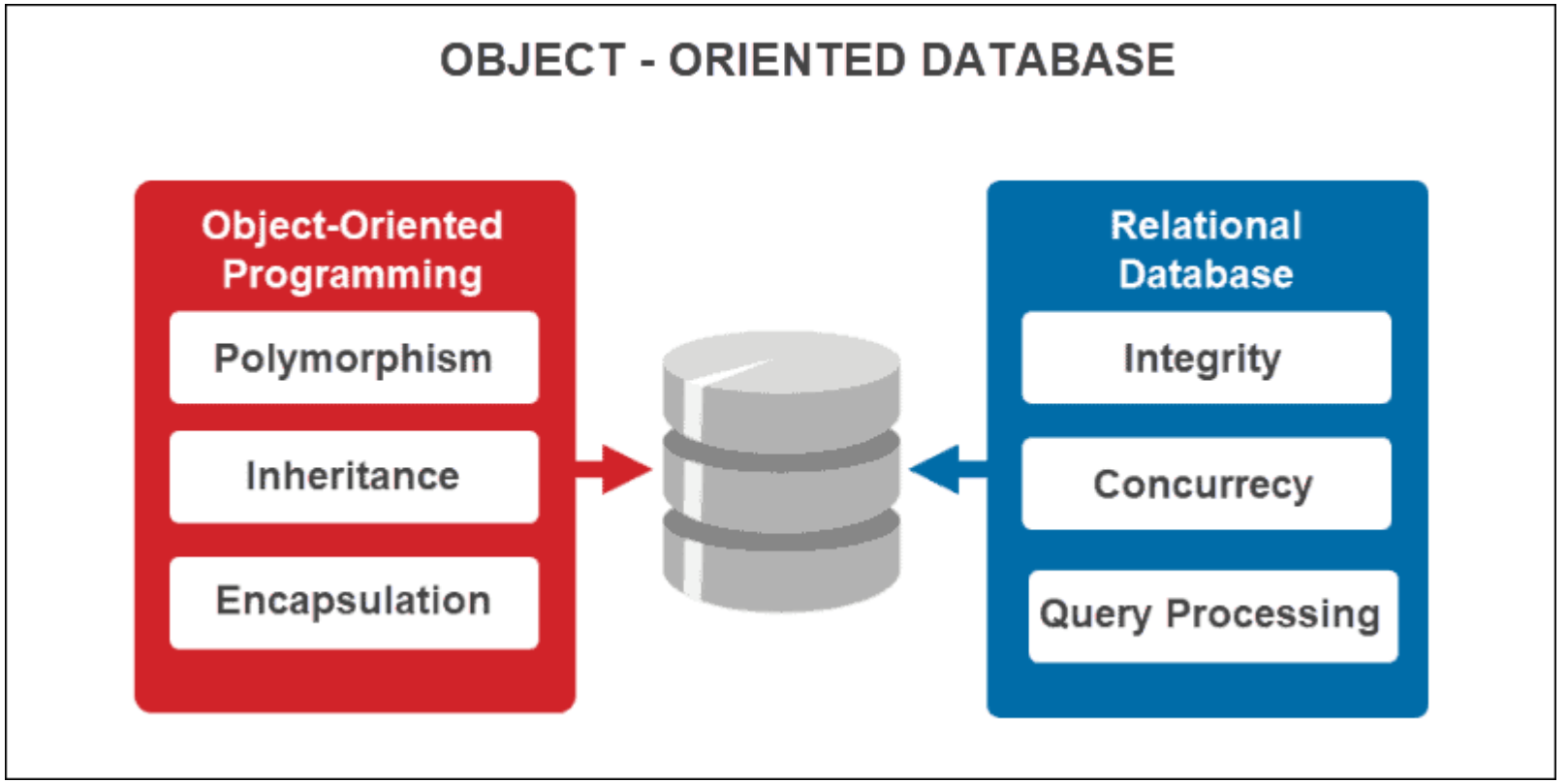
Data is physically stored and managed across multiple computers or sites connected by a network.

Object-oriented:

Data is modeled and manipulated using objects, encapsulating data and behavior (methods).

D-DBMS:

The software system responsible for managing the distributed database, ensuring data consistency, and enabling access from different locations.


OBJECT - ORIENTED DATABASE
Object-Oriented Programming
Polymorphism
Inheritance
Encapsulation
Relational Database
Integrity
Concurrecy
Query Processing

Key Concepts

- **Data Fragmentation:** Dividing the data into smaller units for storage and management.
- **Data Replication:** Maintaining multiple copies of data to improve availability and performance.
- **Query Processing:** Efficiently processing queries that span multiple locations.
- **Transactions:** Ensuring that operations are performed atomically and consistently.

Types of Distributed Databases

Homogeneous: Databases use the same underlying DBMS and data model.

Heterogeneous: Databases may use different DBMS software and data models.

Object-Oriented Databases (OODBMS): Store data as objects, offering flexibility in modeling complex data structures.

NoSQL: Non-relational databases are often designed for distributed environments and large data volumes.

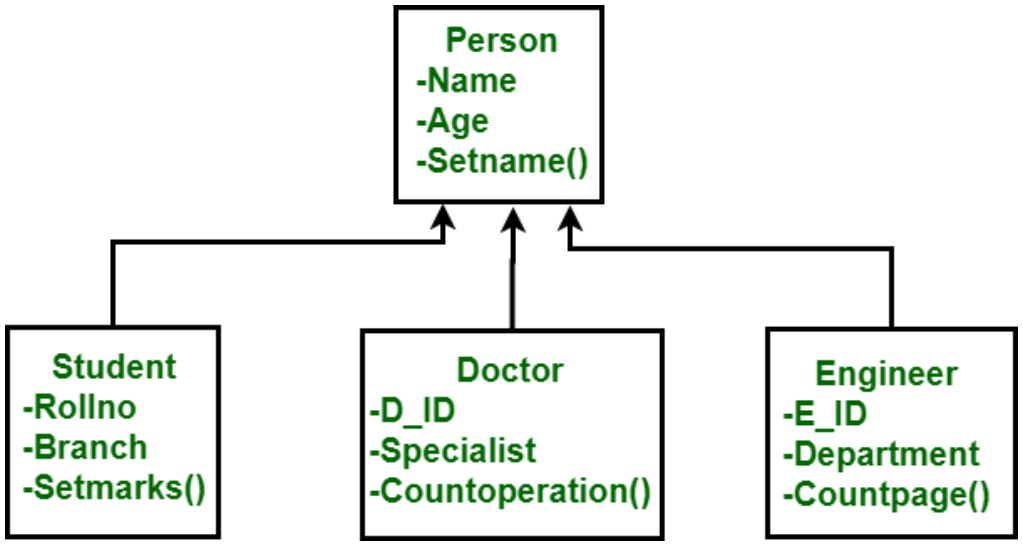
Examples include document databases, key-value stores, and graph databases.

Examples

Azure Cosmos DB: (Microsoft's globally distributed, multi-model database).

MongoDB: (a popular NoSQL database platform that supports distributed deployments).

Components of Object Oriented Data Model



Objects –
An object is an abstraction of a real world entity or we can say it is an instance of class. Objects encapsulates data and code into a single unit which provide data abstraction by hiding the implementation details from the user.
For example: Instances of student, doctor, engineer.

Attribute –
An attribute describes the properties of object. **For example: Object is STUDENT and its attribute are Roll no, Branch, Setmarks() in the Student class.**

Methods –
Method represents the behavior of an object. Basically, it represents the real-world action. **For example: Finding a STUDENT marks in above figure as Setmarks().**

Class –
A class is a collection of similar objects with shared structure i.e. attributes and behavior i.e. methods. An object is an instance of class. **For example: Person, Student, Doctor, Engineer in above figure.**

Object distributed design

- Distributed Object Systems are frameworks that allow objects, which are instances of classes in programming**, to interact with one another over a network**.
- These systems are designed to function in a distributed environment, where components can be spread across **different servers or geographical locations**.
- **The primary goal is to create a unified interface for interacting with objects, regardless of their physical location.**

Key Characteristics

- **Location Transparency:** Users can access objects without knowing their physical location.
- **Interoperability:** Different systems can communicate and work together effectively.
- **Scalability:** Systems can easily expand to accommodate more objects or users.

Object distributed design

Architecture of Distributed Object Systems

The architecture of Distributed Object Systems typically involves several key layers:

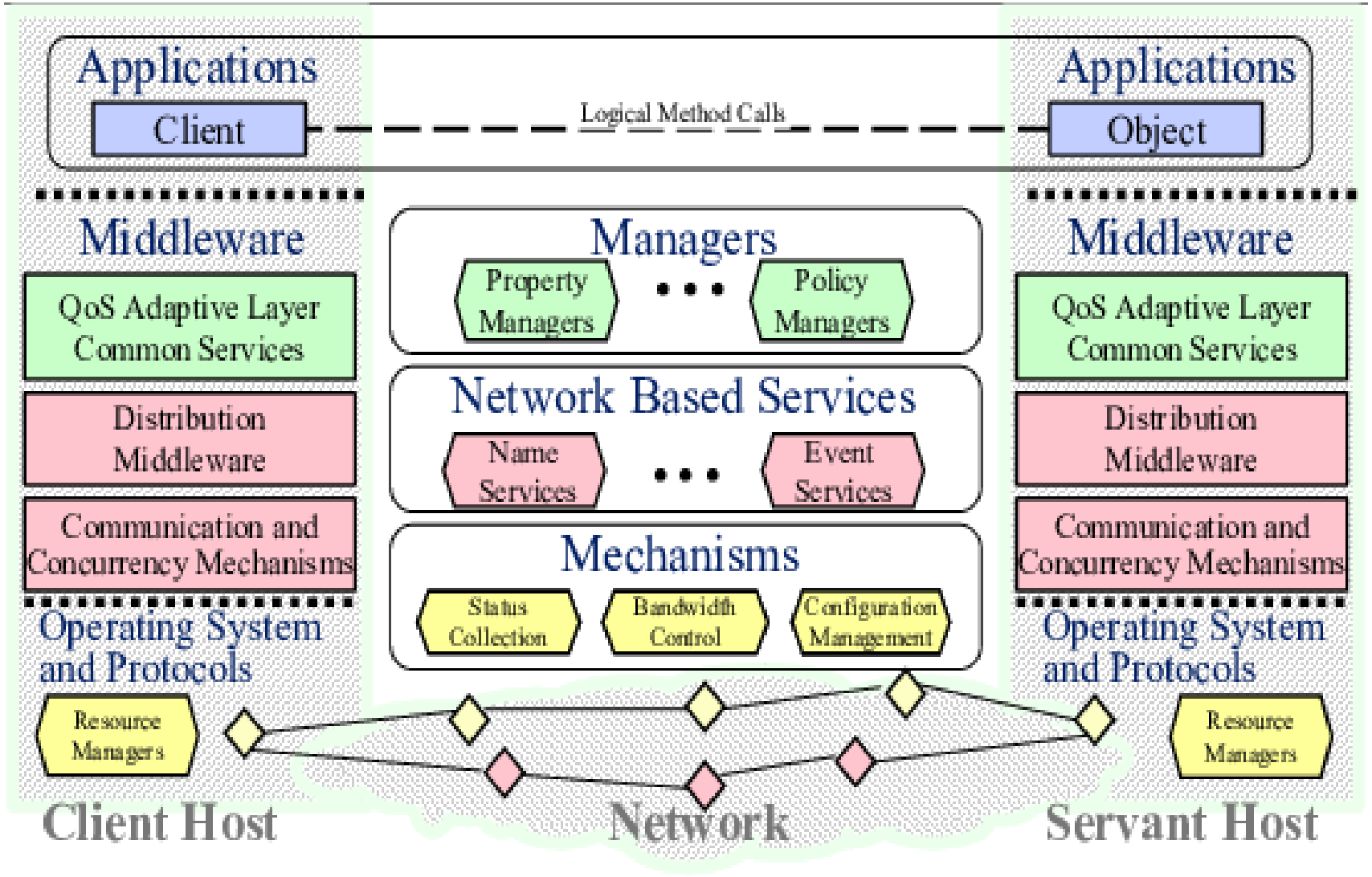
Client Layer: This is where the user interacts with the system. Clients can be anything from web browsers to desktop applications that initiate requests to access distributed objects.

Middleware Layer: Middleware serves as an intermediary that facilitates communication between clients and distributed objects. It handles object location, message routing, and data serialization.

Object Layer: This layer consists of the actual distributed objects that contain the business logic. These objects are often implemented as remote objects, which are instantiated on different servers.

Database Layer: Many distributed object systems rely on a back-end database to store persistent data. This layer manages data integrity and transactions across distributed nodes.

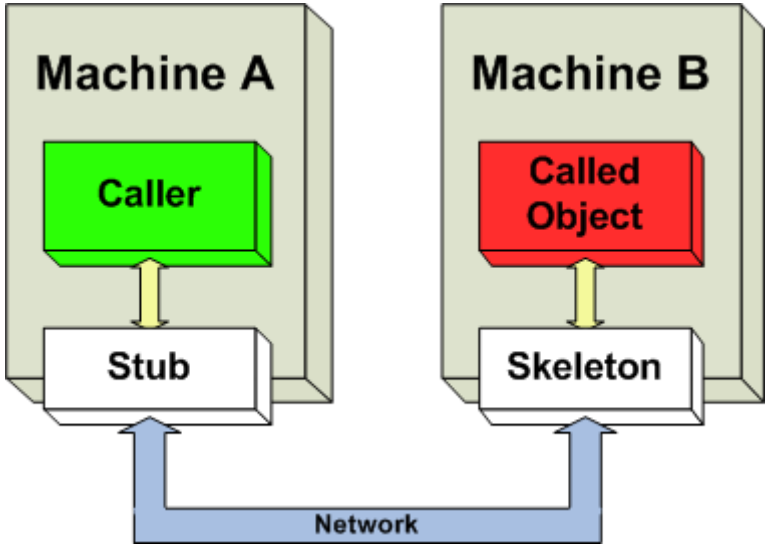
Network Layer: The network layer provides the necessary communication infrastructure, including protocols and services for reliable data transmission.

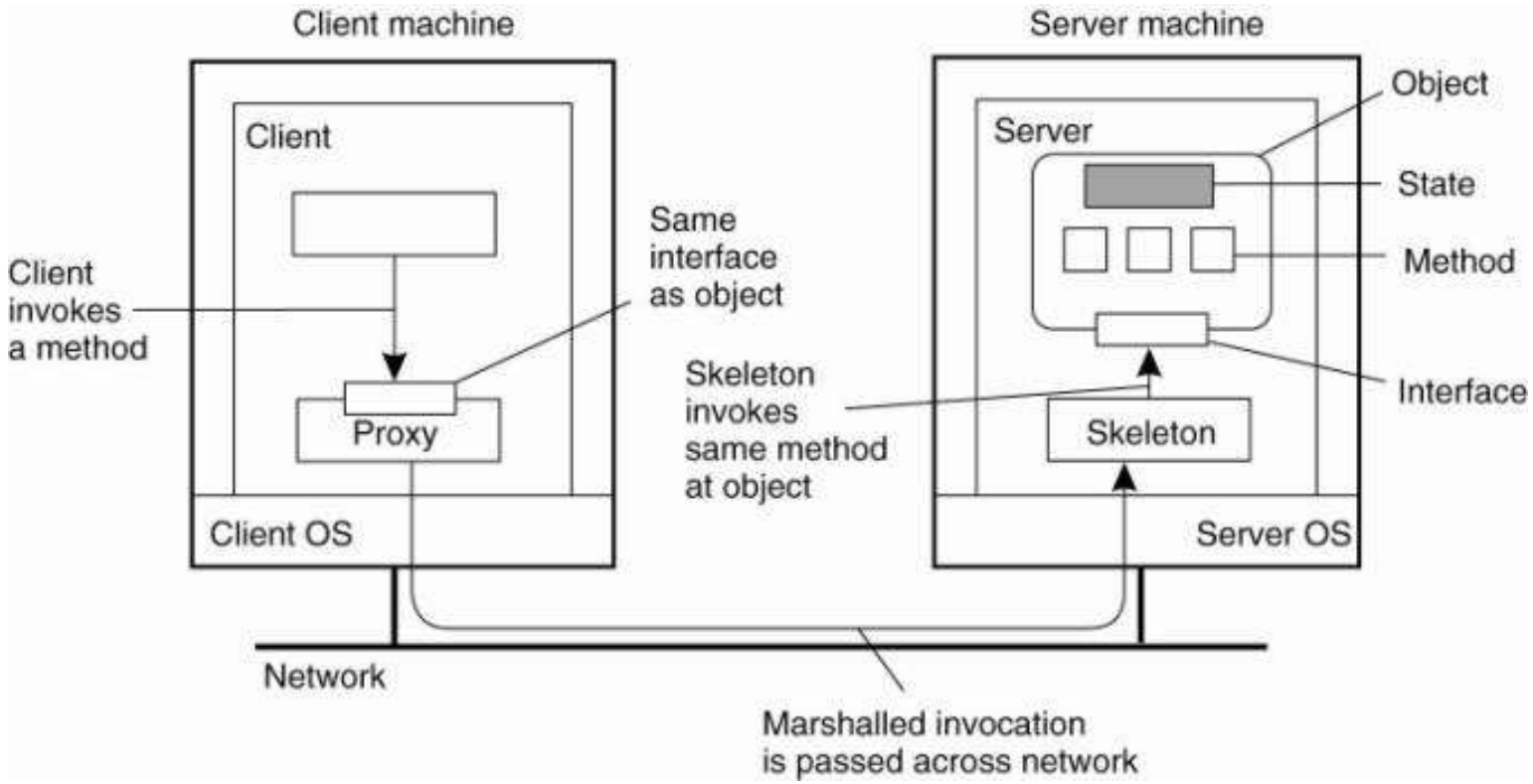

Applications
Client
Logical Method Calls
Applications
Object
Middleware
QoS Adaptive Layer
Common Services
Distribution
Middleware
Communication and
Concurrency Mechanisms
Operating System
and Protocols
Resource
Managers
Managers
Property
Managers
Policy
Managers
Network Based Services
Name
Services
Event
Services
Mechanisms
Status
Collection
Bandwidth
Control
Configuration
Management
Middleware
QoS Adaptive Layer
Common Services
Distribution
Middleware
Communication and
Concurrency Mechanisms
Operating System
and Protocols
Resource
Managers
Client Host
Network
Servant Host

Object distributed design

A “stub’ is a piece of code, acting as a client-side proxy or a gateway for accessing a remote service or object, simplifying the development by handling network communication and marshalling/unmarshalling of data.

The "skeleton" is a server-side object that acts as a gateway for client requests, enabling remote method invocation (RMI) by translating client requests to calls on the actual server-side object and handling communication nuances.




Client machine
Server machine
Client
Server
Object
State
Method
Interface
Client invokes a method
Same interface as object
Skeleton invokes same method at object
Proxy
Skeleton
Client OS
Server OS
Network
Marshalled invocation is passed across network

Object distributed design

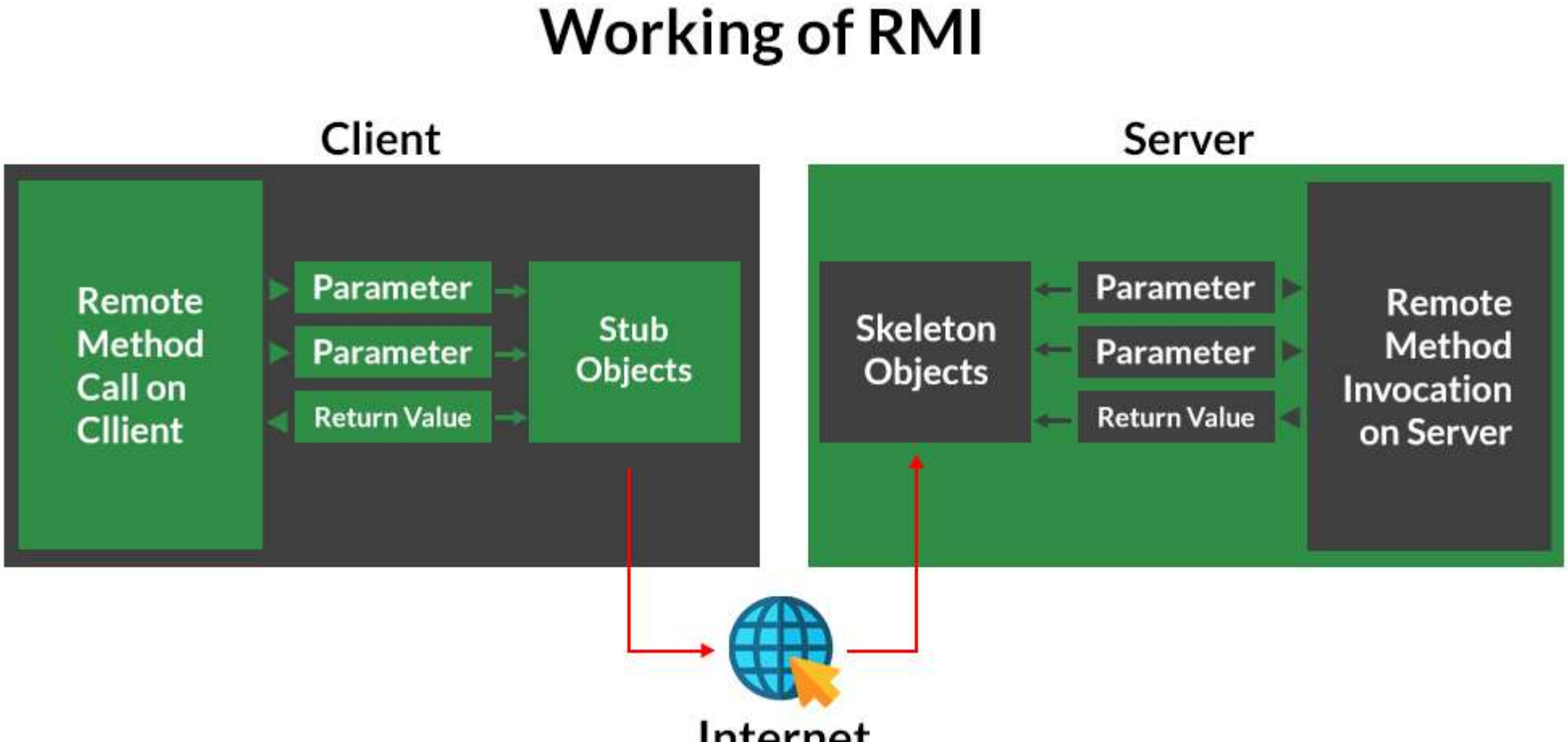
Communication Protocols in Distributed Object Systems

Effective communication is essential for the functionality of Distributed Object Systems. Various protocols are utilized to facilitate object communication:

- **Remote Method Invocation (RMI):** RMI allows a program to invoke methods on an object located in another JVM (Java Virtual Machine). It abstracts the complexity of network communication, making remote calls appear local.
- **Common Object Request Broker Architecture (CORBA):** CORBA is a standard defined by the Object Management Group (OMG) that enables communication between objects in different programming languages. It uses Interface Definition Language (IDL) to define object interfaces.
- **Web Services:** Web services utilize standard protocols like HTTP and XML (SOAP and REST) to enable communication between distributed objects over the web. They are widely used for interoperability between different systems.
- **Message-Oriented Middleware (MOM):** MOM facilitates asynchronous communication between distributed objects. It uses message queues to send and receive messages, decoupling the sender and receiver.
- **gRPC:** gRPC is an open-source RPC framework that uses HTTP/2 for transport and Protocol Buffers for serialization, allowing for efficient communication between distributed components.

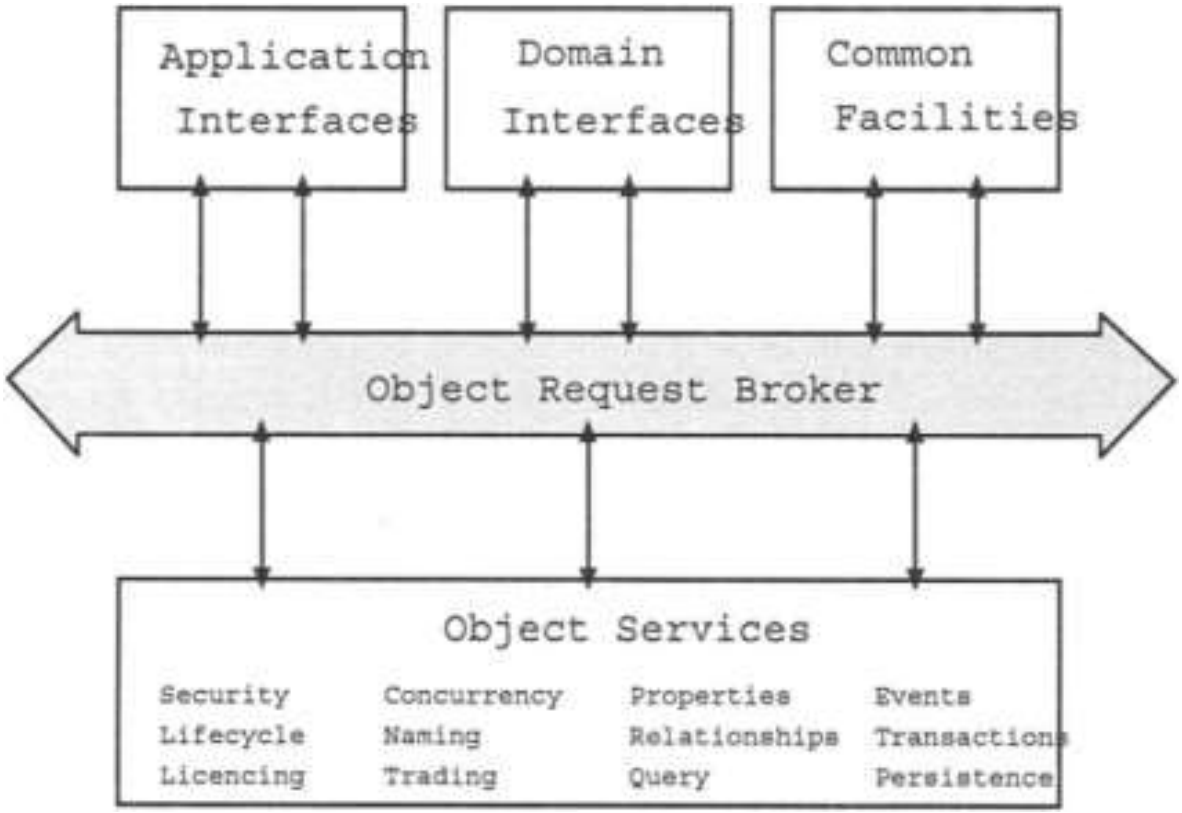
Remote Method Invocation

RMI (Remote Method Invocation) is a Java API that allows objects to communicate with each other across different JVMs (Java Virtual Machines). It enables distributed computing by allowing methods to be invoked on remote objects



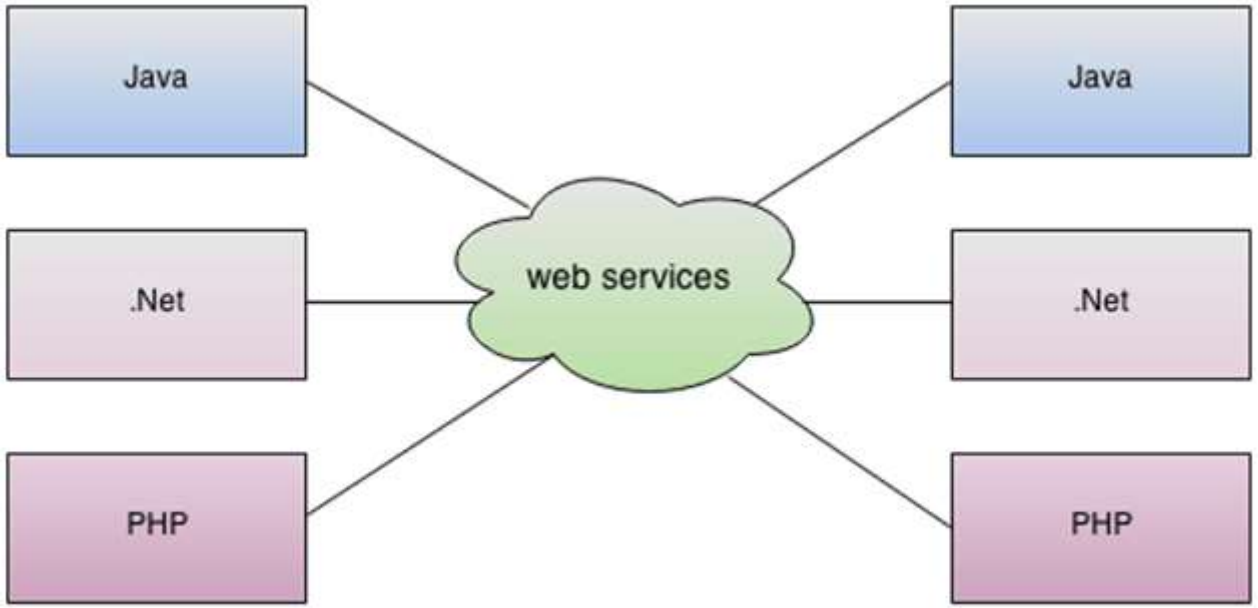
Common Object Request Broker Architecture(CORBA)

It is a middleware standard developed by the **Object Management Group (OMG)** that enables interoperability between distributed applications written in different programming languages and running on different platforms.



Web services

Web services play a crucial role in **distributed databases**, enabling communication between different database systems spread across multiple locations. They provide **platform-independent** and **language-neutral** interfaces for accessing and managing distributed data.



Web services

Types of Web Services in Distributed Databases

SOAP (Simple Object Access Protocol)

Uses **XML** for structured data exchange.
Ensures **high security and reliability**.
Suitable for **enterprise-level** distributed databases.
Example: Banking systems, healthcare databases.

RESTful Web Services (Representational State Transfer)

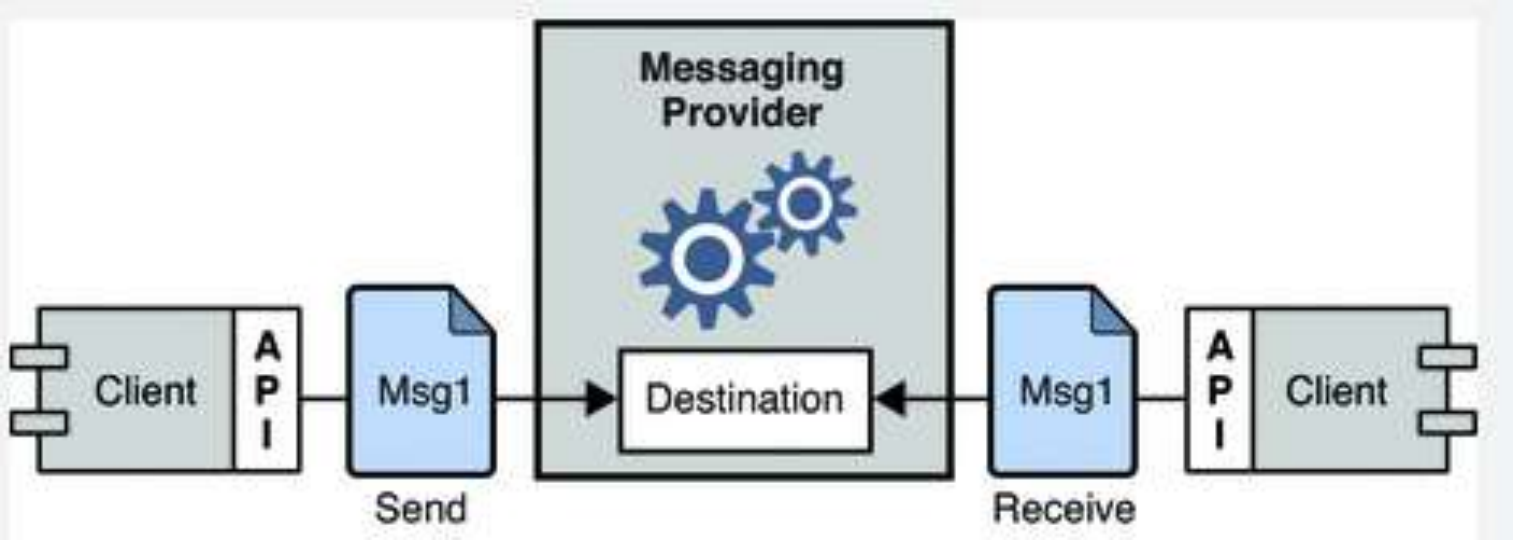
Uses **HTTP methods (GET, POST, PUT, DELETE)**.
Data format: **JSON or XML**.
Lightweight and faster than SOAP.
Example: Cloud-based databases, e-commerce systems.

gRPC (Google Remote Procedure Call)

Uses **Protocol Buffers (protobuf)** instead of JSON/XML.
More efficient than REST and SOAP.
Supports **real-time** database interactions.
Example: Microservices architectures, big data applications.

Message-Oriented Middleware

Message-Oriented Middleware (MOM) is a type of middleware that enables communication between distributed systems using **messages**. It facilitates **asynchronous, loosely coupled** communication, making it ideal for **scalable and resilient** distributed applications.



Architectural Issues

Distributed database systems, while offering advantages like scalability and resilience, present unique architectural challenges, particularly concerning data consistency, concurrency, and fragmentation management.

key architectural issues:

1. Data Consistency & Replication:

Maintaining data integrity is crucial

when data is replicated across multiple nodes or sites. Ensuring that all replicas are synchronized and consistent can be complex, especially with network delays or failures.

Various consistency models exist

(e.g., strong, eventual) each with its trade-offs regarding performance and data accuracy.

Choosing the right consistency level

depends on the specific application requirements and tolerance for data staleness.

Replication strategies

like primary-replica or multi-master can impact consistency and introduce challenges in conflict resolution.

Architectural Issues

2. Concurrency Control:

Distributed transactions across multiple nodes

introduce complex concurrency control challenges.

Managing concurrent access and modifications

to shared data requires efficient locking and transaction management protocols.

Distributed consensus protocols

Used for transaction coordination and maintaining consistency.

Performance bottlenecks and deadlocks

can occur if concurrency control mechanisms are not designed and implemented carefully.

Architectural Issues

3. Data Fragmentation & Allocation:

Dividing data into fragments and allocating them to different sites: can improve performance and reduce data access latency.

The choice of fragmentation strategy: (horizontal, vertical, or mixed) depends on the data access patterns and storage constraints.

Query optimization and data access routing: become more complex in a fragmented environment.

Allocating fragments to sites: requires considering network bandwidth, storage capacity, and site availability.

Architectural Issues

4. Scalability & Performance:

Distributed databases are designed for scalability: but can introduce performance challenges if not designed correctly.

Load balancing and distributed query processing: are crucial for handling large workloads and maintaining performance.

Network latency and bandwidth limitations: can affect performance, especially with complex transactions.

Choosing the appropriate database architecture: (e.g., shared-nothing, shared-disk) is important for scalability and performance.

Architectural Issues

5. Security:

Distributed databases introduce new security challenges: due to the distributed nature of data and communication.

Secure data access control and authentication: are essential to prevent unauthorized access.

Protecting data during transmission and storage: requires robust security protocols and encryption.

Ensuring data confidentiality, integrity, and availability: is paramount in a distributed environment.

Architectural Issues

6. Disaster Recovery & Backup:

Data loss or system failures: in a distributed environment can have significant consequences.

Disaster recovery plans: need to consider the distributed nature of data and the potential for site failures.

Backups and data replication: are crucial for ensuring data availability and recoverability.

Implementing robust backup and recovery procedures: is essential for maintaining business continuity.

Object Management

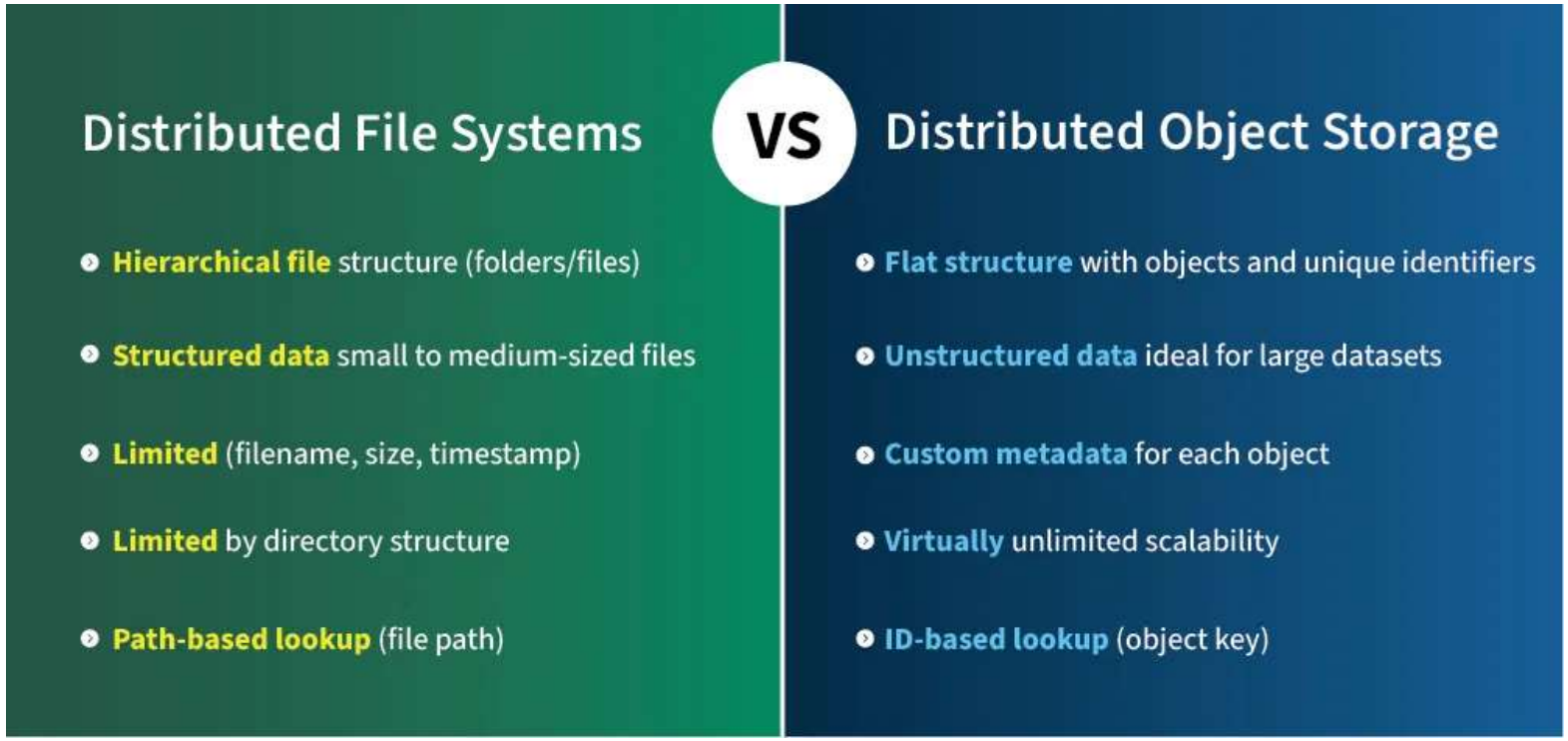
Managing objects in a distributed environment presents unique challenges.

Key aspects include:

- **Object Creation and Destruction:** Distributed objects can be created and destroyed dynamically. Object factories can be used to manage object lifecycle and ensure resources are appropriately allocated and released.
- **Object Persistence:** Distributed objects may need to maintain state over time. Object databases or serialization mechanisms can be employed to store object states persistently.
- **Object Discovery:** Object discovery mechanisms help clients locate objects across a distributed environment. This can involve naming services or registry services that track object locations.
- **Object Versioning**: Managing different versions of an object is crucial for compatibility. Versioning strategies ensure that clients can interact with the correct object version without disruption.

Distributed object storage.

- Distributed object storage is a system that stores data as objects (data with metadata) across multiple storage nodes, offering scalability, durability, and high availability for large amounts of unstructured data.
- It differs from traditional file systems by lacking a hierarchical directory structure and using a unique identifier for each object.



Key Concepts & Characteristics:

Object Storage:

Stores data as objects, which are essentially self-contained units of information consisting of data (content) and metadata (information about the data).

Distributed Architecture:

Data is spread across multiple physical servers or storage nodes, enhancing resilience and scalability.

Scalability & Performance:

Can handle massive amounts of data and can scale horizontally to meet growing storage demands.

Durability & Availability:

Data is replicated across multiple nodes, ensuring data is available and protected even if some nodes fail.

Metadata:

Object storage allows for flexible metadata, allowing for custom metadata to be attached to the object for efficient data retrieval.

What is Metadata?

Metadata is **data about data**—it describes, explains, or gives information about other data. In a **distributed database**, metadata helps manage data by defining its **structure, location, access rules, and usage details**.

Metadata (Schema Information)

Describes the **structure** of the database (tables, columns, indexes).

Metadata Type	Example
Database Name	SalesDB
Table Names	Customers, Orders, Products
Columns & Data Types	CustomerID (INT), Name (VARCHAR), TotalAmount (DECIMAL)
Primary Key	CustomerID
Foreign Key	Orders.CustomerID → Customers.CustomerID
Indexes	INDEX idx_email ON Customers(Email);

Think of Metadata Like a Library System 📚

Imagine a library. The actual books (data) are stored on shelves, but how do you find a book? You use metadata!

Book Title, Author, ISBN → Like a database table's **column names and types**

Library Catalog (Index of Books) → Like a **database index**

Books Arranged by Genre → Like **data partitioning**

Checkout History (Who borrowed a book?) → Like **audit logs**

Metadata Examples in a Distributed Database

Metadata in a distributed database is crucial for:
✅ **Finding where data is stored**
✅ **Managing replication and partitioning**
✅ **Controlling access to data**
✅ **Optimizing queries and performance**

Examples of Object Storage Services:

Amazon S3: A widely used cloud object storage service.

Google Cloud Storage: Another popular cloud object storage offering.

Microsoft Azure Blob Storage: A cloud-based object storage service from Microsoft.

Use Cases:

Cloud Storage: Ideal for storing large amounts of user data, documents, and files.

Data Archives: For storing large amounts of historical data.

Media Storage: For storing images, videos, and other media files.

Backups: As a cost-effective and scalable solution for backing up data.

Big Data Analytics: For storing and accessing data for analysis.